UNIVERSIDAD CAPITAL DE ESTUDIOS SUPERIORES

INGENIERÍA EN SISTEMAS COMPUTACIONALES

DESARROLLO E IMPLEMENTACIÓN DE UN SISTEMA WEB DE RECLUTAMIENTO DE PERSONAL PARA LA UNIVERSIDAD CAPITAL DE ESTUDIOS SUPERIORES

REPORTE DE RESIDENCIAS PROFESIONALES

[NOMBRE DEL AUTOR — PENDIENTE DE CONFIRMACIÓN]

Matrícula: [MATRÍCULA — PENDIENTE]

Asesor: [NOMBRE DEL ASESOR — PENDIENTE DE CONFIRMACIÓN]

Zamora, Michoacán — Octubre 2026

[NOTA: Portada provisional — formato sin plantilla institucional confirmada. Los datos entre corchetes requieren confirmación del autor/asesor.]

RESUMEN

El presente trabajo documenta el desarrollo e implementación de UCES Plataforma de Talento, una aplicación web de tipo MVP (*Minimum Viable Product*) orientada a digitalizar el proceso de reclutamiento y preselección de candidatos del área de Recursos Humanos de la Universidad Capital de Estudios Superiores (UCES), Zamora, Michoacán. La problemática de origen consistía en la ausencia de una etapa de evaluación estructurada previa a la entrevista presencial, lo que implicaba una carga administrativa elevada y una revisión de candidatos sin información estandarizada sobre sus competencias.

El objetivo general fue desarrollar una plataforma funcional que centralizara y digitalizara las actividades principales del proceso, incluyendo el registro de candidatos con protección criptográfica de la CURP, la aplicación de una evaluación psicométrica basada en el modelo de los Cinco Grandes factores de personalidad, la realización de evaluaciones de conocimientos con temporizador, la carga de evidencia práctica y la revisión administrativa de resultados. La metodología empleada fue el prototipado evolutivo, desarrollado en ocho iteraciones documentadas entre diciembre de 2025 y mayo de 2026, tomando como base de especificación el análisis y diseño previo elaborado por Arciniega Robles (2026) en el Trabajo A.

Las tecnologías principales utilizadas fueron Next.js 16.1.1 con App Router, TypeScript 5, React 19, Tailwind CSS, shadcn/ui, Supabase (PostgreSQL 16, Auth, Storage) y despliegue en Vercel. El modelo de datos relacional comprende 13 tablas con Row Level Security habilitado en todas ellas, y la API expone 13 Route Handlers protegidos mediante verificación de JWT y confirmación de rol en el servidor.

La validación funcional se realizó mediante 22 casos de prueba documentados: 16 casos aprobados (72.7 %), 2 aprobados parcialmente (9.1 %), 3 parciales por ausencia de endpoint REST en los componentes correspondientes (13.6 %) y 1 bloqueado por no ser reproducible en el entorno de prueba (4.5 %). Se identificaron cinco discrepancias entre el diseño del Trabajo A y la implementación resultante, todas caracterizadas y documentadas. Las principales limitaciones del MVP incluyen la ausencia de filtro automático por clasificación psicométrica, la inoperatividad del rol de revisor, la falta de notificaciones automáticas y la imposibilidad de verificar el comportamiento del sistema ante expiración de sesión prolongada.

El sistema cumple el objetivo general y cinco de los siete objetivos específicos de forma plena; los dos restantes se cumplen con matices documentados en las discrepancias D-01 y D-02. Se concluye que el MVP constituye una base técnica sólida, funcional y extensible para la gestión digitalizada del proceso de reclutamiento de UCES.

Palabras clave: plataforma web, reclutamiento universitario, prototipado evolutivo, evaluación psicométrica, Supabase, Next.js, MVP.

ABSTRACT

This work documents the development and implementation of UCES Plataforma de Talento, a Minimum Viable Product (MVP) web application designed to digitalize the recruitment and pre-screening process of the Human Resources department at the Universidad Capital de Estudios Superiores (UCES), Zamora, Michoacán, Mexico. The original problem consisted of the absence of a structured evaluation stage prior to the in-person interview, which resulted in high administrative workload and candidate review without standardized competency data.

The general objective was to develop a functional platform that centralizes and digitalizes the main activities of the process, including candidate registration with cryptographic CURP protection, administration of a psychometric assessment based on the Big Five personality model, time-limited knowledge evaluations, practical evidence uploads, and administrative review of results. The development methodology was evolutionary prototyping, carried out in eight documented iterations between December 2025 and May 2026, using as its specification baseline the analysis and design document previously produced by Arciniega Robles (2026).

The primary technologies used were Next.js 16.1.1 with App Router, TypeScript 5, React 19, Tailwind CSS, shadcn/ui, Supabase (PostgreSQL 16, Auth, Storage), and deployment on Vercel. The relational data model comprises 13 tables with Row Level Security enabled on all of them, and the API exposes 13 Route Handlers protected through server-side JWT verification and role confirmation.

Functional validation was conducted through 22 documented test cases: 16 approved (72.7%), 2 partially approved (9.1%), 3 partial due to the absence of a dedicated REST endpoint in the relevant components (13.6%), and 1 blocked due to environment constraints (4.5%). Five discrepancies between the Trabajo A design and the resulting implementation were identified, characterized, and documented. The main MVP limitations include the absence of automatic filtering by psychometric classification, the inoperative reviewer role, the lack of automated notifications, and the inability to test session timeout behavior under extended conditions.

The system fulfills the general objective and five of the seven specific objectives fully; the remaining two are fulfilled with documented caveats related to discrepancies D-01 and D-02. The MVP is concluded to constitute a solid, functional, and extensible technical foundation for the digitalized management of UCES's recruitment process.

Keywords: web platform, university recruitment, evolutionary prototyping, psychometric assessment, Supabase, Next.js, MVP.

ÍNDICE GENERAL

[ÍNDICE GENERAL — Actualizar con Word: Referencias → Tabla de contenido → Actualizar tabla]

[ÍNDICE DE FIGURAS — Insertar con: Referencias → Insertar tabla de ilustraciones]

[ÍNDICE DE TABLAS — Insertar con: Referencias → Insertar tabla de ilustraciones → Tipo: Tabla]

Capítulo I. Introducción y generalidades del proyecto

1.1 Antecedentes del proyecto

El reclutamiento y la selección de personal son actividades que inciden directamente en la calidad del trabajo dentro de cualquier organización. La capacidad de identificar candidatos adecuados para una vacante, evaluar sus competencias de manera estructurada y tomar decisiones de preselección con información organizada no es un problema exclusivo de las grandes empresas: también se presenta en instituciones educativas, organismos públicos y organizaciones de tamaño mediano que deben gestionar procesos de incorporación de personal de forma periódica.

En la Universidad Capital de Estudios Superiores (UCES), el proceso de reclutamiento del área de Recursos Humanos se desarrollaba mediante actividades principalmente manuales. Cuando surgía la necesidad de cubrir una vacante, el personal responsable elaboraba y difundía la convocatoria a través de los canales de comunicación disponibles. Los candidatos interesados enviaban su currículum vitae, y el área asumía entonces la tarea de recibirlo, revisar la información de cada aspirante y determinar quiénes serían convocados a entrevista.

Una de las dificultades más concretas de este procedimiento era la ausencia de una etapa de evaluación estructurada previa a la entrevista presencial. Sin un filtro técnico o de perfil, parte del tiempo del personal se destinaba a entrevistar candidatos que resultaban incompatibles con los requisitos de la vacante. A medida que el número de postulaciones aumentaba para un mismo cargo, la revisión individual de documentos se convertía también en una carga administrativa de gestión difícil sin herramientas específicas.

Ante esta situación surgió la propuesta de desarrollar una plataforma web que incorporara evaluaciones estructuradas como parte del flujo de reclutamiento, antes de la entrevista presencial. El planteamiento contemplaba cuatro componentes fundamentales: la publicación de vacantes organizadas por área funcional, el registro de candidatos con verificación de identidad, la aplicación de una evaluación psicométrica preliminar, y la realización de evaluaciones de conocimientos específicos vinculadas a cada vacante. El propósito era proporcionar al personal de Recursos Humanos información organizada y comparables sobre los candidatos, de forma que la entrevista pudiera concentrarse en los aspectos que los instrumentos digitales no permiten evaluar.

El proyecto se denominó UCES Plataforma de Talento y se desarrolló en el contexto de las residencias profesionales de la carrera de Ingeniería en Sistemas Computacionales. Desde su concepción fue definido como un MVP, sigla del término inglés *Minimum Viable Product*, que describe una versión funcional de un sistema que implementa los flujos esenciales sin pretender abarcar la totalidad de funcionalidades posibles. Esta delimitación fue deliberada: facilitó concentrar los esfuerzos de diseño e implementación en construir una base técnica sólida, funcional y extensible, en lugar de abordar un alcance excesivo que comprometiera la calidad del producto entregable.

El desarrollo implicó aplicar conocimientos de análisis de sistemas, diseño de bases de datos relacionales, desarrollo web con tecnologías modernas, autenticación y autorización, seguridad informática y administración de servicios en la nube. La naturaleza técnica del proyecto se sitúa, por tanto, en la intersección entre la ingeniería de software y la gestión de procesos organizacionales.

1.2 Descripción general del sistema

UCES Plataforma de Talento es una aplicación web orientada a apoyar el proceso de reclutamiento y preselección de candidatos mediante la digitalización de sus actividades principales. El sistema permite a los candidatos registrarse, completar una evaluación psicométrica, explorar vacantes disponibles, postularse a una de ellas, realizar la evaluación de conocimientos correspondiente con un temporizador activo y cargar un archivo de evidencia práctica. Por su parte, el administrador gestiona el catálogo de módulos, vacantes, evaluaciones y preguntas; revisa los intentos enviados por los candidatos; consulta el resumen psicométrico de cada postulante, y emite una decisión de aprobación o rechazo con notas opcionales. El historial de cada proceso queda disponible para el candidato en su perfil dentro de la plataforma.

El sistema está construido sobre un stack tecnológico moderno del ecosistema JavaScript. El frontend y las rutas de API se implementaron con Next.js 16.1.1, haciendo uso del App Router que permite combinar componentes del servidor (Server Components) con componentes del cliente (Client Components) según los requerimientos de cada vista. El lenguaje de programación elegido fue TypeScript, cuyo sistema de tipos estáticos permite detectar errores antes de la ejecución y documentar explícitamente las estructuras de datos que circulan por la aplicación. La interfaz de usuario se construyó con Tailwind CSS y la

biblioteca de componentes shadcn/ui, que provee elementos accesibles y estilizados basados en Radix UI.

La persistencia de datos, la autenticación y el almacenamiento de archivos se delegan a Supabase, un servicio de tipo Backend como Servicio (BaaS) que proporciona una instancia administrada de PostgreSQL, un servicio de autenticación mediante JSON Web Tokens (JWT) y un sistema de almacenamiento de objetos con control de acceso por políticas. Esta arquitectura elimina la necesidad de administrar infraestructura de servidor propia, y traslada responsabilidades operativas —copias de seguridad, escalabilidad de la base de datos, renovación de certificados TLS— al proveedor del servicio.

La seguridad del sistema opera en capas diferenciadas. A nivel de base de datos, PostgreSQL tiene habilitado el mecanismo de Row Level Security (RLS) en todas las tablas de la aplicación, con políticas que garantizan que cada usuario autenticado solo pueda acceder a sus propios registros. A nivel de API, todos los endpoints del área administrativa requieren la presentación de un token JWT válido y la verificación de que el usuario tiene el rol `admin` mediante la función de base de datos `get_my_role()`, invocada a través del mecanismo de llamada a procedimientos remotos (RPC) de Supabase. La información de identidad de los candidatos se protege adicionalmente mediante criptografía: la CURP no se almacena en texto claro, sino que se transforma con HMAC-SHA256 usando una clave de entorno antes de persistirse en la base de datos.

El módulo de evaluación psicométrica utiliza un cuestionario de veinte reactivos organizado con base en las dimensiones del modelo de los Cinco Grandes factores de personalidad. Los reactivos emplean una escala de respuesta tipo Likert de cinco niveles y están agrupados en cinco rasgos: estabilidad emocional, amabilidad, responsabilidad, apertura y extroversión. El instrumento fue proporcionado por personal de la institución y digitalizado en el sistema; sus propiedades psicométricas formales no fueron documentadas durante el desarrollo del proyecto. El sistema calcula automáticamente un resumen de resultados por rasgo al concluir la evaluación y lo almacena como dato estructurado para consulta del administrador.

La aplicación se desplegó en Vercel, plataforma con soporte nativo para proyectos Next.js, que gestiona la construcción, el despliegue continuo desde el repositorio y la administración de variables de entorno para el entorno de producción. Los datos del proyecto residen en el proyecto Supabase identificado como UcesCap.

1.3 Objetivo general

Desarrollar una plataforma web funcional que centralice y digitalice las actividades del proceso de reclutamiento y preselección de candidatos del área de Recursos Humanos de UCES, mediante la implementación de módulos de registro de candidatos, evaluación psicométrica, evaluación de conocimientos con temporizador, carga de evidencia práctica y revisión administrativa de resultados, con mecanismos de autenticación, autorización por roles y seguridad a nivel de base de datos.

1.4 Objetivos específicos

OE-01. Diseñar un modelo de datos relacional en PostgreSQL que soporte los flujos de registro de usuarios, gestión de vacantes, aplicación de evaluaciones y almacenamiento de resultados, con restricciones de integridad referencial y políticas de Row Level Security que garanticen el acceso diferenciado según el rol del usuario.

OE-02. Implementar un módulo de registro que valide el formato de la CURP mediante expresión regular, almacene únicamente su representación criptográfica mediante HMAC-SHA256 y garantice la unicidad de cada candidato en el sistema mediante restricción de clave única en la tabla de hashes.

OE-03. Desarrollar el módulo de evaluación psicométrica que permita presentar el cuestionario institucional basado en el modelo Big Five, registrar las respuestas con escala Likert, calcular automáticamente los scores por rasgo y generar un resumen estructurado persistente disponible para revisión administrativa.

OE-04. Construir el módulo de evaluación de conocimientos con temporizador regresivo configurable, guardado automático de respuestas por selección, envío automático al expirar el tiempo sin posibilidad de doble envío, carga de archivo de evidencia práctica hacia almacenamiento privado en la nube y procesamiento del intento al momento del envío.

OE-05. Implementar el panel administrativo con operaciones CRUD sobre módulos, vacantes, evaluaciones y preguntas, así como las funciones de revisión de intentos individuales, consulta del resumen psicométrico, acceso a evidencia mediante URLs firmadas y registro de decisiones de aprobación o rechazo.

OE-06. Aplicar un modelo de control de acceso con dos roles operativos diferenciados — usuario y administrador— respaldado por políticas RLS en la base de datos y verificación de

JWT en los endpoints de la API, de forma que las operaciones privilegiadas no puedan ejecutarse sin autenticación y autorización verificadas en el servidor.

OE-07. Desplegar el sistema en un entorno accesible públicamente mediante Vercel, con las variables de entorno sensibles correctamente configuradas en la plataforma y la infraestructura de Supabase vinculada al proyecto.

1.5 Justificación

El problema que motiva el desarrollo de este proyecto no es ajeno al funcionamiento de la mayoría de las organizaciones: los procesos de selección de personal son costosos en tiempo cuando carecen de una etapa de filtrado preliminar. Recibir currículum vitae, revisar su contenido manualmente y convocar a entrevista sin información previa sobre las competencias del candidato implica que la entrevista carga con una responsabilidad diagnóstica que podría haberse reducido con antelación.

La literatura de gestión de recursos humanos documenta extensamente la utilidad de las evaluaciones estructuradas en los procesos de selección. Una evaluación de conocimientos permite verificar, antes de la entrevista, que el candidato posee los fundamentos técnicos o conceptuales requeridos para la vacante. Una evaluación psicométrica puede aportar indicadores sobre dimensiones de personalidad que tienen correlatos con el desempeño en determinados contextos laborales, aunque la interpretación de estos resultados siempre debe realizarse con criterio profesional y en conjunto con otros elementos del proceso. Estas herramientas no eliminan la entrevista ni sustituyen la valoración del personal especializado; la complementan con información que de otro modo sería difícil de obtener de forma estandarizada para cada candidato.

Desde la perspectiva del desarrollo de software, el proyecto tiene un valor técnico concreto e independiente de sus resultados operativos en producción. El sistema implementa decisiones de ingeniería no triviales: el diseño de un modelo relacional de trece tablas con control de acceso a nivel de fila, la arquitectura de un módulo de evaluación con restricciones de tiempo e idempotencia ante envíos duplicados, la integración del almacenamiento de archivos con políticas de acceso granular, y la construcción de una capa de seguridad de API que verifica identidad y rol sin depender de la interfaz de usuario. Cada uno de estos componentes requirió decisiones de diseño justificadas, iteraciones de depuración documentadas en el historial del repositorio, y la integración coherente de tecnologías distintas bajo una arquitectura unificada.

El carácter de MVP del proyecto es, además, una elección técnica justificable. Construir un sistema de reclutamiento completo —con notificaciones automáticas, analítica avanzada, soporte multiinstrumento psicométrico y gestión documental— habría excedido los recursos y el tiempo disponibles en el contexto de las residencias profesionales, además de comprometer la profundidad con que podrían tratarse los aspectos centrales del sistema. La delimitación deliberada del alcance permitió entregar un producto funcional, evaluable y extensible, que cumple el propósito de digitalizar el flujo principal del proceso.

Es necesario delimitar también lo que este proyecto no pretende demostrar. No se dispone de indicadores cuantitativos sobre el proceso de reclutamiento anterior que permitan medir el impacto de la plataforma en términos de tiempo ahorrado o candidatos procesados. El sistema se presenta como un MVP funcional validado técnicamente; su evaluación en condiciones de operación institucional sostenida constituye una actividad posterior que depende de las decisiones del área de Recursos Humanos.

1.6 Alcance y limitaciones del proyecto

Funcionalidades dentro del alcance

Módulo de registro y autenticación. El sistema permite el registro de nuevos candidatos con validación del formato de CURP mediante expresión regular, almacenamiento de su representación criptográfica con HMAC-SHA256 y garantía de unicidad entre registros. La autenticación de sesión se gestiona mediante Supabase Auth con tokens JWT. El sistema protege el registro contra duplicados por CURP sin almacenar el dato en texto claro.

Módulo de evaluación psicométrica. El candidato completa un cuestionario de veinte reactivos basado en el modelo Big Five antes de poder postularse a cualquier vacante. El sistema calcula y almacena automáticamente el resumen de resultados por rasgo al momento del envío. La evaluación es obligatoria como requisito previo a la postulación y tiene un tiempo límite de diez minutos.

Módulo del candidato. Incluye la exploración de módulos y vacantes, la postulación a vacantes abiertas, la realización de la evaluación de conocimientos con temporizador, la carga de evidencia práctica hacia almacenamiento privado y la consulta del historial de intentos y decisiones en el perfil.

Panel administrativo. El administrador dispone de operaciones CRUD sobre módulos, vacantes, evaluaciones y preguntas. Puede revisar los intentos enviados, consultar las

respuestas individuales, el archivo de evidencia descargable y el resumen psicométrico de cada candidato, y registrar una decisión de aprobación o rechazo con notas opcionales.

Infraestructura de seguridad. Row Level Security habilitado en todas las tablas de la base de datos. Verificación de JWT y confirmación de rol mediante RPC en todos los endpoints del área administrativa. Almacenamiento privado de archivos con URLs firmadas de acceso temporal de una hora.

Despliegue. La aplicación está configurada para desplegarse en Vercel con gestión de variables de entorno en la plataforma y la base de datos administrada por Supabase.

Funcionalidades fuera del alcance del MVP

Restricción de postulación por clasificación psicométrica. El endpoint `POST /api/vacancies/[id]/apply` verifica únicamente que el psicométrico haya sido completado; no evalúa la clasificación obtenida (Alta, Adecuada o Riesgo) para condicionar el acceso a la postulación. La implementación de un bloqueo automático basado en el resultado del instrumento se identifica como trabajo futuro.

Interfaz administrativa para tareas prácticas. La tabla `practical_tasks` está definida en el modelo de datos y las evaluaciones de prueba incluyen ejemplos funcionales, pero no existe un formulario en el panel administrativo para administrar estas tareas. Su configuración requiere intervención directa en la base de datos mediante SQL.

Rol de revisor. El esquema original contempló un tercer rol (`reviewer`) para usuarios que pudieran revisar evaluaciones sin privilegios administrativos completos. Este rol no está implementado operativamente; las funciones de revisión están asociadas exclusivamente al rol de administrador.

Módulo de estadísticas en la interfaz. El sistema incluye un endpoint funcional `GET /api/admin/vacancies/[id]/stats` que retorna el conteo de postulaciones e intentos por vacante, pero no existe una vista en el panel que consuma estos datos. Su integración se propone como trabajo futuro.

Middleware de protección de rutas en el servidor. El proyecto no incluye un archivo `middleware.ts` de Next.js. La redirección de usuarios no autenticados se maneja mediante verificación del lado del cliente. La seguridad efectiva de los datos recae en las políticas RLS y la verificación de JWT en los endpoints de la API, que son mecanismos independientes de la navegación del cliente y no se ven afectados por esta limitación.

Notificaciones automáticas. El sistema no envía correos electrónicos ni genera notificaciones en tiempo real. Los cambios en el estado de un intento son consultables por el candidato en su perfil; la detección de actualizaciones nuevas se implementa mediante comparación con el estado almacenado localmente en el navegador.

Validación psicométrica formal. El instrumento utilizado fue proporcionado por personal de la institución. La procedencia exacta del cuestionario, las condiciones de su elaboración y las propiedades psicométricas de la adaptación específica empleada no fueron documentadas formalmente durante el desarrollo del proyecto. Los umbrales de clasificación (≥16 Alta, 11-15 Adecuada, <11 Riesgo) corresponden a criterios institucionales y no derivan de un proceso de validación psicométrica publicado.

1.7 Organización del documento

El presente reporte documenta el proceso completo de análisis, diseño, desarrollo y evaluación de UCES Plataforma de Talento. Se organiza en siete capítulos que avanzan del contexto institucional hacia los detalles técnicos de la implementación y concluyen con una reflexión sobre los resultados obtenidos.

El **Capítulo II** describe el contexto organizacional del proyecto: la institución receptora de las residencias, el área de adscripción, la problemática identificada en el proceso de reclutamiento y los requerimientos generales que orientaron el desarrollo.

El **Capítulo III** desarrolla el marco teórico. Aborda el modelo de renderizado híbrido de Next.js, el paradigma de Backend como Servicio representado por Supabase, el sistema de autenticación con JWT, el mecanismo de Row Level Security en PostgreSQL, el control de tipos con TypeScript, el modelo de componentes de interfaz con shadcn/ui y Radix UI, los fundamentos de seguridad web aplicados al proyecto, y los conceptos del modelo Big Five de personalidad que sustentan el módulo psicométrico.

El **Capítulo IV** presenta el análisis y diseño del sistema. Incluye la descripción de la metodología de desarrollo empleada, el análisis de requerimientos funcionales y no funcionales, los diagramas de casos de uso, el modelo entidad-relación completo con sus trece tablas, el diseño de la arquitectura de componentes, el diseño de seguridad y el diseño del módulo psicométrico.

El **Capítulo V** documenta el desarrollo del sistema módulo por módulo: configuración del entorno, base de datos y migraciones, módulo de autenticación, módulo del candidato,

módulo de evaluación, módulo psicométrico, panel administrativo e integración con el almacenamiento. Cada sección referencia los archivos del repositorio relevantes, cita fragmentos de código representativos y explica las decisiones de diseño adoptadas durante la construcción.

El **Capítulo VI** registra las pruebas realizadas sobre el sistema. Documenta la estrategia de verificación, los resultados de los 22 casos de prueba ejecutados sobre el sistema activo, los casos de prueba por módulo, la verificación de las restricciones de seguridad y las incidencias técnicas resueltas durante el desarrollo.

El **Capítulo VII** presenta las conclusiones del proyecto: el análisis del cumplimiento de los objetivos específicos, las competencias adquiridas durante las residencias, las limitaciones del MVP actual y las recomendaciones para etapas futuras del sistema.

El documento cierra con la sección de **Referencias bibliográficas** y los **Anexos**, que incluyen el esquema completo de la base de datos, el listado de endpoints de la API con descripción, los reactivos del cuestionario psicométrico, el manual de configuración y despliegue, un resumen de capturas de pantalla del sistema y el historial de commits relevantes del repositorio.

Capítulo II. Contexto organizacional

2.1 Descripción general de UCES y su contexto institucional

La Universidad Capital de Estudios Superiores (UCES) es una institución educativa en cuya estructura organizacional el área de Recursos Humanos desempeña funciones relacionadas con la gestión del personal y los procesos de incorporación de colaboradores. Como parte de sus actividades ordinarias, el departamento atiende requerimientos de contratación, organiza los procedimientos necesarios para identificar candidatos y administra las etapas que llevan desde la identificación de una vacante hasta la toma de una decisión de selección.

> ***[Evidencia pendiente EV-01]** La adscripción formal del presente proyecto de residencias profesionales a esta institución y al área de Recursos Humanos deberá acreditarse mediante la carta de adscripción institucional correspondiente, cuya obtención se encuentra en proceso. Hasta su incorporación al expediente, la información del contexto institucional proviene de la declaración del desarrollador responsable del proyecto y se presenta con esa atribución.*

La información disponible sitúa al área de Recursos Humanos como el ámbito de aplicación de la solución tecnológica desarrollada. Esta área fue identificada por el desarrollador como la principal beneficiaria operativa del sistema, tanto en la gestión de las evaluaciones como en la consulta de los resultados generados. La relación entre la problemática organizacional descrita en las siguientes secciones y las decisiones técnicas adoptadas durante el desarrollo del proyecto tiene su origen en el análisis de las condiciones de trabajo de este departamento.

No se dispone de información institucional formal sobre la misión, la visión, la estructura orgánica completa ni la historia de la institución. Tampoco se cuenta con estadísticas internas sobre el volumen histórico de contrataciones, el número de vacantes gestionadas por periodo o los indicadores de desempeño del área. La descripción que se presenta a continuación se limita, en consecuencia, al proceso de reclutamiento tal como fue caracterizado durante el desarrollo del proyecto, sin extrapolar información que no haya sido declarada o verificada.

2.2 Función del departamento de Recursos Humanos

Dentro de una organización educativa, el área de Recursos Humanos cumple funciones que van más allá de la administración del personal activo. Una de sus responsabilidades

centrales consiste en gestionar los procesos mediante los cuales nuevos colaboradores se identifican, evalúan y eventualmente se incorporan a la institución. Este proceso, conocido comúnmente como reclutamiento y selección, implica una secuencia de actividades que demandan tiempo del personal del área, coordinación con otras dependencias y criterios de evaluación que deben aplicarse de forma razonablemente consistente entre los distintos candidatos.

En el contexto de UCES, el área de Recursos Humanos era la responsable operativa de este proceso en su conjunto. Sus funciones incluían la definición del perfil requerido para cada vacante, la difusión de las convocatorias, la recepción y organización de las postulaciones, la revisión de los perfiles de los candidatos y la conducción de las entrevistas de selección. El resultado esperado de este proceso era identificar, entre los aspirantes que se habían postulado, a aquellos que reunían las condiciones para ocupar el cargo disponible.

La capacidad del área para cumplir estas funciones de manera eficiente está condicionada, entre otros factores, por el volumen de postulaciones recibidas y por los instrumentos disponibles para evaluarlas. Cuando el número de candidatos es reducido y el perfil del puesto es sencillo de verificar mediante el currículum vitae, el proceso manual puede ser suficiente. Sin embargo, cuando las vacantes generan un mayor número de postulantes o cuando los requisitos del puesto incluyen competencias que no se reflejan directamente en el historial declarado, la revisión individual de documentos se vuelve un cuello de botella que puede afectar tanto la velocidad como la consistencia del proceso de selección.

Esta situación no es exclusiva de UCES ni de las instituciones educativas en general. Es un desafío estructural de cualquier proceso de selección que opere sin el apoyo de herramientas de evaluación previas a la entrevista. La particularidad del contexto de este proyecto radica en que el área disponía de un instrumento psicométrico institucional que no había sido integrado al flujo digital y que tampoco existía una plataforma centralizada para gestionar las postulaciones. Ambos elementos representaron oportunidades concretas de mejora que orientaron el diseño del sistema.

2.3 Procedimiento de reclutamiento previo a la plataforma

El procedimiento de reclutamiento que el área de Recursos Humanos llevaba a cabo antes de la implementación de UCES Plataforma de Talento seguía una secuencia de pasos mayoritariamente manuales, según la descripción proporcionada por el desarrollador del proyecto a partir del análisis del proceso realizado durante la etapa de definición del sistema.

Cuando se identificaba la necesidad de cubrir una vacante, el personal responsable elaboraba la descripción del puesto con sus requisitos y características principales, y difundía la convocatoria a través de los canales de comunicación disponibles en la institución. Los candidatos interesados respondían enviando su currículum vitae por los medios habilitados para ello.

El área de Recursos Humanos recibía los documentos de cada postulante y asumía la tarea de revisarlos. Esta revisión consistía en leer la información declarada por cada aspirante, verificar si su historial correspondía al perfil solicitado y decidir cuáles candidatos serían convocados a la siguiente etapa. El criterio principal disponible en este punto del proceso era el propio currículum vitae: un documento que informa sobre la trayectoria del candidato pero que no permite verificar directamente sus conocimientos actuales ni explorar dimensiones de su perfil que podrían ser relevantes para el desempeño en el puesto.

La siguiente etapa era la entrevista presencial. En este momento el equipo del área establecía contacto directo con los candidatos preseleccionados y realizaba la conversación de evaluación. El problema central identificado en esta estructura consiste en que, hasta este punto del proceso, el área no contaba con información objetiva sobre las competencias concretas de los candidatos. La entrevista absorbía entonces una doble función: conocer a la persona y verificar sus conocimientos o aptitudes, dos objetivos que requieren enfoques distintos y que compiten por el tiempo disponible en una sola sesión.

Esta dinámica generaba una consecuencia práctica frecuente en los procesos de selección sin filtro técnico previo: el personal de Recursos Humanos invertía tiempo en entrevistar a candidatos que, al ser evaluados con mayor detalle, resultaban incompatibles con los requisitos del puesto. No porque la revisión inicial fuera descuidada, sino porque los instrumentos disponibles en esa etapa —fundamentalmente el currículum vitae— no permitían identificar esa incompatibilidad antes del contacto directo.

Un segundo aspecto del proceso que representaba una oportunidad de mejora era la organización y el seguimiento de las postulaciones. Sin un sistema de registro centralizado, el estado de cada candidato —quién había sido contactado, cuál era el resultado preliminar de la revisión, qué observaciones había dejado el evaluador— dependía de los registros propios del personal responsable. Esta situación puede derivar en inconsistencias cuando varias personas participan en el proceso o cuando el seguimiento de múltiples vacantes activas se realiza simultáneamente.

2.4 Identificación de necesidades y oportunidades de mejora

El análisis del procedimiento anterior permite identificar necesidades concretas que orientaron el diseño de UCES Plataforma de Talento. Estas necesidades no surgen de un diagnóstico formal documentado, sino del análisis funcional del proceso realizado durante la etapa de definición del sistema, y se presentan como la base que justifica las decisiones de diseño adoptadas.

La primera necesidad identificada es contar con un mecanismo de evaluación preliminar que aporte información objetiva antes de la entrevista presencial. Si los candidatos completan una evaluación de conocimientos vinculada a la vacante específica a la que se postulan, el personal de Recursos Humanos recibe datos comparables que complementan el currículum vitae. Esta información permite priorizar a quienes cumplen los requisitos mínimos del puesto, identificar con mayor precisión a los candidatos que ameritan ser convocados a entrevista, y preparar esa conversación con elementos más específicos sobre cada perfil.

La segunda necesidad es la incorporación de la evaluación psicométrica al flujo digital de reclutamiento. El área de Recursos Humanos disponía de un cuestionario institucional basado en dimensiones del modelo de los Cinco Grandes factores de personalidad, proporcionado por personal de la institución. **[EV-06]** *La incorporación de este instrumento al expediente del proyecto requiere obtener el cuestionario original para documentar su procedencia.* Sin embargo, la existencia del instrumento y su uso como parte del proceso de evaluación fue declarado por el desarrollador como información de contexto. Su digitalización y automatización dentro de la plataforma representaba la oportunidad de generar, de forma sistemática, un perfil psicométrico de cada candidato accesible para el evaluador al momento de la revisión.

La tercera necesidad es la centralización del registro y el seguimiento de postulaciones. Un sistema donde cada candidato tiene un perfil registrado, cada postulación tiene un estado actualizable y cada decisión del administrador queda vinculada a un registro específico, reduce la dependencia del seguimiento informal y facilita la consulta del historial del proceso. Esta centralización es particularmente útil cuando el área gestiona varias vacantes simultáneamente o cuando más de una persona del equipo participa en la revisión de los candidatos.

Estas tres necesidades están interrelacionadas. Una evaluación de conocimientos es más útil cuando su resultado aparece junto al perfil psicométrico del mismo candidato y ambos

están asociados a la vacante correspondiente. El seguimiento centralizado pierde valor si no hay datos estructurados que registrar. Por esa razón, la propuesta no consistió en digitalizar un instrumento de forma aislada, sino en construir un flujo completo que integrara registro, evaluación y revisión administrativa en una misma plataforma.

2.5 Participación de los actores institucionales y técnicos

En el desarrollo de UCES Plataforma de Talento intervinieron actores con roles distintos. Identificar la contribución específica de cada uno es importante tanto para la comprensión del proyecto como para distinguir con claridad la información que proviene de declaraciones del desarrollador de aquella que cuenta con respaldo documental.

El área de Recursos Humanos de UCES participó como el ámbito organizacional de referencia del proyecto. La descripción de su proceso de reclutamiento, la identificación de sus necesidades y la definición inicial de los requerimientos funcionales del sistema derivaron de la relación entre el desarrollador y el personal del área durante la etapa de análisis. Adicionalmente, el personal del área proporcionó el cuestionario institucional que sirvió de base para el módulo psicométrico, según declaración del desarrollador. **[EV-01, EV-06]** *La documentación formal de esta participación, incluyendo la carta de adscripción de las residencias y el cuestionario original, se encuentra en proceso de obtención y deberá incorporarse al expediente antes de la defensa del reporte.*

El desarrollador, en su calidad de estudiante de residencias profesionales de Ingeniería en Sistemas Computacionales, participó como responsable técnico del proyecto. Sus funciones comprendieron el análisis de los requerimientos organizacionales, el diseño del modelo de datos, la implementación de todos los módulos del sistema, la configuración del entorno de despliegue y la elaboración del presente reporte. La residencia profesional constituyó el marco institucional bajo el cual se realizó este trabajo.

La asesoría externa al proyecto correspondió al área institucional receptora, que orientó el proceso en los aspectos relacionados con el reclutamiento y los requerimientos funcionales del sistema. La asesoría interna, vinculada al seguimiento académico del proceso de residencias, está a cargo del personal docente del programa. **[EV-01]** *Los datos formales de los asesores —nombres, cargos y firmas— se incorporarán en la página de firmas del documento una vez disponible la carta de adscripción.*

2.6 Propuesta de transformación del proceso de reclutamiento

UCES Plataforma de Talento propone incorporar etapas digitales de evaluación en el flujo de reclutamiento, antes de la entrevista presencial, de modo que el personal de Recursos Humanos disponga de información organizada sobre cada candidato cuando llegue el momento de revisarlos y tomar decisiones.

La transformación que el sistema introduce en el proceso puede describirse comparando la secuencia anterior con la secuencia propuesta. En el proceso anterior, el flujo era: difundir vacante → recibir currículum → revisar manualmente → entrevistar → decidir. En el proceso propuesto con la plataforma, el flujo se convierte en: publicar vacante en el sistema → candidato se registra y completa psicométrico → candidato se postula → candidato realiza evaluación de conocimientos → administrador revisa resultados estructurados → administrador registra decisión.

El cambio sustantivo no es la eliminación de la entrevista, que permanece como actividad del área de Recursos Humanos, sino la incorporación de dos instrumentos de evaluación —psicométrico y de conocimientos— que generan información estructurada antes de que el evaluador humano intervenga. Cuando el administrador accede al panel de revisión, dispone del resultado de la evaluación de conocimientos del candidato, con sus respuestas individuales y el indicador de aciertos; del archivo de evidencia práctica que el candidato adjuntó; y del resumen psicométrico organizado por rasgo con clasificación.

Esta información no sustituye el juicio del evaluador ni constituye por sí sola una decisión de selección. Su valor consiste en enfocar la atención del personal de Recursos Humanos en los aspectos más relevantes de cada perfil y en permitirle preparar la entrevista con elementos más específicos. Un candidato con buenas respuestas en la evaluación técnica pero con notas bajas en algún rasgo del psicométrico puede merecer una exploración particular en ese aspecto durante la entrevista; un candidato con perfil psicométrico favorable pero con resultados bajos en la evaluación de conocimientos puede requerir aclaraciones sobre su experiencia práctica. La plataforma no interpreta estos datos: los presenta y deja la interpretación al criterio del evaluador.

Es importante precisar que el MVP desarrollado no implementa ningún mecanismo de descarte automático de candidatos basado en su clasificación psicométrica. El requisito vigente en el sistema es que la evaluación psicométrica haya sido completada antes de postularse; el contenido del resultado no condiciona automáticamente el acceso del

candidato a las vacantes disponibles. Esta decisión de diseño refleja la postura de que la información psicométrica debe ser un insumo para el juicio profesional, no un filtro automático sin supervisión humana.

2.7 Alcance organizacional y consideraciones de implementación

El alcance organizacional del sistema se circunscribe al proceso de preselección de candidatos: las etapas que van desde la publicación de la vacante hasta la revisión de los resultados de evaluación y el registro de una decisión preliminar por parte del administrador. Las etapas posteriores del proceso de selección —entrevistas presenciales, verificación de referencias, contratación formal y trámites administrativos asociados— no están contempladas en la plataforma y permanecen como actividades del área de Recursos Humanos.

Esta delimitación tiene implicaciones prácticas. La plataforma no es un sistema de gestión de recursos humanos completo: no administra contratos, no gestiona nóminas, no registra desempeño del personal activo ni integra información de otras áreas de la institución. Es una herramienta específica para el proceso de preselección, construida con el propósito declarado de digitalizar las etapas de evaluación y organizar la información que el área necesita para tomar decisiones informadas antes de la entrevista.

Desde el punto de vista de la implementación organizacional, el funcionamiento efectivo del sistema requiere que se cumplan algunas condiciones que van más allá de la disponibilidad técnica de la plataforma. Los candidatos deben contar con acceso a un dispositivo con conexión a internet para completar su registro y sus evaluaciones. El personal administrativo debe conocer el panel de gestión para crear vacantes, configurar evaluaciones, revisar los intentos enviados y registrar decisiones. La configuración inicial de las evaluaciones —preguntas, opciones de respuesta, tiempo límite— debe realizarse por el administrador antes de que la vacante esté disponible para los postulantes.

Ninguno de estos requerimientos es inusual en el contexto de una institución educativa con infraestructura tecnológica básica, pero su cumplimiento es una condición necesaria para la operación del sistema. Adicionalmente, la transición de un proceso manual a un proceso con apoyo digital requiere un periodo de adaptación. Las primeras operaciones del sistema con usuarios reales generarán aprendizajes sobre aspectos que no son evidentes en la fase de desarrollo: la claridad de las instrucciones para los candidatos, la usabilidad del panel de

administración en condiciones de uso real, la pertinencia de las preguntas configuradas para cada vacante.

Una consideración adicional relevante tiene que ver con los datos gestionados por el sistema. La plataforma registra información de identidad, respuestas a evaluaciones y perfiles psicométricos de los candidatos. El tratamiento responsable de esta información exige que el área institucional establezca criterios claros sobre su uso, los periodos de retención de los datos y las condiciones de acceso por parte del personal. Desde el punto de vista técnico, el sistema implementa controles de acceso y mecanismos de seguridad apropiados para el alcance del MVP; sin embargo, la definición de las políticas institucionales de gestión de datos personales es una responsabilidad que corresponde al área receptora, no al sistema mismo.

2.8 Conclusiones del capítulo

El análisis del contexto organizacional de UCES revela un proceso de reclutamiento con una estructura funcional que operaba sin herramientas digitales de evaluación previas a la entrevista. Las consecuencias de esta estructura —tiempo invertido en entrevistar candidatos sin información previa sobre sus competencias y seguimiento informal del estado de las postulaciones— configuraron las necesidades concretas que orientaron el diseño del sistema.

Las tres necesidades identificadas en la sección 2.4 —evaluación de conocimientos preliminar, evaluación psicométrica integrada al flujo digital, y centralización del registro de postulaciones— tienen una correspondencia directa con los módulos implementados en UCES Plataforma de Talento. El módulo de evaluación con temporizador responde a la primera necesidad; el módulo psicométrico responde a la segunda; la estructura de postulaciones, intentos y panel administrativo responde a la tercera. Esta correspondencia no es casual: refleja que las decisiones de diseño del sistema partieron de un análisis del proceso que se buscaba apoyar, no de una selección arbitraria de funcionalidades.

La propuesta de transformación descrita en la sección 2.6 no elimina la función del evaluador humano ni pretende que la tecnología tome decisiones de selección. Introduce etapas de evaluación estructurada que generan información útil y comparable para quien finalmente emite la decisión. Este posicionamiento es coherente con el alcance declarado del MVP y con las limitaciones reconocidas del instrumento psicométrico utilizado, cuyas propiedades formales no han sido documentadas.

Los capítulos siguientes describirán las tecnologías empleadas para construir esta solución, las decisiones de diseño adoptadas en el proceso de análisis y la implementación técnica de cada uno de los módulos que componen el sistema. La comprensión del contexto organizacional presentado en este capítulo es el punto de referencia desde el cual esas decisiones técnicas adquieren su sentido.

Capítulo III. Marco teórico y fundamentos tecnológicos

3.1 Ingeniería de software y desarrollo de aplicaciones web

La ingeniería de software es la disciplina que aplica principios de la ingeniería al proceso de construcción, mantenimiento y evolución de sistemas de software. A diferencia de la programación entendida como actividad individual, la ingeniería de software aborda el desarrollo como un proceso sistemático que incluye análisis de requisitos, diseño, implementación, verificación y mantenimiento (Sommerville, 2016). Su propósito es producir software que satisfaga los requisitos del cliente con niveles razonables de calidad, dentro de restricciones reales de tiempo y recursos.

Uno de los conceptos centrales de la disciplina es el ciclo de vida del software, que describe las fases por las que atraviesa un sistema desde su concepción hasta su retiro. La forma en que estas fases se organizan y se relacionan entre sí define la metodología de desarrollo. Algunas metodologías plantean un recorrido lineal por etapas bien delimitadas; otras proponen ciclos iterativos en los que las versiones parciales del sistema se construyen, evalúan y refinan sucesivamente. La elección de metodología influye de manera directa en cómo se administra la incertidumbre sobre los requisitos, cómo se gestiona la retroalimentación del cliente y qué tipo de artefactos se producen en cada etapa.

El desarrollo de aplicaciones web hereda estos principios y los aplica a un contexto particular: sistemas que se ejecutan en un servidor accesible a través de la red y cuya interfaz se presenta en un navegador web. Este tipo de sistemas comparte con cualquier software de propósito general la necesidad de modelar correctamente el dominio del problema, organizar la lógica en componentes coherentes y manejar adecuadamente los estados de la información. La diferencia radica en que también deben considerar la latencia de red, la naturaleza sin estado del protocolo HTTP, la ejecución de código en entornos distintos —cliente y servidor— y los mecanismos específicos de autenticación y autorización que este modelo de despliegue impone.

En el contexto de UCES Plataforma de Talento, la ingeniería de software se concretó en un proceso de transformación: el área de Recursos Humanos tenía un conjunto de necesidades identificadas —evaluación psicométrica, evaluación de conocimientos, panel de revisión— que el Trabajo A formalizó como requisitos funcionales y casos de uso. El Trabajo B tomó esos artefactos como punto de partida para construir el sistema, validar cada funcionalidad contra los requisitos documentados y registrar las diferencias que surgieron durante la

implementación. Esta separación entre análisis y desarrollo no es solo una decisión académica: refleja una práctica reconocida en proyectos de software donde distintos equipos o etapas se hacen responsables de distintas fases del ciclo de vida.

3.2 Arquitectura de aplicaciones web modernas

Una aplicación web es un sistema distribuido: parte de su lógica se ejecuta en un servidor y parte en el navegador del usuario. La comunicación entre ambos extremos ocurre a través del protocolo HTTP (Hypertext Transfer Protocol) o su variante cifrada HTTPS, que funciona bajo un modelo de solicitud y respuesta: el cliente envía una solicitud a una URL determinada y el servidor devuelve una respuesta con un código de estado y, opcionalmente, un cuerpo de datos (Fielding et al., 1999).

La arquitectura más extendida para aplicaciones web modernas sigue el patrón cliente-servidor con separación de responsabilidades. El servidor expone una interfaz de programación de aplicaciones (API) que acepta solicitudes en formatos estructurados —habitualmente JSON— y devuelve respuestas en el mismo formato. El cliente, ejecutándose en el navegador, consume esta API para obtener datos y actualizar la interfaz sin necesidad de recargar la página completa. Este modelo, conocido como aplicación de página única (SPA, por sus siglas en inglés), mejora la fluidez de la experiencia de usuario pero traslada responsabilidades de renderizado al navegador, lo que puede afectar el tiempo de carga inicial y el posicionamiento en motores de búsqueda.

Una tendencia más reciente en el desarrollo web, impulsada principalmente por Next.js y frameworks similares, combina el renderizado del lado del servidor (SSR) con el del lado del cliente (CSR) en función de las características de cada vista. Las páginas o componentes que requieren datos que no cambian con frecuencia o que deben estar disponibles para motores de búsqueda pueden generarse en el servidor antes de enviarse al navegador. Las partes interactivas de la interfaz —formularios, actualizaciones en tiempo real, componentes que dependen de la sesión del usuario— se ejecutan como código cliente. Este modelo híbrido busca equilibrar el rendimiento percibido por el usuario, los tiempos de carga y la complejidad de la lógica del sistema.

La seguridad en aplicaciones web modernas opera en capas superpuestas. Las conexiones se cifran mediante TLS para proteger los datos en tránsito. La autenticación verifica la identidad de quien hace la solicitud. La autorización determina si esa identidad tiene permiso para realizar la operación solicitada. Las restricciones a los datos se aplican tanto en la capa

de API como en la capa de base de datos para evitar que una vulnerabilidad en un punto exponga información de toda la aplicación. Esta arquitectura de seguridad en profundidad es la que se empleó en UCES Plataforma de Talento, como se describirá en detalle en el Capítulo V.

3.3 Next.js, React y TypeScript

3.3.1 React y el modelo de componentes

React es una biblioteca de JavaScript para construir interfaces de usuario mediante componentes (Meta Open Source, 2024a). Un componente es una unidad de interfaz autocontenida: encapsula su estructura visual, su estado interno y su lógica de presentación en una función o clase que recibe propiedades externas (*props*) y devuelve una descripción de la interfaz. Esta descripción se escribe en JSX, una extensión de la sintaxis de JavaScript que permite expresar estructuras similares a HTML dentro del código.

El modelo reactivo de React significa que el componente vuelve a renderizarse automáticamente cuando cambian sus propiedades o su estado. Esta característica simplifica la gestión de interfaces que deben actualizarse en respuesta a acciones del usuario o a datos recibidos de un servidor, sin necesidad de manipular el DOM directamente. React 19, la versión utilizada en UCES Plataforma de Talento, incorporó mejoras en la gestión de formularios y en el manejo de acciones asíncronas, reduciendo el código repetitivo en operaciones comunes como enviar formularios y mostrar estados de carga (Meta Open Source, 2024b).

3.3.2 Next.js y el App Router

Next.js es un framework para React que añade al modelo de componentes capacidades de renderizado del lado del servidor, generación estática, enrutamiento basado en el sistema de archivos y manejo de solicitudes HTTP mediante Route Handlers (Vercel, 2024a). En su arquitectura App Router, introducida en la versión 13 y consolidada en versiones posteriores, los archivos bajo el directorio `app/` definen tanto las rutas de la aplicación como el tipo de renderizado que aplica a cada una.

El App Router distingue dos tipos de componentes según dónde se ejecutan. Los **Server Components** son el tipo predeterminado: se ejecutan únicamente en el servidor, pueden acceder directamente a bases de datos o servicios sin exponer credenciales al cliente, y sus datos se serializan y se envían al navegador como HTML. Los **Client Components**,

marcados con la directiva `"use client"` al inicio del archivo, se envían al navegador y pueden usar los *hooks* de React —como `useState` o `useEffect`— para gestionar estado local e interacciones del usuario. Esta distinción permite que las vistas con datos sensibles o que requieren consultas a la base de datos se procesen en el servidor, mientras que las partes interactivas se hidratan en el cliente.

En UCES Plataforma de Talento, los Route Handlers —archivos `route.ts` ubicados dentro de `app/api/`— actúan como los endpoints de la API. Cada archivo exporta funciones con los nombres de los métodos HTTP que maneja (`GET`, `POST`, `PATCH`, `DELETE`). Estas funciones reciben un objeto `Request` estándar de la Web API y devuelven un objeto `Response` o una respuesta estructurada mediante el helper `NextResponse` de Next.js. Este modelo sigue la especificación estándar de la plataforma web, lo que lo hace compatible con los mecanismos de seguridad y validación que aplican a cualquier API HTTP.

3.3.3 TypeScript y el tipado estático

TypeScript es un superconjunto tipado de JavaScript desarrollado por Microsoft (Microsoft, 2024). Añade al lenguaje un sistema de tipos estáticos que permite declarar la forma de los objetos, los parámetros de las funciones y los valores de retorno, y que el compilador verifica antes de ejecutar el código. Los tipos no existen en tiempo de ejecución —TypeScript se transpila a JavaScript puro— pero su verificación en tiempo de compilación elimina una categoría amplia de errores que de otro modo solo aparecerían durante la ejecución.

En un proyecto full-stack donde el mismo lenguaje se usa para la lógica del cliente, los Route Handlers y la definición de estructuras de datos, TypeScript facilita la coherencia entre las distintas capas. Cuando el código de un Route Handler especifica que devuelve un objeto con campos `theoryScore: number` y `summary: PsicometricoSummary`, el componente de React que consume esa respuesta puede declarar que espera exactamente esa forma. Si en algún momento el Route Handler cambia sin actualizar el tipo, el compilador señala la discrepancia antes de que el error llegue al usuario.

En UCES Plataforma de Talento, TypeScript se usó en toda la aplicación, incluyendo las rutas de API, las funciones de biblioteca (`lib/`) y los componentes de interfaz. El tipo `PsicometricoSummary` definido en `lib/psicometricoResults.ts` es un ejemplo de cómo TypeScript documenta las estructuras de datos del sistema: especifica los cinco rasgos del instrumento psicométrico, los campos de cada resultado de rasgo y los totales de

respuestas, de forma que cualquier consumidor de esa función conoce exactamente qué puede esperar.

3.4 Supabase y PostgreSQL

3.4.1 Backend como Servicio

Un Backend como Servicio (BaaS) es un modelo de plataforma que proporciona infraestructura de servidor —base de datos, autenticación, almacenamiento, en ocasiones funciones computacionales— como un servicio administrado accesible a través de APIs (Supabase, 2024a). A diferencia de construir y administrar un servidor propio, un BaaS traslada responsabilidades operativas como la configuración del sistema operativo, la administración de la base de datos, los respaldos automáticos y la disponibilidad del servicio al proveedor. El desarrollador interactúa con la plataforma a través de SDKs y APIs documentadas.

Supabase es un BaaS de código abierto construido sobre PostgreSQL (Supabase, 2024a). Provee una instancia administrada del gestor de base de datos, un sistema de autenticación basado en JWT, un servicio de almacenamiento de objetos compatible con la API de Amazon S3, y un motor de funciones de borde (*Edge Functions*). La capa de API que expone Supabase hacia los clientes es generada automáticamente a partir del esquema de la base de datos mediante PostgREST, una herramienta que transforma las tablas y vistas de PostgreSQL en endpoints REST (Supabase, 2024b).

En UCES Plataforma de Talento, Supabase cumple tres funciones diferenciadas: gestionar la autenticación de usuarios, persistir todos los datos de la aplicación en PostgreSQL y almacenar los archivos de evidencia cargados por los candidatos. El SDK `@supabase/supabase-js` proporciona el cliente que el código de la aplicación usa para interactuar con estas tres capas desde el entorno del navegador y desde los Route Handlers del servidor.

3.4.2 PostgreSQL como sistema gestor de bases de datos relacionales

PostgreSQL es un sistema gestor de bases de datos objeto-relacional de código abierto con más de 35 años de desarrollo activo (The PostgreSQL Global Development Group, 2024). Implementa el estándar SQL con extensiones propias y es conocido por su robustez, su soporte de tipos de datos avanzados —incluyendo JSON, arrays y tipos geométricos— y su

mecanismo de extensibilidad que permite añadir funciones, operadores y tipos de datos definidos por el usuario.

El modelo relacional organiza los datos en tablas compuestas por filas y columnas. Cada tabla representa una entidad del dominio; las relaciones entre entidades se expresan mediante claves foráneas (*foreign keys*) que garantizan la integridad referencial: no puede existir un registro hijo sin su registro padre correspondiente. Esta propiedad es fundamental en sistemas donde los datos están interconectados de forma compleja. En UCES Plataforma de Talento, la tabla `attempts` tiene una clave foránea hacia `applications`, que a su vez apunta a `profiles` y `vacancies`. Esta cadena de referencias garantiza que un intento de evaluación no puede existir sin una postulación válida, y que una postulación no puede existir sin un usuario y una vacante registrados.

El esquema de la aplicación, definido en `supabase/schema.sql`, declara 13 tablas con sus relaciones, tipos de datos y restricciones. La tabla `attempts`, por ejemplo, incluye columnas `started_at` y `deadline_at` de tipo `timestamptz` para registrar los tiempos con zona horaria, un campo `theory_score` de tipo `numeric` para el resultado calculado, y una columna `psychometric_summary` de tipo `jsonb` para almacenar el resumen estructurado del instrumento psicométrico. El tipo JSONB de PostgreSQL almacena datos JSON en formato binario, permitiendo realizar consultas sobre sus campos, indexarlos y utilizarlos en condiciones de filtrado, con mejor rendimiento que el tipo JSON textual equivalente.

3.4.3 Supabase Auth

Supabase Auth es el servicio de autenticación de la plataforma (Supabase, 2024c). Gestiona el registro de usuarios, la verificación de credenciales, la emisión de tokens JWT y su renovación automática. El servicio mantiene su propia tabla interna `auth.users`, separada de las tablas de la aplicación, y provee funciones de base de datos —como `auth.uid()`— que pueden invocarse desde las políticas de seguridad para identificar al usuario autenticado en el contexto de una consulta SQL.

Cuando un usuario se autentica con correo y contraseña, Supabase Auth verifica las credenciales, genera un par de tokens —un token de acceso de corta duración y un token de renovación (*refresh token*)— y los entrega al cliente. El token de acceso es un JWT firmado con la clave secreta del proyecto que incluye, entre otros datos, el identificador del usuario (`sub`), el momento de expiración y el rol configurado en Supabase. El cliente adjunta este

token en el encabezado `Authorization` de las solicitudes subsecuentes para que el servidor pueda verificar la identidad sin consultar la base de datos en cada petición.

En UCES Plataforma de Talento, el cliente de navegador se instancia en `lib/supabaseClient.ts` con la clave anónima pública (`NEXT_PUBLIC_SUPABASE_ANON_KEY`), que permite operaciones de lectura sujetas a las políticas de RLS y la autenticación estándar. El cliente de servidor, instanciado en `lib/supabaseServer.ts` con la clave de rol de servicio (`SUPABASE_SERVICE_ROLE_KEY`), omite las políticas de RLS y se usa exclusivamente en los Route Handlers donde la lógica de negocio requiere acceso administrativo. Esta separación evita que la clave de servicio —que otorga acceso irrestricto a la base de datos— sea expuesta al navegador.

3.4.4 Supabase Storage

Supabase Storage es el servicio de almacenamiento de objetos de la plataforma (Supabase, 2024d). Los archivos se organizan en *buckets*, contenedores lógicos análogos a directorios de alto nivel. Cada bucket puede configurarse como público —sus URLs son accesibles sin autenticación— o privado —el acceso requiere autenticación y se sujeta a políticas de seguridad.

En UCES Plataforma de Talento, los archivos de evidencia —currículos, diplomas u otros documentos que los candidatos cargan durante la evaluación— se almacenan en el bucket `evidences`, configurado como privado. Para que un usuario o un administrador pueda acceder a un archivo, el sistema genera una URL firmada (*signed URL*) con tiempo de expiración mediante la API de Supabase Storage. Este mecanismo garantiza que los archivos no sean accesibles mediante URLs directas predecibles y que solo las partes autorizadas puedan obtener acceso temporal a ellos.

3.4.5 Row Level Security

Row Level Security (RLS) es una funcionalidad de PostgreSQL que permite definir políticas de acceso a nivel de fila dentro de una tabla (The PostgreSQL Global Development Group, 2024). Cuando RLS está habilitado en una tabla, cada operación de lectura, escritura o eliminación sobre esa tabla es evaluada contra las políticas definidas antes de ejecutarse. Una política especifica, para cada tipo de operación y rol de base de datos, una condición booleana que debe satisfacerse para que la operación sea permitida.

La ventaja de RLS frente a la validación exclusiva en la capa de aplicación es que las restricciones de acceso residen en la base de datos misma, no en el código que la consulta. Esto significa que si un fallo en la lógica de la aplicación omitiera una verificación de autorización, las políticas de RLS actuarían como segunda barrera de protección. Es el principio de defensa en profundidad aplicado al modelo de datos.

En UCES Plataforma de Talento, RLS está habilitado en todas las tablas de la aplicación. El archivo `supabase/rls.sql` define las políticas. Por ejemplo, la política `users-manage-own-attempts` sobre la tabla `attempts` utiliza una subconsulta que verifica si el `user_id` de la aplicación asociada al intento coincide con `auth.uid()`, la función que devuelve el identificador del usuario autenticado en el contexto de la sesión de base de datos. Esto garantiza que un usuario solo pueda leer o modificar los intentos que le pertenecen, independientemente de cómo se construya la consulta desde el código de la aplicación.

La función `get_my_role()`, definida también en `supabase/rls.sql`, es una función almacenada con la cláusula `SECURITY DEFINER`. Esto significa que se ejecuta con los permisos del usuario que la definió —en este caso, el rol administrador de la base de datos— en lugar de con los permisos del usuario que la invoca. Esta propiedad es necesaria porque la función necesita acceder a la tabla `user_roles` para determinar el rol del usuario actual, y esa misma tabla tiene una política de RLS que podría generar una recursión si la función se ejecutara en el contexto de seguridad del usuario. Al usar `SECURITY DEFINER`, la función puede consultar `user_roles` directamente sin activar sus propias políticas, y devolver únicamente el rol del usuario autenticado, sin exponer información de otros usuarios.

3.5 Autenticación y autorización

La autenticación y la autorización son dos conceptos de seguridad relacionados pero distintos (OWASP Foundation, 2021). La **autenticación** responde a la pregunta "¿quién eres?": es el proceso mediante el cual un sistema verifica la identidad de quien solicita acceso. La **autorización** responde a "¿qué puedes hacer?": una vez que la identidad ha sido establecida, determina qué operaciones tiene permitido realizar esa identidad.

En sistemas web, la autenticación tradicional se basa en credenciales —combinación de nombre de usuario y contraseña— que el usuario presenta al servidor. Si las credenciales son válidas, el servidor emite algún tipo de prueba de identidad que el cliente puede presentar en solicitudes posteriores sin necesidad de repetir las credenciales en cada una.

Esta prueba puede tomar la forma de una cookie de sesión almacenada en el servidor, o de un token que el cliente conserva y adjunta en cada solicitud.

3.5.1 JSON Web Tokens

Un JSON Web Token (JWT) es un estándar abierto (RFC 7519) para representar afirmaciones (*claims*) de forma segura entre dos partes (Jones, Bradley & Sakimura, 2015). Un JWT está compuesto por tres segmentos codificados en Base64URL y separados por puntos: el encabezado, que especifica el algoritmo de firma; la carga útil (*payload*), que contiene las afirmaciones; y la firma, que garantiza la integridad del token.

Las afirmaciones son pares clave-valor. Algunas están estandarizadas: `sub` identifica al sujeto del token, `exp` indica el momento de expiración, `iat` registra el momento de emisión. Otras son específicas de la aplicación. La firma se produce aplicando el algoritmo especificado en el encabezado sobre la concatenación del encabezado codificado y la carga útil codificada, usando una clave secreta. Quien recibe el token puede verificar que no fue modificado recalculando la firma con la misma clave y comparando el resultado.

La propiedad clave de un JWT es que es **autocontenido**: el servidor puede verificar la identidad y las afirmaciones del portador sin consultar una base de datos, únicamente verificando la firma criptográfica. Esto lo hace eficiente para sistemas distribuidos y APIs sin estado. La contrapartida es que un JWT válido no puede revocarse antes de su expiración sin mecanismos adicionales —como listas de revocación o validación contra la base de datos—, por lo que los tiempos de expiración cortos (típicamente 1 hora) son una práctica de seguridad recomendada.

En UCES Plataforma de Talento, Supabase Auth emite JWTs al autenticar a un usuario. Los Route Handlers de administración reciben este token en el encabezado `Authorization` y lo verifican mediante la función `requireAdmin()` en `app/api/admin/utils.ts`. Esta función extrae el token del encabezado, llama a `supabase.auth.getUser(token)` para verificar su validez contra la clave del proyecto, y posteriormente invoca el RPC `get_my_role()` —usando un cliente instanciado con las credenciales del usuario, no las del servidor— para confirmar que el rol devuelto por la base de datos es `admin`. Este diseño aplica verificación tanto a nivel del token como a nivel de datos de rol.

3.5.2 Control de acceso basado en roles



Figura 5.4. Vista de la página de inicio pública del sistema (/). Fuente: captura de pantalla propia, 2026.

El control de acceso basado en roles (RBAC, *Role-Based Access Control*) es un modelo de autorización en el que los permisos se asignan a roles, y los roles se asignan a usuarios (Ferraiolo & Kuhn, 1992). En lugar de definir individualmente qué puede hacer cada usuario, el administrador define las capacidades de cada rol y asigna a cada usuario el rol correspondiente a sus funciones. Cuando los permisos de un rol cambian, el cambio se propaga automáticamente a todos los usuarios que lo tienen asignado.

En UCES Plataforma de Talento, el modelo de roles es sencillo: existen dos roles relevantes en la lógica de la aplicación, `user` (candidato) y `admin` (administrador). La tabla `user_roles` almacena la asignación de un rol a cada usuario identificado por su UUID. La función `get_my_role()` implementa una lógica de resolución con dos niveles: primero consulta `user_roles` y, si no encuentra una entrada, cae hacia la columna `role` de la tabla `profiles`. Este mecanismo de respaldo garantiza que el sistema funcione incluso si la entrada en `user_roles` no se insertó correctamente, devolviendo el rol `user` como valor predeterminado en ausencia de información.

La función `resolveRole()` en `lib/auth.ts` aplica una validación adicional sobre el valor devuelto: acepta únicamente las cadenas `"admin"` o `"user"` como roles válidos y retorna

`"user"` ante cualquier otro valor. Esto previene que valores inesperados en la base de datos sean aceptados como roles con permisos elevados.

3.6 Protección de identificadores personales mediante HMAC

La Clave Única de Registro de Población (CURP) es un identificador alfanumérico de 18 caracteres asignado a cada persona en México por el Registro Nacional de Población (SEGOB, 2024). Contiene información derivada del nombre completo, fecha de nacimiento, sexo y entidad federativa de nacimiento. Por su naturaleza, la CURP es un dato personal que, si se expone o se asocia indebidamente con otro dato, puede usarse para identificar a una persona de forma inequívoca.

La Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP) y su reglamento establecen que los datos personales deben protegerse mediante medidas técnicas adecuadas al riesgo que representa su exposición (Congreso de la Unión, 2010). Una de las medidas técnicas reconocidas es el uso de funciones criptográficas para transformar los datos de forma que no puedan ser recuperados por terceros no autorizados.

3.6.1 Funciones hash criptográficas y HMAC

Una función hash criptográfica es una función matemática que transforma una entrada de longitud arbitraria en una salida de longitud fija —el *digest* o resumen— con las siguientes propiedades: determinismo (la misma entrada siempre produce la misma salida), irreversibilidad práctica (no es computacionalmente factible recuperar la entrada a partir de la salida), y resistencia a colisiones (es computacionalmente difícil encontrar dos entradas distintas que produzcan la misma salida) (Menezes, van Oorschot & Vanstone, 1997).

SHA-256 es un algoritmo de hash perteneciente a la familia SHA-2 (Secure Hash Algorithm 2), diseñado por la Agencia de Seguridad Nacional de los Estados Unidos y estandarizado por el NIST (National Institute of Standards and Technology, 2015). Produce resúmenes de 256 bits (32 bytes), habitualmente representados como 64 caracteres hexadecimales. Es ampliamente usado en sistemas de seguridad, certificados digitales y protocolos de comunicación.

HMAC (*Hash-based Message Authentication Code*) es un mecanismo estándar (RFC 2104) para construir un código de autenticación de mensajes basado en una función hash (Krawczyk, Bellare & Canetti, 1997). A diferencia de aplicar la función hash directamente

sobre el mensaje, HMAC incorpora una clave secreta en el cálculo. La operación puede describirse conceptualmente como: `HMAC(K, m) = H((K ⊕ opad) || H((K ⊕ ipad) || m))`, donde `H` es la función hash subyacente, `K` es la clave, `m` es el mensaje, y `opad` e `ipad` son constantes definidas en el estándar.

La diferencia fundamental entre un hash convencional y HMAC es que el primero produce siempre el mismo resultado para la misma entrada, independientemente de quién lo calcule. Esto hace al hash simple vulnerable a ataques de diccionario o de tabla arcoíris: un atacante que obtenga la base de datos puede precalcular los hashes de todas las CURP posibles y compararlos con los valores almacenados. HMAC resuelve este problema porque el resultado depende tanto de la entrada como de una clave secreta. Sin conocer la clave, no es posible reproducir el cálculo, lo que invalida los ataques de diccionario estándar.

3.6.2 Implementación en UCES Plataforma de Talento

En `lib/curp.ts`, la función `hashCurpHmac` implementa este mecanismo usando el módulo `crypto` de Node.js: recibe la clave secreta y la CURP, normaliza la CURP a mayúsculas sin espacios, y produce el resumen hexadecimal del HMAC-SHA256. La clave secreta se lee de la variable de entorno `CURP_HASH_SECRET`, que no se expone al navegador.

Este mecanismo permite detectar registros duplicados —si dos intentos de registro presentan la misma CURP, producirán el mismo hash, y la restricción de unicidad sobre la columna `curp_hash` en la tabla `curp_hashes` rechazará el segundo— sin almacenar en ningún momento la CURP en texto claro. El único valor que persiste en la base de datos es el hash, que por las propiedades de HMAC no permite recuperar la CURP original ni ser utilizado por un tercero que no posea la clave secreta.

Es importante distinguir este mecanismo de la **criptografía simétrica** (como AES), que sí permite recuperar el valor original a partir del texto cifrado y una clave. HMAC es una transformación unidireccional; una vez calculado el hash, no existe operación de descifrado. Esta propiedad es la adecuada para el caso de uso de UCES: el sistema no necesita conocer la CURP original una vez completado el registro, solo necesita detectar si una CURP ya fue usada, lo que puede determinarse comparando hashes.

3.7 Evaluaciones digitales y control de tiempo

Las evaluaciones digitales de conocimientos son instrumentos que presentan al candidato una serie de reactivos —preguntas de opción múltiple, verdadero/falso, respuesta abierta— y

registran sus respuestas en un sistema centralizado (Clariana & Wallace, 2002). Frente a las evaluaciones en papel, ofrecen ventajas de automatización: la corrección puede ejecutarse en el momento del envío, los resultados quedan disponibles inmediatamente y la información puede centralizarse y compararse entre candidatos sin procesamiento manual.

Un elemento crítico en muchas evaluaciones digitales es el control del tiempo disponible. Los temporizadores sirven para simular condiciones de presión reales, limitar la ventaja de quienes disponen de más tiempo para investigar respuestas, o garantizar que todas las evaluaciones se completen dentro de un periodo comparable. El diseño del control de tiempo en un sistema web requiere resolver una tensión inherente: la representación visual del tiempo transcurrido debe estar en el cliente —para actualizarse en tiempo real sin solicitudes constantes al servidor— pero la verificación de que el tiempo no ha expirado debe ocurrir en el servidor, ya que el cliente puede ser manipulado.

3.7.1 Modelo de control de tiempo en UCES Plataforma de Talento

En el sistema, cuando se crea un intento de evaluación, el servidor registra en la tabla `attempts` los campos `started_at` (momento de inicio) y `deadline_at` (fecha y hora límite), calculada como `started_at + duration_minutes`. El cliente calcula la diferencia entre `deadline_at` y la hora actual para mostrar el temporizador visual. Cuando el candidato envía la evaluación —o cuando el temporizador visual llega a cero y activa el envío automático— el Route Handler en `app/api/attempts/[attemptId]/submit/route.ts` verifica si `Date.now() > new Date(attempt.deadline_at).getTime()`. Si el tiempo ha expirado, el intento es igualmente procesado pero el campo `timedOutBeforeSubmit` en la respuesta y en el resumen almacenado refleja que el envío ocurrió fuera de tiempo.

Este diseño garantiza que la información sobre cumplimiento del tiempo provenga del servidor, que opera en un entorno controlado, y no de la representación visual en el navegador, que puede ser alterada por el usuario. El administrador que revisa el intento puede verificar los campos de tiempo para determinar si la evaluación se completó dentro del período establecido.

3.7.2 Persistencia de intentos y unicidad

El sistema implementa la restricción de un único intento por evaluación mediante el estado del registro en la tabla `attempts`. El guard de la ruta `/evaluacion/[attemptId]` verifica el campo `submitted_at`: si ya tiene un valor —lo que indica que el intento fue enviado—,

redirige al candidato a la página de resultados en lugar de permitirle continuar. Esta verificación ocurre en el servidor durante la carga de la ruta, antes de que se presente la interfaz de la evaluación.

La respuesta del Route Handler de envío también verifica este estado antes de procesarlo: si `attempt.submitted_at` ya tiene un valor, devuelve un error 400 indicando que el intento ya fue enviado. Esto previene la condición de carrera en la que el candidato podría enviar el formulario dos veces de forma rápida antes de que el estado del primer envío se refleje en el cliente.

3.8 Modelo Big Five y evaluación psicométrica

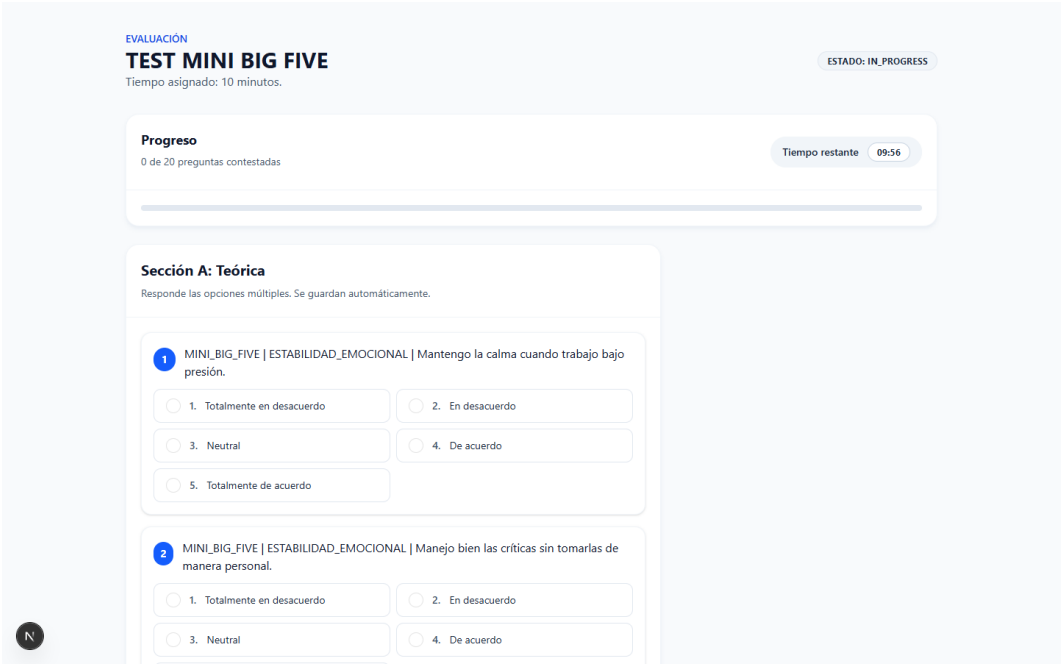


Figura 5.19. Pantalla del TEST MINI BIG FIVE con 20 ítems Likert 1–5 y temporizador de 10 minutos. Fuente: captura de pantalla propia, 2026.

La psicometría es la disciplina que estudia la medición de atributos psicológicos —aptitudes, rasgos de personalidad, actitudes— mediante instrumentos estandarizados (Muñiz, 2010). Los instrumentos psicométricos son cuestionarios o pruebas diseñados para cuantificar estos atributos de forma sistemática y reproducible. En contextos de selección de personal, se usan para obtener información sobre características de los candidatos que no son directamente observables a partir del currículum vitae o durante una entrevista breve.

3.8.1 El modelo de los Cinco Grandes factores

El modelo de los Cinco Grandes factores de personalidad (*Big Five*) es uno de los marcos teóricos más respaldados empíricamente en la psicología de la personalidad (McCrae & Costa, 1992). Propone que la personalidad puede describirse de forma comprensiva a través de cinco dimensiones ortogonales —es decir, estadísticamente independientes entre sí— que emergen de forma consistente en estudios de análisis factorial sobre diferentes poblaciones, idiomas e instrumentos de medición:

- **Apertura a la experiencia** (*Openness to experience*): tendencia hacia la curiosidad intelectual, la receptividad a ideas nuevas, la creatividad y la flexibilidad de pensamiento.
- **Responsabilidad** (*Conscientiousness*): disposición hacia la organización, la autodisciplina, la confiabilidad y la orientación al cumplimiento de metas.
- **Extroversión** (*Extraversion*): tendencia a la sociabilidad, la asertividad, el entusiasmo y la búsqueda de estimulación social.
- **Amabilidad** (*Agreeableness*): inclinación hacia la cooperación, la empatía, la confianza en los demás y el comportamiento prosocial.
- **Estabilidad emocional** (*Neuroticism*, medido inversamente): grado en que la persona experimenta con menor frecuencia emociones negativas como ansiedad, irritabilidad y vulnerabilidad al estrés.

El modelo ha sido validado en estudios transculturales y ha demostrado relaciones estadísticamente significativas con resultados relevantes en contextos laborales, como el desempeño en el trabajo, la satisfacción laboral y la permanencia en la organización (Barrick & Mount, 1991). Esta consistencia empírica lo ha convertido en el marco de referencia dominante para el diseño de instrumentos de selección de personal basados en personalidad.

3.8.2 Escalas tipo Likert

Las escalas Likert son un método de medición actitudinal en el que el respondiente expresa su grado de acuerdo o desacuerdo con una afirmación sobre una escala ordenada de opciones (Likert, 1932). La escala de cinco puntos es la más común: generalmente los extremos corresponden a "Totalmente en desacuerdo" y "Totalmente de acuerdo", y los puntos intermedios representan posiciones graduales entre ambos extremos. Los ítems se

suman —o se promedian— para obtener una puntuación que representa la posición del respondiente en la dimensión medida.

La interpretación de puntuaciones en escalas Likert requiere cautela. Las puntuaciones obtenidas son ordinales —los intervalos entre puntos no son necesariamente iguales en términos psicológicos— aunque en la práctica se tratan frecuentemente como intervalares para facilitar el cálculo estadístico. La validez y confiabilidad del instrumento deben establecerse mediante estudios específicos que evalúen si el instrumento mide consistentemente lo que pretende medir (Muñiz, 2010).

3.8.3 Implementación en UCES Plataforma de Talento

El instrumento psicométrico utilizado en el sistema consta de 20 ítems de escala Likert con valores del 1 al 5, organizados en cinco rasgos con cuatro ítems cada uno. Esta estructura es consistente, en número de ítems y organización por dimensiones, con versiones reducidas del Big Five como el Mini-IPIP de 20 ítems de Donnellan et al. (2006). Esta semejanza estructural es una referencia de comparación y no implica que el instrumento utilizado sea el Mini-IPIP ni ninguna otra versión publicada del modelo: el cuestionario fue proporcionado por personal del área de Recursos Humanos de UCES con la denominación "TEST MINI BIG FIVE", y su origen, proceso de adaptación y propiedades psicométricas formales no están disponibles en el expediente técnico del proyecto.

La lógica de cálculo, implementada en `lib/psicometricoResults.ts`, suma las respuestas Likert de los cuatro ítems de cada rasgo para obtener una puntuación por rasgo en el rango de 4 a 20. Los umbrales de clasificación aplicados —puntuación ≥ 16 como "Alta", entre 11 y 15 como "Adecuada", y menor a 11 como indicador de riesgo— son criterios institucionales informados por el desarrollador durante el proceso de especificación del sistema y no corresponden a puntos de corte derivados de estudios de estandarización publicados.

La extroversión tiene un tratamiento diferenciado en la implementación: su puntuación se calcula pero su clasificación se etiqueta como "No es filtro de contratación, solo indicador de integración". Esta distinción refleja una decisión de diseño del sistema que reconoce la complejidad de usar la extroversión como criterio de selección, dado que su pertinencia varía considerablemente según el tipo de puesto.

Los resultados del instrumento se almacenan como objeto JSONB en el campo `psychometric_summary` de la tabla `attempts`, preservando tanto las puntuaciones por rasgo como el número de preguntas respondidas y metadatos del envío. Esto permite al

administrador consultar el perfil completo del candidato sin necesidad de reconstruirlo a partir de las respuestas individuales.

3.9 Diseño de interfaces y experiencia de usuario

La experiencia de usuario (UX) es la disciplina que estudia y diseña la forma en que las personas interactúan con un sistema, con el objetivo de que esa interacción sea eficiente, comprensible y satisfactoria (Nielsen & Norman Group, 2024). En el contexto de aplicaciones web, la UX abarca desde la organización de la información y la coherencia visual hasta la retroalimentación que el sistema ofrece ante las acciones del usuario y la prevención de errores.

Algunos principios de usabilidad relevantes para sistemas de evaluación como el de UCES son:

- **Visibilidad del estado del sistema**: el usuario debe saber en todo momento en qué parte del proceso se encuentra, qué ha completado y qué falta. En el sistema, el temporizador visible y los indicadores de sección del cuestionario cumplen esta función.
- **Prevención de errores**: es preferible diseñar para que los errores sean difíciles de cometer, antes que depender en la detección y recuperación. La validación de la CURP en el registro —que verifica el formato antes de enviar la solicitud— es un ejemplo de esta estrategia.
- **Consistencia y estándares**: los usuarios no deben cuestionarse si palabras, situaciones o acciones diferentes significan lo mismo. El uso de una biblioteca de componentes como shadcn/ui garantiza que los elementos interactivos —botones, campos de formulario, modales— se comporten y presenten de manera coherente en toda la aplicación.
- **Retroalimentación**: las acciones del usuario deben tener respuesta visible. Los estados de carga, los mensajes de error descriptivos y las confirmaciones de operaciones exitosas informan al usuario que el sistema recibió su acción y está procesándola.

3.9.1 Tailwind CSS y shadcn/ui

Tailwind CSS es un framework de estilos que provee clases utilitarias de bajo nivel —`flex`, `p-4`, `text-sm`, `rounded-lg`— que se aplican directamente sobre los elementos HTML para construir la apariencia de la interfaz sin escribir CSS personalizado para cada componente

(Tailwind Labs, 2024). A diferencia de frameworks como Bootstrap, que proveen componentes prediseñados con una apariencia fija, Tailwind provee las herramientas para construir cualquier diseño sin imponer una estética particular.

shadcn/ui es una colección de componentes de interfaz construidos sobre Radix UI, una biblioteca de primitivos de accesibilidad sin estilo para React (shadcn, 2024). Los componentes de shadcn/ui son accesibles por diseño —implementan los patrones de interacción y los atributos ARIA definidos por WAI-ARIA— y se estilizan con Tailwind CSS. Una característica distintiva de shadcn/ui es que los componentes no se instalan como dependencia de paquete: se copian al código fuente del proyecto, lo que permite modificarlos directamente sin depender de una versión específica de la biblioteca.

En UCES Plataforma de Talento, la combinación de Tailwind CSS y shadcn/ui permitió construir una interfaz coherente y funcional con tiempos de desarrollo razonables, aprovechando componentes base accesibles para formularios, tablas, botones y diálogos que se integran con el sistema de estilos global.

3.10 Despliegue de aplicaciones web en la nube

El despliegue (*deployment*) es el proceso de trasladar una aplicación desde el entorno de desarrollo del programador hacia un entorno accesible para los usuarios finales. En aplicaciones web modernas, este entorno es típicamente una plataforma en la nube que gestiona la infraestructura de servidores, la distribución geográfica del contenido y la disponibilidad del servicio (Kleppmann, 2017).

Un aspecto fundamental del despliegue es la separación entre entornos. El entorno de **desarrollo** es donde el programador escribe y prueba el código; normalmente opera con bases de datos locales o de prueba y con variables de configuración que no corresponden al servicio real. El entorno de **producción** es donde opera el sistema para los usuarios finales, con las credenciales definitivas, la base de datos real y las restricciones de seguridad completas. Mezclar ambos entornos —por ejemplo, conectar el entorno de desarrollo a la base de datos de producción— representa un riesgo operativo significativo.

3.10.1 Variables de entorno

Las variables de entorno son el mecanismo estándar para configurar aplicaciones web sin incluir valores sensibles en el código fuente (Twelve-Factor App, 2017). En lugar de escribir las credenciales de la base de datos directamente en el código, la aplicación las lee de

variables definidas en el entorno de ejecución. Esto permite usar credenciales distintas en desarrollo y producción sin modificar el código, y evita que secretos como claves de API o contraseñas queden expuestos en el repositorio de código fuente.

En UCES Plataforma de Talento, las variables de entorno relevantes son: `NEXT_PUBLIC_SUPABASE_URL` y `NEXT_PUBLIC_SUPABASE_ANON_KEY` (accesibles desde el navegador, por eso el prefijo `NEXT_PUBLIC_`), `SUPABASE_SERVICE_ROLE_KEY` (disponible solo en el servidor, nunca expuesta al cliente) y `CURP_HASH_SECRET` (la clave de HMAC, solo disponible en el servidor). La convención de Next.js de usar el prefijo `NEXT_PUBLIC_` para variables accesibles en el cliente y omitirlo para las exclusivas del servidor es un mecanismo de seguridad por convención: garantiza que las claves sensibles no se incluyan en el bundle de JavaScript enviado al navegador.

3.10.2 Vercel y el despliegue de Next.js

Vercel es la plataforma de despliegue desarrollada por el mismo equipo que mantiene Next.js (Vercel, 2024b). Provee integración nativa con el framework: detecta automáticamente la estructura de un proyecto Next.js, lo construye y lo despliega en su red de distribución de contenido (CDN). Los Server Components y los Route Handlers se ejecutan en funciones sin servidor (*serverless functions*) distribuidas geográficamente, lo que elimina la necesidad de administrar servidores dedicados.

La integración con servicios de control de versiones como GitHub permite configurar despliegues automáticos: cada *push* a la rama principal activa una nueva construcción y despliegue, y cada *pull request* genera un entorno de vista previa (*preview deployment*) con una URL única. Este flujo de trabajo facilita la revisión de cambios antes de que lleguen a producción.

Supabase, por su parte, opera como el servicio de datos independiente del despliegue de la aplicación. La conexión entre la aplicación desplegada en Vercel y la base de datos en Supabase se configura mediante las variables de entorno del proyecto de Vercel, separando claramente la capa de presentación y lógica del sistema de persistencia de datos.

3.11 Conclusiones del capítulo

Los fundamentos revisados en este capítulo forman la base sobre la que se construyó UCES Plataforma de Talento. La ingeniería de software proveyó el marco conceptual para transformar requisitos en una solución implementable mediante un proceso sistemático. El

modelo de arquitectura web moderna —con su separación de entornos, su API como punto de integración y sus mecanismos de seguridad por capas— determinó la estructura general del sistema.

Next.js y React definieron el modelo de componentes, el enrutamiento y la forma en que se procesa la lógica en el servidor y en el cliente. TypeScript añadió verificación estática que redujo los errores de integración entre capas. Supabase integró la base de datos relacional, el sistema de autenticación basado en JWT y el almacenamiento de archivos en una plataforma cohesionada con sus propias garantías de seguridad. El modelo RLS de PostgreSQL y la función `get_my_role()` permitieron definir reglas de acceso que operan directamente sobre los datos, independientemente del código de la aplicación.

El HMAC-SHA256 aplicado a la CURP resolvió la necesidad de detectar registros duplicados respetando la privacidad del dato original. El modelo Big Five proveyó el marco conceptual para el instrumento psicométrico, y las escalas Likert el mecanismo de captura de respuestas. El control de tiempo en las evaluaciones se diseñó para que la verificación definitiva ocurriera en el servidor, no en el cliente. Las herramientas de interfaz —Tailwind CSS y shadcn/ui— facilitaron construir una experiencia coherente y accesible sin comprometer la velocidad de desarrollo.

El Capítulo V retomará cada uno de estos fundamentos en el contexto específico de las decisiones de implementación: qué fragmentos de código aplican estos principios, qué diferencias emergieron entre el diseño previo y la implementación resultante, y qué consideraciones técnicas determinaron la forma final del sistema.

Capítulo IV. Metodología de desarrollo

4.1 Selección y justificación de la metodología

La elección de una metodología de desarrollo no es un requisito burocrático: es la decisión que define cómo se distribuyen el esfuerzo, el riesgo y la capacidad de respuesta ante lo que se descubre durante la construcción. Para UCES Plataforma de Talento, el modelo seleccionado fue el **prototipado evolutivo**, identificado formalmente en el Trabajo A de análisis y diseño como la metodología más adecuada para el contexto del proyecto (Arciniega Robles, 2026).

El prototipado evolutivo es un modelo de ciclo de vida del software en el que la primera versión del sistema —el prototipo inicial— no es un artefacto desechable, sino la base real sobre la que se construyen todas las versiones posteriores (Pressman & Maxim, 2015). Cada iteración incorpora nuevas funcionalidades, corrige deficiencias de la versión anterior y refina el comportamiento del sistema a partir de la verificación con requisitos y con los resultados observados. A diferencia del modelo en cascada, donde cada fase debe completarse antes de avanzar a la siguiente, el prototipado evolutivo acepta que los requisitos se clarifican y se estabilizan de manera progresiva conforme el sistema toma forma.

Para el proyecto de UCES, tres características del contexto hacen pertinente esta metodología:

Naturaleza progresiva de los requisitos. Si bien el Trabajo A formalizó 18 requisitos funcionales y 14 casos de uso antes de iniciar el desarrollo, varios aspectos del comportamiento del sistema —particularmente del módulo psicométrico y del flujo de revisión administrativa— no quedaron completamente especificados hasta que se construyó una primera versión funcional. La estructura del instrumento psicométrico fue proporcionada por el área de Recursos Humanos durante el desarrollo; no existía al momento de iniciar la implementación.

Necesidad de verificación mediante software funcional. En un sistema de evaluación digital, la validez de decisiones como la estructura del formulario psicométrico, el mecanismo de control de tiempo o la presentación de resultados al administrador no puede determinarse completamente con diagramas. Se necesita software ejecutable para verificar que el comportamiento se corresponde con lo esperado. El prototipado evolutivo facilita esta verificación en ciclos cortos.

Equipo reducido y alcance de MVP. El desarrollo fue realizado por un único desarrollador en el marco de residencias profesionales. En este contexto, los modelos de proceso con alta carga administrativa —como Scrum con sus ceremonias formales, roles definidos y artefactos de planificación— habrían generado más carga que valor. El prototipado evolutivo provee estructura suficiente sin exigir un equipo multi-rol.

4.2 Organización general del desarrollo

El proceso de desarrollo se organizó en torno a las siguientes actividades técnicas, que se repitieron de manera cíclica a medida que el sistema crecía en funcionalidades:

1. **Comprensión de los requisitos del incremento.** Antes de implementar cada grupo de funcionalidades, se revisaban los requisitos correspondientes del Trabajo A, se identificaban las tablas de base de datos involucradas y se determinaban los endpoints de API necesarios.
2. **Diseño de los componentes del incremento.** Se definían las rutas de Next.js que se crearían o modificarían, los tipos TypeScript necesarios para las estructuras de datos, y las consultas o mutaciones a Supabase que cada operación requería.
3. **Construcción de las funcionalidades.** Se implementaban los Server Components, Client Components y Route Handlers correspondientes, junto con los ajustes necesarios al esquema de base de datos si el incremento requería nuevas tablas o columnas.
4. **Integración con Supabase.** Se verificaba que las políticas de RLS aplicaran correctamente al nuevo código, que los clientes de Supabase (browser vs. server) se usaran en el contexto adecuado, y que las relaciones entre tablas funcionaran según el esquema.
5. **Verificación funcional.** Se ejecutaba el sistema localmente y se recorría el flujo completo del incremento para confirmar que el comportamiento visible coincidía con lo esperado.
6. **Identificación de ajustes.** Los problemas detectados durante la verificación —errores de tipos TypeScript, comportamientos inesperados de RLS, incompatibilidades de versión con las APIs de Next.js— se diagnosticaban y se incorporaban como trabajo del mismo incremento o del siguiente.
7. **Registro en Git.** Los cambios verificados se registraban con un mensaje descriptivo que identificaba el incremento. El historial de commits es el registro primario de cómo evolucionó el sistema.

Sobre la reconstrucción del proceso. La descripción de este capítulo combina dos tipos de información: lo que el historial de Git evidencia directamente (qué archivos se crearon o modificaron, en qué fecha y con qué propósito según el mensaje del commit) y lo que el desarrollador puede reconstruir con base en el conocimiento de las decisiones tomadas. Cuando la reconstrucción se basa únicamente en el historial, se indica explícitamente.

4.3 Preparación del entorno de desarrollo

El entorno de desarrollo se configuró como un monorepo con dos directorios principales: `my-app/`, que contiene la aplicación Next.js completa, y `supabase/`, que concentra los archivos de definición de la base de datos (esquema, roles, políticas, datos de prueba y migraciones). Esta separación mantiene los artefactos de cada capa independientes y facilita el rastreo de cambios en base de datos de forma separada al código de la aplicación.

Las herramientas principales del entorno fueron:

- **Node.js** (versión ^20 según `package.json`) como entorno de ejecución de JavaScript y gestor del servidor de desarrollo de Next.js.
- **npm** como gestor de paquetes. El archivo `package-lock.json` registra las versiones exactas de todas las dependencias instaladas.
- **TypeScript 5** como lenguaje de programación, con configuración en `tsconfig.json`.
- **Next.js 16.1.1** con modo desarrollo activado mediante `npm run dev`, que incluye recarga en caliente y compilación incremental.
- **Turbopack**, habilitado con `root: __dirname` en `next.config.ts`, como compilador experimental de alta velocidad para el entorno de desarrollo.
- **Supabase CLI** para gestionar el proyecto de base de datos, aplicar migraciones y sincronizar el esquema con la instancia remota del proyecto UcesCap.
- **Git** para el control de versiones local.
- **GitHub** (repositorio `github.com/MoyRb/uces-plataforma`) para el alojamiento del repositorio remoto y el registro de ramas de trabajo.

Las variables de entorno del proyecto —`NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY` y `CURP_HASH_SECRET`— se gestionaron mediante un archivo `.env.local` excluido del repositorio. Su estructura se documenta en el `README.md` del proyecto.

4.4 Construcción incremental del sistema

A partir del análisis del historial de Git, el desarrollo del sistema puede reconstruirse en ocho iteraciones funcionales, organizadas cronológicamente desde el inicio del proyecto (diciembre de 2025) hasta la estabilización final (mayo de 2026).

Iteración 1 — Andamiaje del proyecto (2025-12-30)

Commits de referencia: `2183d58`, `355a0cf`, `f562cea`, `c83fb06`

La primera iteración estableció la base técnica completa del sistema. Se creó el proyecto Next.js con su configuración de TypeScript, Tailwind CSS y shadcn/ui; se definió el esquema completo de la base de datos en `supabase/schema.sql` con las 13 tablas y sus relaciones; se escribieron las políticas iniciales de Row Level Security en `supabase/rls.sql`; y se implementaron las bibliotecas base de `lib/curp.ts`, `lib/supabaseClient.ts` y `lib/supabaseServer.ts`.

La decisión de definir el esquema completo de la base de datos en la primera iteración —en lugar de ir agregando tablas incrementalmente— responde al diseño previo del Trabajo A: el modelo de datos de 13 tablas ya existía documentado, por lo que podía implementarse desde el inicio sin necesidad de migraciones destructivas posteriores. Esta decisión redujo la cantidad de cambios estructurales necesarios en etapas avanzadas del desarrollo.

Iteración 2 — Autenticación, perfil candidato y flujo principal (2025-12-31)

Commits de referencia: `c3ddc61`, `24adf91`

En una sola jornada de trabajo intensa, se implementaron las funcionalidades fundamentales del flujo del candidato:

- Las páginas `/login`, `/register`, `/panel`, con sus lógicas de autenticación vía Supabase Auth.
- El Route Handler `POST /api/auth/register` con validación de CURP, generación de HMAC-SHA256 y creación transaccional del usuario.
- Una primera versión del panel `/admin`.
- Los módulos de evaluación: `/vacantes/[id]`, `/modulos/[moduleId]`, `/evaluacion/[attemptId]` (con `cliente.tsx`, `page.tsx` y `utils.ts`), y `/resultado/[attemptId]`.

- Los Route Handlers de candidato: `POST /api/vacancies/[id]/apply`, `POST /api/attempts/[attemptId]/answers` y `POST /api/attempts/[attemptId]/submit`.

Esta iteración produjo un sistema funcional de extremo a extremo en la ruta del candidato: era posible registrarse, autenticarse, ver una vacante, postularse, responder preguntas con temporizador, cargar evidencia y recibir un resultado. El commit `24adf91` que registra estos cambios muestra la adición de 1,170 líneas de código en 12 archivos nuevos, lo que refleja la magnitud del esfuerzo concentrado en esta etapa.

Iteración 3 — Roles reales con Supabase (2026-02-17)

Commits de referencia: `4d9f058`, `ac9c552`

La primera versión del sistema gestionaba el rol del usuario usando únicamente la columna `role` de la tabla `profiles`. Esta aproximación presentaba un problema de arquitectura: las políticas de RLS que necesitaban verificar el rol del usuario debían consultar `profiles` directamente, lo que generaba dependencias circulares al intentar aplicar la política sobre la propia tabla.

En esta iteración se introdujo la tabla `user_roles` como mecanismo principal de gestión de roles, junto con la función `get_my_role()` con cláusula `SECURITY DEFINER` que resuelve el rol consultando primero `user_roles` y luego cayendo al valor de `profiles.role`. El archivo `supabase/roles.sql` formalizó esta lógica y estableció también la migración de backfill para copiar los roles existentes en `profiles` hacia `user_roles`.

Iteración 4 — Panel administrativo completo (2026-02-18)

Commits de referencia: `ff2657f`, `6c30675`

Se implementó el panel administrativo con su conjunto completo de endpoints de API protegidos. El commit `ff2657f` creó 13 archivos nuevos:

- CRUD de módulos (`/api/admin/modules`, `/api/admin/modules/[id]`)
- CRUD de vacantes y su evaluación asociada (`/api/admin/vacancies`, `/api/admin/vacancies/[id]`, `/api/admin/vacancies/[id]/assessment`, `/api/admin/vacancies/[id]/stats`)
- CRUD de evaluaciones y preguntas (`/api/admin/assessments`, `/api/admin/assessments/[id]`, `/api/admin/assessments/[id]/questions`, `/api/admin/questions/[id]`)

- Gestión de intentos desde el panel admin (`/api/admin/attempts`, `/api/admin/attempts/[id]`)
- La función `requireAdmin()` en `app/api/admin/utils.ts`, que desde ese momento protege todos los endpoints administrativos verificando el JWT y el rol antes de ejecutar cualquier operación.

Iteración 5 — Estabilización de la compatibilidad con Next.js (2026-02-18)

Commits de referencia: `4f9ca47`, `c77284d`, `518e343`, `9563ccc`, `065f8d1`, `0905c1c`, `a25f08d`, `9fd7193`, `694d53c`

La migración al App Router de Next.js introdujo un cambio significativo en la forma en que las rutas dinámicas acceden a sus parámetros: a partir de ciertas versiones, el objeto `params` de los Route Handlers y las páginas dinámicas es una promesa que debe resolverse con `await`. Esta diferencia con respecto al comportamiento esperado generó una serie de errores de tiempo de ejecución y advertencias del compilador que se corrigieron en múltiples commits de esta iteración.

Adicionalmente, se corrigieron: la normalización de opciones de preguntas en el panel admin (`9fd7193`), un problema con la importación del helper `formatBytes` que impedía la compilación de la página de resultados (`a25f08d`), marcado HTML incorrecto con etiquetas `<p>` anidadas que generaban advertencias de hidratación (`065f8d1`), y la adición de la ruta `/perfil` que se había omitido en la iteración anterior (`694d53c`).

Iteración 6 — Flujo de revisión administrativa y decisiones (2026-02-18)

Commits de referencia: `9350275`, `022692f`, `58fa9d3`, `bfe4088`

Se completó el flujo de revisión: el administrador puede ahora ver el detalle completo de un intento —respuestas, evidencias con URL firmada, perfil del candidato— y registrar una decisión (APPROVED, REJECTED, UNDER_REVIEW) con notas. La decisión queda visible para el candidato en su página `/perfil`.

En esta iteración surgió también la necesidad de añadir columnas `decision` y `reviewer_notes` a la tabla `attempts` para soportar un mecanismo de respaldo cuando la tabla `reviews` no está disponible. Esto se formalizó en el archivo de migración `supabase/migration_attempts_decision_columns.sql`. La coexistencia de dos mecanismos

de almacenamiento de decisiones —`reviews` como primario, `attempts.decision` como respaldo— constituye la discrepancia D-02 documentada en la matriz de requisitos.

Iteración 7 — Módulo psicométrico (2026-03-04)

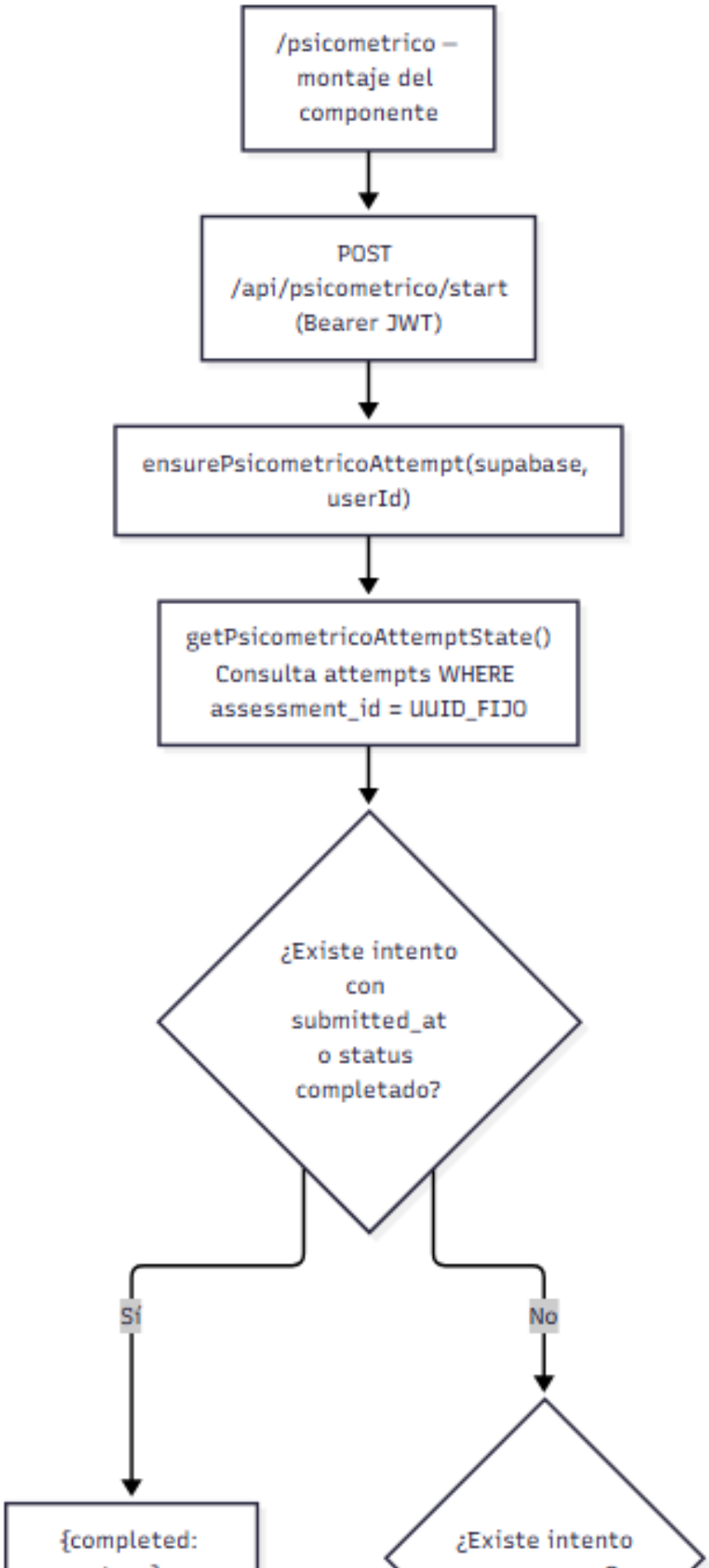

/psicometrico – montaje del componente
POST /api/psicometrico/start (Bearer JWT)
ensurePsicometricoAttempt(supabase, userId)
getPsicometricoAttemptState() Consulta attempts WHERE assessment_id = UUID_FIJO
¿Existe intento con submitted_at o status completado?
Sí
No
{completed:
¿Existe intento

Figura 5.21. Diagrama de inicio del flujo psicométrico con verificación de prerequisito. Fuente: elaboración propia, 2026.

Commits de referencia: `07fca11`, `c9188af`, `d7c0824`, `5870d26`

El módulo psicométrico se construyó en cuatro commits secuenciales que reflejan el proceso de clarificación de requisitos propio del prototipado evolutivo:

8. `07fca11`: Se crearon los datos iniciales del cuestionario psicométrico en `supabase/seed.sql` y `supabase/seed_psicometrico_base.sql`. El cuestionario se sembró con su estructura de 20 ítems codificados con el prefijo `MINI_BIG_FIVE|RASGO|` en el campo `prompt` de la tabla `questions`, lo que permite a la función de cálculo identificar el rasgo sin una columna adicional.
9. `c9188af`: Se implementaron la ruta `/psicometrico`, el Route Handler `POST /api/psicometrico/start` y la biblioteca `lib/psicometrico.ts`. Se agregó la verificación en `POST /api/vacancies/[id]/apply` que requiere que el psicométrico esté completado antes de permitir la postulación.
10. `d7c0824`: Se implementó `lib/psicometricoResults.ts` con la lógica completa de cálculo del puntaje por rasgo y la clasificación por umbrales. El campo `psychometric_summary` se añadió a la tabla `attempts` mediante la migración `migration_attempts_psychometric_summary.sql`. La función de submit se actualizó para calcular y almacenar el resumen al enviar.
11. `5870d26`: Se ajustó la página de resultados para ocultar los puntajes numéricos al candidato y mostrar únicamente clasificaciones textuales. El panel admin recibió la sección de resumen psicométrico con los cinco rasgos.

Iteración 8 — Ajuste del instrumento y mejoras de diseño (2026-05-08 a 2026-05-20)

Commits de referencia: `dd26d9a`, `d00ba8b`, `46027cb`, `0b84137`, `4075114`, `ca422c8`

La última iteración agrupó ajustes de fondo que surgieron de la revisión del sistema completo:

- El cuestionario psicométrico se reescribió para ajustar los ítems a la versión definitiva del instrumento proporcionado por UCES (`dd26d9a`). El número de ítems se confirmó en 20 (4 por rasgo) y el nombre del instrumento cambió a "TEST MINI BIG FIVE".
- Se creó una migración idempotente en `supabase/migrations/20260509_fix_admin_roles.sql` para garantizar que los roles de

administrador existentes en `profiles` se migraran correctamente a `user_roles` (`d00ba8b`).

- Se realizó una revisión general del diseño visual de la interfaz, actualizando 22 archivos de la carpeta `app/` para mejorar la apariencia y consistencia de todas las vistas (`46027cb`).
- Se introdujo el tipo exportado `ReviewDecision` en `lib/reviewDecision.ts` para resolver un error de tipos TypeScript que impedía la construcción del proyecto en producción (`0b84137`).
- Se corrigieron incompatibilidades de tipos en las respuestas de Supabase que relacionan tablas anidadas (`application.user_id` en submit route), desbloqueando la compilación limpia del proyecto (`4075114`).
- Se corrigió el orden de los `grant` y `revoke` en `supabase/rls.sql` para garantizar que la función `get_my_role()` fuera visible para los roles correctos, junto con ajustes en `lib/auth.ts` para mejorar el manejo de errores del RPC (`ca422c8`).

4.5 Integración y revisión de incrementos

La integración del sistema se produjo de forma continua: cada incremento se construía sobre el estado existente del repositorio, y los componentes que dependen unos de otros se conectaban en el mismo ciclo. La relación entre el frontend, los Route Handlers y Supabase sigue un patrón consistente en toda la aplicación:

12. Un componente de interfaz (Client Component) recopila la entrada del usuario y envía una solicitud HTTP al Route Handler correspondiente adjuntando el JWT en el encabezado `Authorization`.
13. El Route Handler valida el JWT (mediante `supabase.auth.getUser(token)` o `requireAdmin()`), ejecuta la lógica de negocio y realiza las operaciones sobre Supabase usando el cliente de servidor.
14. Supabase evalúa las políticas de RLS y ejecuta las consultas SQL.
15. La respuesta regresa al componente de interfaz, que actualiza el estado local y re-renderiza la vista correspondiente.

Este patrón tiene una consecuencia que es importante registrar: la verificación de permisos ocurre en dos capas independientes. La capa de API rechaza solicitudes sin token válido o sin rol suficiente; la capa de base de datos rechaza operaciones que las políticas de RLS no

permitan. Ambas capas actúan de forma independiente, de manera que un fallo en una no compromete la otra.

La distinción entre **pruebas de desarrollo** y **pruebas formales de aceptación** es relevante para este reporte. Las pruebas realizadas durante el desarrollo fueron pruebas funcionales manuales: el desarrollador ejecutaba el sistema localmente, recorría los flujos completos y verificaba que el comportamiento coincidiera con los requisitos. No se realizaron pruebas automatizadas (unitarias ni de integración) dentro del alcance de este MVP. Las pruebas formales de aceptación —verificación sistemática con casos documentados y condiciones de pasa/falla— se documentan en el Capítulo VI.

4.6 Gestión de modificaciones y resolución de problemas técnicos

El historial de Git registra al menos cinco categorías de problemas técnicos que requirieron modificaciones sustanciales al código o a la base de datos durante el desarrollo.

Problema T-01 — Acceso asíncrono a parámetros dinámicos en Next.js App Router

Elemento	Detalle
Situación técnica	En Next.js con App Router, el objeto `params` que reciben los Server Components y los Route Handlers de rutas dinámicas es una promesa a partir de ciertas versiones. El acceso directo como `params.id` produce una advertencia en desarrollo y puede causar errores en producción.
Componentes afectados	`app/modulos/[moduleId]/page.tsx`, `app/vacantes/[id]/page.tsx`, `app/api/attempts/[attemptId]/answers/route.ts`, `app/api/attempts/[attemptId]/submit/route.ts`, y otras rutas dinámicas.
Modificación realizada	Acceso mediante `const { id } = await params` en lugar de `const { id } = params` en todos los componentes y handlers dinámicos.
Evidencia en Git	Commits `c77284d`, `518e343`, `9563ccc`, `0905c1c`, `4075114` (2026-02-18 y 2026-05-15).
Resultado observable	Eliminación de advertencias de compilación y comportamiento correcto en rutas dinámicas.

Aprendizaje técnico	La documentación de Next.js sobre el modelo de parámetros en App Router especifica el comportamiento asíncrono. Leer las notas de versión antes de comenzar cada incremento previene este tipo de correcciones en cascada.

Problema T-02 — Circularidad en las políticas de RLS al verificar el rol

Elemento	**Detalle**
Situación técnica	Las políticas de RLS sobre `user_roles` necesitaban verificar si el usuario era administrador para determinar quién podía leer o modificar esa tabla. Si la verificación del rol consultaba `user_roles` directamente, se producía una recursión: la política intentaba evaluar la condición que dependía de la misma tabla que estaba siendo evaluada.
Componentes afectados	`supabase/rls.sql`, función `get_my_role()`.
Modificación realizada	La función `get_my_role()` se declaró con `SECURITY DEFINER`, lo que la hace ejecutarse con los permisos del propietario (el rol administrador de la base de datos) en lugar del usuario que la invoca. Esto le permite consultar `user_roles` sin activar sus propias políticas.
Evidencia en Git	Commit `4d9f058` (2026-02-17), `ca422c8` (2026-05-20). El segundo commit corrigió adicionalmente el orden de los `REVOKE`/`GRANT` sobre la función, que impedía que ciertos roles la invocaran correctamente.
Resultado observable	Políticas de RLS funcionales sin circularidad; el rol se resuelve correctamente en todas las consultas.
Aprendizaje técnico	Al diseñar funciones que se usan dentro de políticas de RLS, la cláusula `SECURITY`

	`DEFINER` es la herramienta estándar de PostgreSQL para este patrón. Debe usarse con precaución y con `SET search_path = public` para evitar que el cambio de contexto de seguridad permita inyecciones mediante objetos homónimos en esquemas no controlados.

Problema T-03 — Tipado de relaciones anidadas en respuestas de Supabase

Elemento	**Detalle**		
Situación técnica	Al usar el cliente de Supabase para consultas con relaciones (`select("..., application:applications!inner(user_id)")`), el SDK infería el tipo de la relación como un tipo de unión que incluía tanto el objeto como un arreglo del objeto. TypeScript rechazaba el acceso directo a `attempt.application.user_id` porque el tipo inferido era `{user_id: string}	{user_id: string}[]	null`.
Componentes afectados	`app/api/attempts/[attemptId]/submit/route.ts`, `app/api/attempts/[attemptId]/answers/route.ts`.		
Modificación realizada	Se agregó la normalización explícita `const application = Array.isArray(attempt.application) ? attempt.application[0] : attempt.application` antes de acceder a sus propiedades, junto con la declaración del tipo esperado mediante el genérico `.maybeSingle<AttemptWithApplication>()`.		
Evidencia en Git	Commit `4075114` (2026-05-15).		
Resultado observable	Compilación limpia del proyecto; el campo `user_id` es accesible sin errores de tipos.		
Aprendizaje técnico	Los tipos generados automáticamente por el SDK de Supabase para relaciones anidadas no siempre coinciden con el comportamiento real de la consulta. El uso de tipos		

	genéricos explícitos en combinación con la normalización del resultado es el patrón recomendado.		

Problema T-04 — Almacenamiento del resumen psicométrico: columna ausente en producción

Elemento	**Detalle**
Situación técnica	La columna `psychometric_summary jsonb` se agregó a la tabla `attempts` mediante migración (`migration_attempts_psychometric_summary.sql`), pero si la migración no se había aplicado en el entorno de producción al momento del despliegue, el intento de actualizar esa columna producía un error de base de datos que impedía el envío del intento por parte del candidato.
Componentes afectados	`app/api/attempts/[attemptId]/submit/route.ts`.
Modificación realizada	Se implementó un mecanismo de respaldo: si el error devuelto por Supabase contiene el mensaje `"column"` y `"psychometric_summary"`, el Route Handler reintenta la actualización sin ese campo y guarda el resumen en la tabla `reviews` como alternativa. El marcador `isMissingColumnError()` detecta esta condición.
Evidencia en Git	Commit `d7c0824` (2026-03-04).
Resultado observable	El sistema puede funcionar en ambos escenarios: con la columna presente (ruta principal) o sin ella (ruta de respaldo).
Aprendizaje técnico	Las migraciones de base de datos deben aplicarse al entorno de producción antes de desplegar el código que las usa. El mecanismo de respaldo es una solución de compatibilidad que no debe mantenerse indefinidamente; una vez confirmada la migración en producción, puede simplificarse. Este patrón es también el origen de la discrepancia D-02 documentada en el Trabajo B.

Problema T-05 — Tipado del enumerado de decisiones de revisión

Elemento	Detalle		
Situación técnica	Las decisiones administrativas (APPROVED, REJECTED, INFO_REQUIRED) se validaban mediante comprobaciones de cadena de texto en varios lugares del código. La ausencia de un tipo centralizado hacía que TypeScript no pudiera verificar que todos los usos fueran consistentes, y un error en un valor de cadena solo aparecería en tiempo de ejecución. Al compilar el proyecto para producción, TypeScript detectó inconsistencias que no eran evidentes durante el desarrollo local.		
Componentes afectados	`app/api/admin/attempts/[id]/route.ts`, `app/admin/attempts/[id]/page.tsx`, `app/api/admin/utils.ts`.		
Modificación realizada	Se creó el archivo `lib/reviewDecision.ts` que exporta el tipo unión `ReviewDecision = "APPROVED"	"REJECTED"	"INFO_REQUIRED"` y la función de guardia `isReviewDecision(value)`. Todos los usos del valor de decisión en el código fueron actualizados para usar este tipo.
Evidencia en Git	Commit `0b84137` (2026-05-15).		
Resultado observable	Compilación limpia; cualquier valor de decisión no reconocido es detectado por TypeScript en tiempo de compilación.		
Aprendizaje técnico	Los valores de enumerado que se usan en múltiples partes de la aplicación deben centralizarse en un único módulo de tipos		

	desde el inicio del desarrollo. TypeScript solo puede verificar la consistencia de tipos si el tipo está declarado explícitamente.		

4.7 Herramientas para el control de versiones

El sistema de control de versiones utilizado fue **Git**, con el repositorio remoto alojado en **GitHub** bajo la cuenta `MoyRb`. El historial del repositorio al cierre del desarrollo registra 56 commits distribuidos a lo largo de seis meses, desde el 30 de diciembre de 2025 hasta el 20 de mayo de 2026.

La estrategia de trabajo visible en el historial combina commits directos a la rama `main` para el trabajo de desarrollo individual, con ramas de trabajo que se integran mediante *pull requests*. Los nombres de las ramas siguen un patrón consistente con el prefijo `codex/` seguido de una descripción del cambio (por ejemplo, `codex/fix-typescript-build-errors`, `codex/implement-assessment-completion-requirement`). Esta convención facilita identificar el propósito de cada integración al revisar el historial.

No existen en el repositorio evidencias de configuración de CI/CD (integración y despliegue continuos), revisiones de código formales ni reglas de protección de ramas. El flujo de trabajo registrado es el de un desarrollador individual que usa las ramas de GitHub como mecanismo de organización del trabajo, no como pipeline de revisión grupal.

El archivo `.gitignore` excluye correctamente los archivos `.env.local` y `.env*.local` que contienen las variables de entorno con credenciales sensibles, garantizando que éstas no queden expuestas en el historial del repositorio.

4.8 Relación con el proceso de prototipado evolutivo

La siguiente tabla vincula las fases del modelo de prototipado evolutivo, tal como se describe en el Trabajo A (Arciniega Robles, 2026, sección 3.1), con las actividades técnicas del Trabajo B:

Fase del modelo (Trabajo A)	**Actividad técnica (Trabajo B)**	**Iteración**	**Evidencia**
Definición de requisitos iniciales	Lectura del catálogo de RF, RNF y CU del	Previa a cada iteración	`docs/titulacion/02-matriz-requisitos-`

	Trabajo A; identificación de tablas y endpoints necesarios		`implementacion.md`
Diseño rápido del prototipo	Definición de rutas Next.js, tipos TypeScript y consultas Supabase antes de implementar	Iteraciones 1-8	Arquitectura visible en `app/` y `supabase/`
Construcción del prototipo	Implementación de Server Components, Client Components y Route Handlers	Iteraciones 1-8	Commits en Git (ver sección 4.4)
Evaluación del prototipo	Verificación funcional manual del flujo completo de cada incremento	Tras cada iteración	Correcciones en la misma o siguiente iteración
Refinamiento	Corrección de errores, ajuste de comportamiento, mejoras de interfaz	Iteraciones 5, 7, 8	Commits de fix y ajuste (T-01 a T-05)
Entrega de la versión funcional	Sistema completo con 18 RF implementados, ejecutable localmente	Iteración 8	Estado final del repositorio `main`

El modelo de prototipado evolutivo aplicado en este proyecto tuvo una variante práctica: dado que el Trabajo A había formalizado los requisitos antes del inicio del desarrollo, las primeras iteraciones avanzaron con más confianza que en un proyecto donde los requisitos se descubren durante la construcción. Sin embargo, el carácter evolutivo se manifestó claramente en el módulo psicométrico —cuya estructura final no estaba completamente definida al inicio— y en la gestión de roles, que requirió dos iteraciones para alcanzar la arquitectura definitiva.

4.9 Evaluación de la metodología

Ventajas observadas en el contexto del proyecto

Flexibilidad ante requisitos emergentes. La estructura del instrumento psicométrico y los detalles del flujo de decisión administrativa no estaban completamente especificados al inicio. El prototipado evolutivo permitió construir versiones iniciales funcionales y ajustarlas cuando los requisitos se clarificaron, sin necesidad de rediseñar la arquitectura desde cero.

Verificación temprana del comportamiento. Tener un sistema ejecutable desde la primera semana de desarrollo permitió detectar incompatibilidades de la plataforma —el problema de los parámetros asíncronos en Next.js App Router (T-01)— antes de que afectaran a múltiples módulos. En un modelo secuencial, estos problemas podrían no haberse descubierto hasta etapas avanzadas.

Adaptabilidad del equipo reducido. Para un desarrollador individual, la capacidad de decidir en cada ciclo qué funcionalidad implementar a continuación —sin overhead de planificación formal— fue una ventaja operativa real.

Limitaciones observadas

Ausencia de pruebas automatizadas. El prototipado evolutivo no prescribe mecanismos de verificación específicos, y en este proyecto no se implementaron pruebas automatizadas. Como resultado, algunas correcciones en iteraciones tardías (T-01 a T-05) podrían haberse detectado antes con una suite de pruebas básica.

Acumulación de mecanismos de compatibilidad. El mecanismo de respaldo para `psychometric_summary` (T-04) y el doble almacenamiento de decisiones administrativas (D-02) son ejemplos de código que acumula soluciones de compatibilidad que en un ciclo de refactorización posterior deberían simplificarse.

Dependencia del historial de Git como documentación. En ausencia de bitácoras de desarrollo o diarios de iteración, el historial de commits es la única evidencia ordenada del proceso. Los mensajes de algunos commits —particularmente `b42b10b` con el mensaje "si"— no aportan información sobre el propósito del cambio, lo que reduce la trazabilidad en esos puntos.

Adecuación al proyecto

En balance, el prototipado evolutivo fue adecuado para las condiciones reales del proyecto: un desarrollador individual, un alcance de MVP bien definido por el Trabajo A, y un contexto donde los requisitos eran mayoritariamente conocidos pero algunos detalles de implementación solo se clarificaron durante la construcción. El resultado es un sistema funcional que implementa los 18 RF documentados, con un historial verificable que reconstruye el proceso de desarrollo.

4.10 Conclusiones del capítulo

El Capítulo IV documentó el proceso mediante el cual UCES Plataforma de Talento pasó de ser un conjunto de requisitos formalizados a un sistema de software funcional. El prototipado evolutivo proveyó el marco para organizar ese trayecto en iteraciones identificables, cada una con un conjunto de funcionalidades nuevas y un ciclo de verificación y ajuste.

El análisis del historial de Git permitió reconstruir ocho iteraciones de desarrollo, distribuidas entre diciembre de 2025 y mayo de 2026. Cada iteración produjo software ejecutable que se integraba directamente sobre la versión anterior. Los cinco problemas técnicos documentados en la sección 4.6 ilustran cómo el refinamiento iterativo no se limita a añadir funcionalidades, sino también a corregir incompatibilidades, reforzar la arquitectura de seguridad y resolver problemas de tipos que solo se manifiestan al compilar para producción.

La comprensión de este proceso es el prerequisito para el Capítulo V, donde se describe la implementación técnica de cada módulo del sistema con el nivel de detalle que permite entender las decisiones de código, las diferencias respecto al diseño del Trabajo A y la forma en que los fundamentos del Capítulo III se materializaron en código funcional.

Capítulo V. Desarrollo e implementación del sistema

5.1 Organización y arquitectura del software implementado

5.1.1 Estructura de directorios

UCES Plataforma de Talento es un monorepo organizado en dos grandes carpetas: `my-app/`, que contiene toda la aplicación Next.js, y `supabase/`, que concentra los artefactos de base de datos. Esta separación refleja la independencia entre la lógica de la aplicación y la definición de la infraestructura de datos.

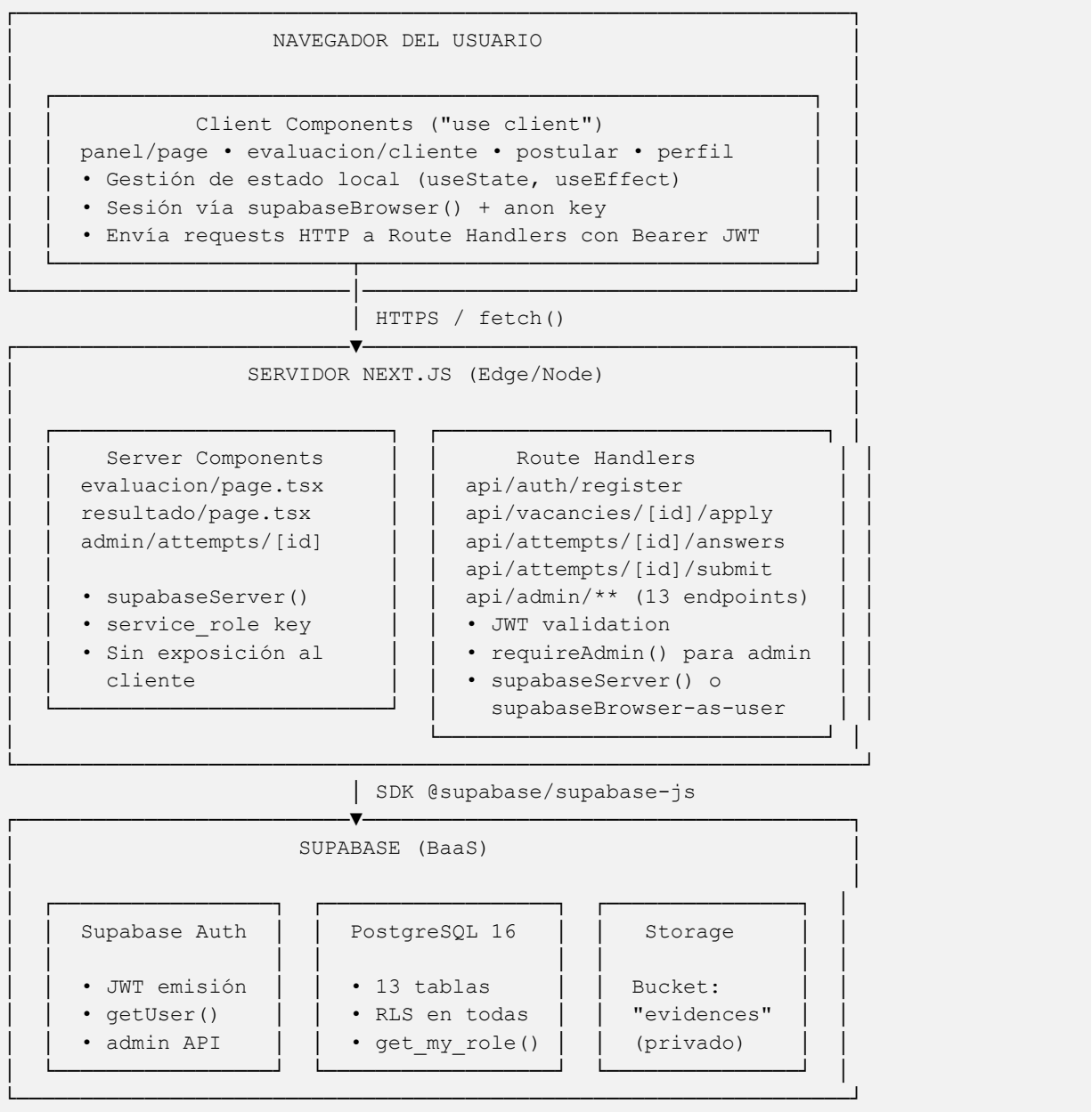
```
uces-plataforma/
├── my-app/                          ← Aplicación Next.js 16.1.1
│   ├── app/                         ← App Router (rutas, páginas, APIs)
│   │   ├── page.tsx                 ← Landing pública
│   │   ├── layout.tsx               ← Layout raíz con estilos globales
│   │   ├── login/page.tsx
│   │   ├── register/page.tsx
│   │   ├── panel/page.tsx
│   │   ├── perfil/page.tsx
│   │   ├── psicometrico/page.tsx
│   │   ├── modulos/[moduleId]/page.tsx
│   │   ├── vacantes/[id]/page.tsx
│   │   ├── vacantes/[id]/postular.tsx
│   │   ├── evaluacion/[attemptId]/
│   │   │   ├── page.tsx             ← Server Component: carga datos
│   │   │   ├── cliente.tsx          ← Client Component: timer e interacción
│   │   │   └── utils.ts             ← Helper formatBytes
│   │   ├── resultado/[attemptId]/page.tsx
│   │   ├── admin/page.tsx
│   │   ├── admin/attempts/[id]/page.tsx
│   │   └── api/                     ← Route Handlers
│   │       ├── auth/register/route.ts
│   │       ├── psicometrico/start/route.ts
│   │       ├── vacancies/[id]/apply/route.ts
│   │       ├── attempts/[id]/answers/route.ts
│   │       ├── attempts/[id]/submit/route.ts
│   │       └── admin/               ← 13 endpoints protegidos con requireAdmin()
│   ├── components/ui/               ← Componentes shadcn/ui (Button, Card, Badge...)
│   └── lib/                         ← Bibliotecas de negocio
│       ├── auth.ts                  ← getRoleForSession, resolveRole
│       ├── curp.ts                  ← hashCurpHmac, normalizeCurp
│       ├── psicometrico.ts          ← getPsicometricoAttemptState,
ensurePsicometricoAttempt
│       ├── psicometricoResults.ts ← calculatePsicometricoSummary
│       ├── reviewDecision.ts        ← ReviewDecision type, isReviewDecision
│       ├── supabaseClient.ts        ← Cliente browser (singleton)
│       └── supabaseServer.ts        ← Cliente servidor (service role)
├── supabase/
│   ├── schema.sql                   ← Definición de las 13 tablas
│   ├── rls.sql                      ← Función get_my_role() y todas las políticas RLS
```

```
│   ├── roles.sql                  ← Tabla user_roles con backfill
│   ├── seed.sql                   ← Datos de prueba y cuestionario psicométrico
│   └── migration_*.sql            ← Migraciones incrementales
```

5.1.2 Arquitectura implementada

La arquitectura sigue el patrón de tres capas adaptado a la plataforma Next.js/Supabase:

Figura 5.1 — Diagrama de arquitectura implementada



5.1.3 Componentes principales

Componente	Tipo	Responsabilidad	Archivo
`EvaluationPage`	Server Component	Carga intento, preguntas y respuestas guardadas desde servidor	`app/evaluacion/[attemptId]/page.tsx`
`EvaluationClient`	Client Component	Timer, guardado de respuestas, carga de evidencia, envío	`app/evaluacion/[attemptId]/cliente.tsx`
`PanelPage`	Client Component	Sesión, rol, módulos, estado psicométrico	`app/panel/page.tsx`
`ApplyButton`	Client Component	Flujo de postulación con manejo de errores y redirección	`app/vacantes/[id]/postular.tsx`
`requireAdmin()`	Middleware de API	Validación JWT + verificación de rol admin	`app/api/admin/utils.ts`
`supabaseBrowser()`	Singleton	Cliente Supabase para el navegador con clave anónima	`lib/supabaseClient.ts`

`supabaseServer()`	Factory	Cliente Supabase con clave de servicio para el servidor	`lib/supabaseServer.ts`
`getRoleForSession()`	Función utilitaria	Invoca RPC `get_my_role()` y normaliza el resultado	`lib/auth.ts`
`hashCurpHmac()`	Función criptográfica	HMAC-SHA256 de la CURP con clave de entorno	`lib/curp.ts`
`calculatePsicometricoSummary()`	Función de cálculo	Score Big Five, clasificación por rasgo	`lib/psicometricoResults.ts`

5.1.4 Diferencia entre la arquitectura propuesta y la implementada

El Trabajo A propuso una arquitectura web en tres capas (presentación, lógica de negocio, datos) sin especificar el framework. La implementación realiza esa separación dentro del modelo de Next.js App Router:

Capa propuesta (Trabajo A)	**Realización en Trabajo B**
Capa de presentación	Server Components + Client Components en `app/`
Capa de lógica de negocio	Route Handlers en `app/api/` + funciones en `lib/`
Capa de datos	PostgreSQL en Supabase, accedido vía SDK
Control de acceso	RLS en PostgreSQL + `requireAdmin()` en API

Una diferencia notable es que el Trabajo A contemplaba un servidor de aplicaciones independiente de la base de datos. En la implementación, Next.js y Supabase operan como plataformas separadas: la aplicación se despliega en Vercel y la base de datos en Supabase. La comunicación entre ambas ocurre a través del SDK de Supabase usando las credenciales de servicio almacenadas como variables de entorno del servidor.

5.2 Configuración e integración de Supabase



Figura 5.4. Vista de la página de inicio pública del sistema (/). Fuente: captura de pantalla propia, 2026.

5.2.1 Dos clientes, dos contextos



Figura 5.4. Vista de la página de inicio pública del sistema (/). Fuente: captura de pantalla propia, 2026.

La integración con Supabase se divide en dos instancias del cliente con propósitos y niveles de privilegio distintos.

Cliente de navegador — `lib/supabaseClient.ts`

```
// lib/supabaseClient.ts
export const supabaseBrowser = () => {
  if (!browserClient) {
    browserClient = createClient(supabaseUrl, supabaseKey);  // anon key
  }
  return browserClient;
};
```

Este cliente usa la clave anónima (`NEXT_PUBLIC_SUPABASE_ANON_KEY`), que es pública e incluida en el bundle de JavaScript enviado al navegador. Las operaciones que realiza están sujetas a las políticas de Row Level Security: solo puede leer o modificar los registros que las políticas permiten al usuario autenticado. Se implementa como singleton para evitar crear múltiples conexiones en re-renders.

Cliente de servidor — `lib/supabaseServer.ts`

```
// lib/supabaseServer.ts
export const supabaseServer = () => {
  return createClient(supabaseUrl, serviceRole, {
    auth: { autoRefreshToken: false, persistSession: false }
```

```
  });
};
```

Este cliente usa la clave de rol de servicio (`SUPABASE_SERVICE_ROLE_KEY`), que tiene acceso irrestricto a la base de datos y omite las políticas de RLS. Se usa exclusivamente en los Route Handlers del servidor y en los Server Components. Las opciones `autoRefreshToken: false` y `persistSession: false` indican que este cliente no mantiene estado entre solicitudes, lo que es correcto para un entorno sin estado como las funciones de servidor.

5.2.2 Variables de entorno y separación de credenciales



Figura 5.4. Vista de la página de inicio pública del sistema (/). Fuente: captura de pantalla propia, 2026.

Variable	Prefijo	Accesible en	Propósito
`NEXT_PUBLIC_SUPABASE_URL`	`NEXT_PUBLIC_`	Cliente + Servidor	URL del proyecto Supabase
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	`NEXT_PUBLIC_`	Cliente + Servidor	Clave pública para operaciones RLS
`SUPABASE_SERVICE_ROLE_KEY`	Sin prefijo	Solo servidor	Clave privilegiada, omite RLS
`CURP_HASH_SECRET`	Sin prefijo	Solo servidor	Clave HMAC para protección de

			CURP

La convención de Next.js que distingue variables con `NEXT_PUBLIC_` (incluidas en el bundle del cliente) de las que no llevan ese prefijo (solo disponibles en el servidor) es el mecanismo que garantiza que las credenciales privilegiadas no lleguen al navegador.

5.3 Implementación del registro y autenticación

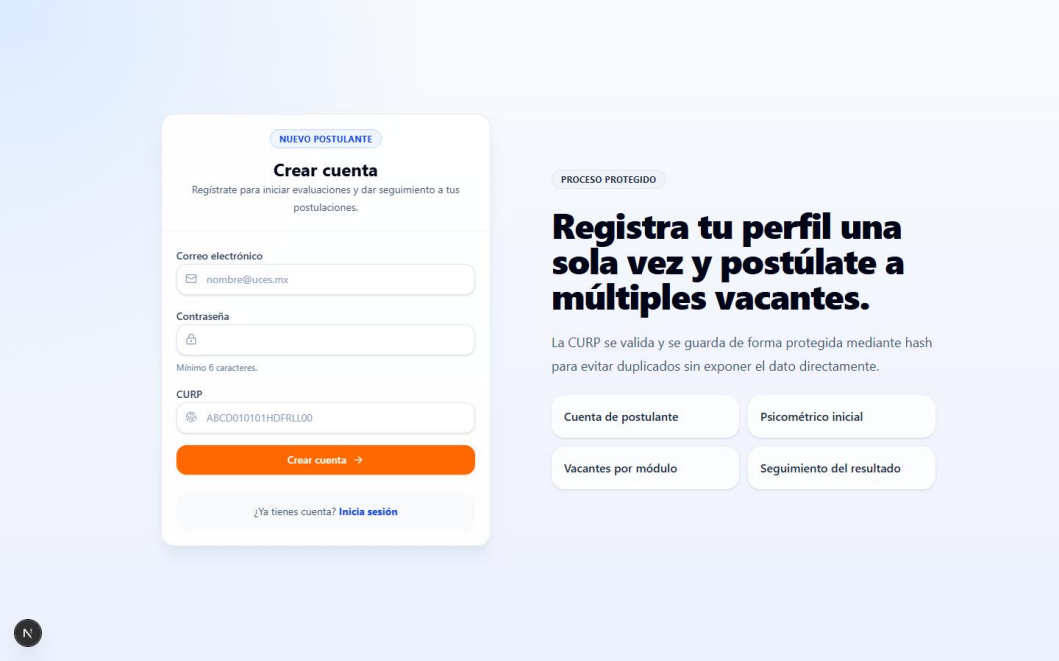


Figura 5.5. Formulario de registro de candidato (/register). Fuente: captura de pantalla propia, 2026.

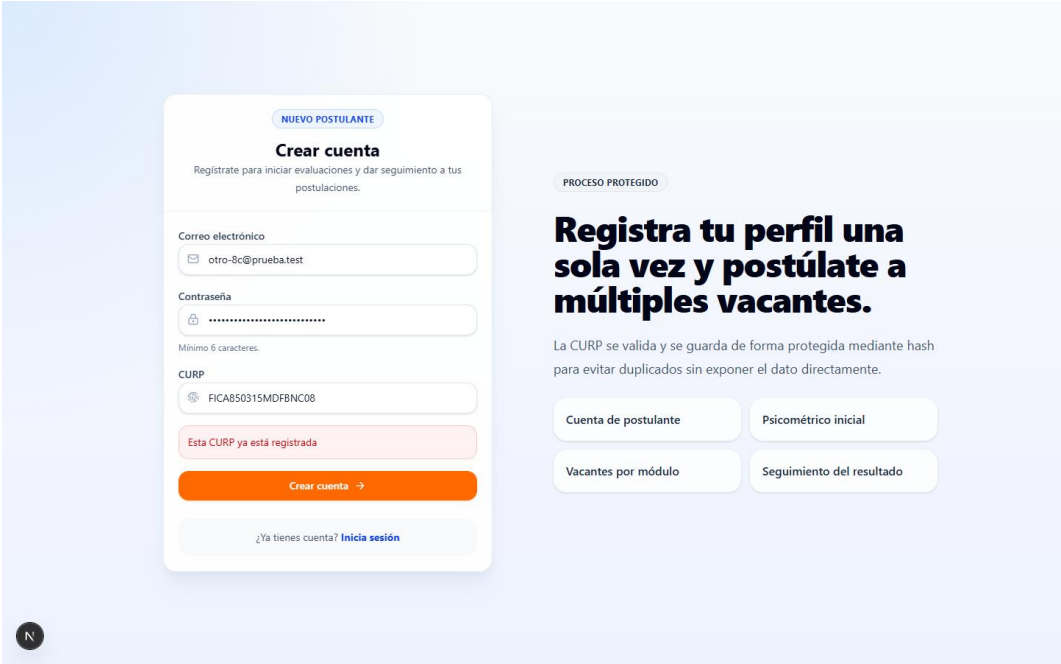


Figura 5.6. Mensaje de error por CURP ya registrada (CP-02a). Fuente: captura de pantalla propia, 2026.

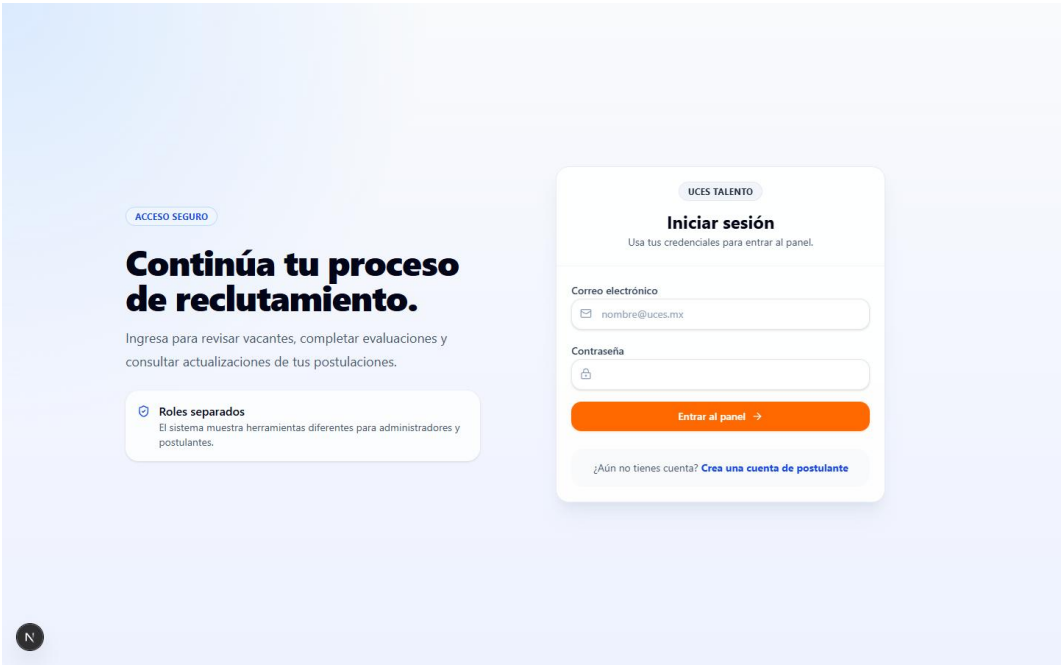


Figura 5.7. Formulario de inicio de sesión (/login). Fuente: captura de pantalla propia, 2026.

5.3.1 Flujo de registro (RF-01, RF-02)

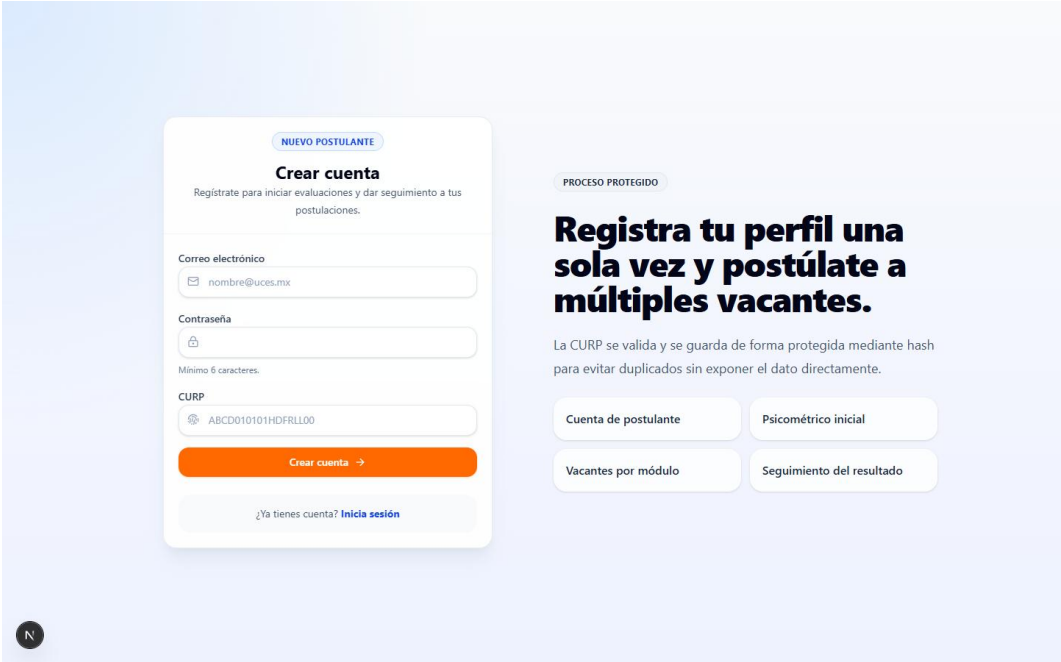


Figura 5.5. Formulario de registro de candidato (/register). Fuente: captura de pantalla propia, 2026.

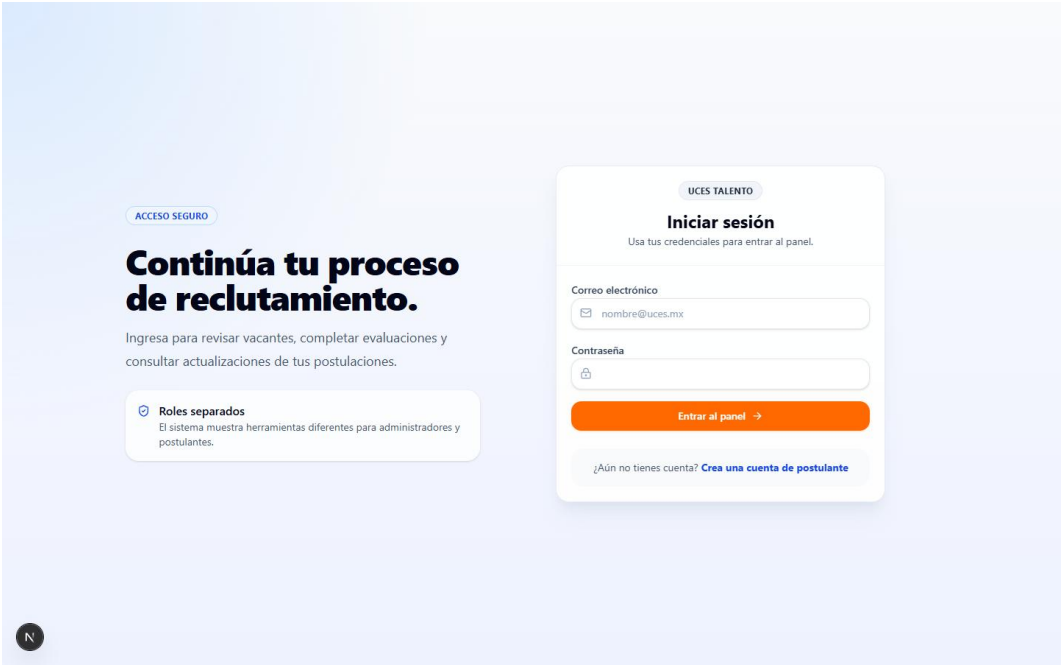


Figura 5.7. Formulario de inicio de sesión (/login). Fuente: captura de pantalla propia, 2026.

El registro de candidatos es la operación de mayor complejidad transaccional del sistema. El Route Handler `POST /api/auth/register` coordina cuatro operaciones sobre tres tablas diferentes y el servicio de autenticación, con reversión automática si cualquier paso falla.

Figura 5.2 — Diagrama de secuencia del registro de candidato

```
sequenceDiagram
    participant Cliente as Navegador<br/>(register/page.tsx)
    participant API as Route Handler<br/>POST /api/auth/register
    participant Auth as Supabase Auth
    participant DB as PostgreSQL

    Cliente->>API: POST {email, password, curp}
    API->>API: normalizeCurp(curp) → MAYÚSCULAS sin espacios
    API->>API: curpPattern.test(curp) → valida formato regex
    API->>API: hashCurpHmac(secret, curp) → digest hex (HMAC-SHA256)
    API->>DB: SELECT user_id FROM curp_hashes WHERE curp_hash = $hash
    alt CURP ya registrada
        DB-->>API: {user_id: "..."}
        API-->>Cliente: 400 "Esta CURP ya está registrada"
    else CURP disponible
        DB-->>API: null
        API->>Auth: admin.createUser({email, password, email_confirm: true})
        Auth-->>API: {user: {id: uuid}}
        API->>DB: INSERT INTO profiles (user_id, email)
        alt Error en profiles
            API->>Auth: admin.deleteUser(userId) ← rollback
            API-->>Cliente: 500 "No se pudo guardar el perfil"
        end
        API->>DB: INSERT INTO curp_hashes (user_id, curp_hash)
        alt Error en curp_hashes (race condition)
            API->>Auth: admin.deleteUser(userId) ← rollback
            API-->>Cliente: 400 "Esta CURP ya está registrada"
        end
        API->>DB: INSERT INTO user_roles (user_id, role='user')
        alt Error en user_roles
            API->>Auth: admin.deleteUser(userId) ← rollback
            API-->>Cliente: 500 "No se pudo asignar el rol inicial"
        end
        API-->>Cliente: 201 {message: "Usuario creado"}
    end
```

Análisis de las validaciones del registro:

El Route Handler ejecuta dos tipos de validaciones antes de crear el usuario:

16. **Validación de formato de CURP:** La expresión regular `/^[A-Z]{4}\d{6}[HM][A-Z]{5}[A-Z0-9]\d$/` verifica la estructura formal de 18 caracteres. Esta validación ocurre después de normalizar la CURP a mayúsculas mediante `normalizeCurp()`, lo que garantiza que "garca800101hdfrrr01" y "GARCA800101HDFRRR01" sean tratados como equivalentes.
17. **Verificación de duplicado mediante HMAC:** El hash HMAC-SHA256 se calcula antes de cualquier operación de base de datos. La consulta sobre `curp_hashes` verifica si ya existe ese hash. Este orden es deliberado: consultar antes de crear evita crear el usuario y luego descubrir que la CURP estaba registrada.

Patrón de rollback transaccional:

PostgreSQL soporta transacciones nativas (`BEGIN / COMMIT / ROLLBACK`), pero el SDK de Supabase no expone una API de transacciones multitabla en JavaScript. El código resuelve esto de forma manual: cada paso verifica si ocurrió un error y, de ser así, invoca `supabase.auth.admin.deleteUser(userId)` para revertir la creación del usuario en Supabase Auth antes de retornar el error al cliente. Esto garantiza que no queden usuarios huérfanos en el sistema de autenticación cuando la creación de registros dependientes falla.

La opción `email_confirm: true` en `admin.createUser()` indica que la cuenta se crea ya confirmada, sin requerir que el usuario haga clic en un enlace de verificación. Esta decisión refleja el contexto de uso controlado del sistema, donde los candidatos son invitados a registrarse dentro de un proceso institucional.

5.3.2 Autenticación y resolución de rol (RF-03)

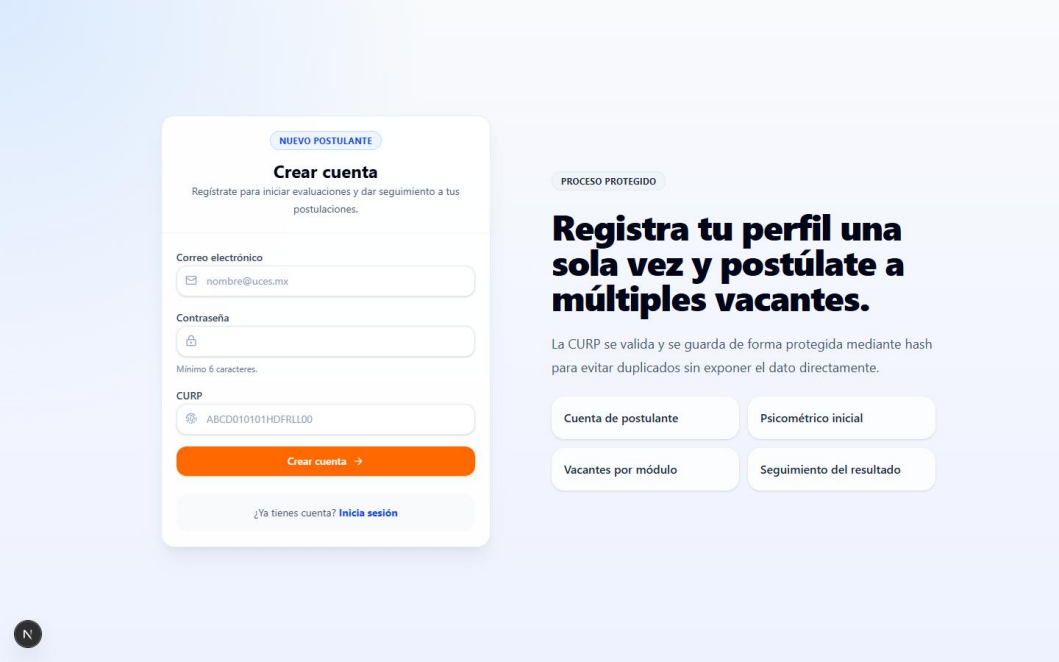


Figura 5.5. Formulario de registro de candidato (/register). Fuente: captura de pantalla propia, 2026.



Figura 5.7. Formulario de inicio de sesión (/login). Fuente: captura de pantalla propia, 2026.

El inicio de sesión utiliza el método estándar de Supabase Auth:

```
// Dentro de app/login/page.tsx (Client Component)
const { error } = await supabase.auth.signInWithPassword({ email, password });
if (!error) router.push("/panel");
```

Una vez iniciada la sesión, el token JWT emitido por Supabase Auth queda almacenado en el almacenamiento local del navegador. Los componentes cliente acceden a él mediante `supabase.auth.getSession()`.

La resolución del rol del usuario ocurre en `lib/auth.ts`:

```
// lib/auth.ts
export const getRoleForSession = async (supabase: SupabaseClient):
Promise<ProfileRole> => {
  const { data, error } = await supabase.rpc("get_my_role");
  if (error) return "user";  // Fallback seguro ante cualquier error
  return resolveRole(typeof data === "string" ? data : null);
};
```

Esta función invoca el RPC `get_my_role()` definido en la base de datos. El RPC, a su vez, consulta primero `user_roles` para el usuario autenticado (`auth.uid()`), y luego cae a `profiles.role` si no encuentra entrada. La función `resolveRole()` aplica la sanitización final: solo acepta los valores literales `"admin"` o `"user"`, retornando `"user"` ante cualquier otro valor. Este triple nivel de defensa (RPC → resolveRole → fallback en error de RPC) garantiza que un valor inesperado en la base de datos no conceda privilegios elevados accidentalmente.

5.4 Desarrollo del panel del candidato

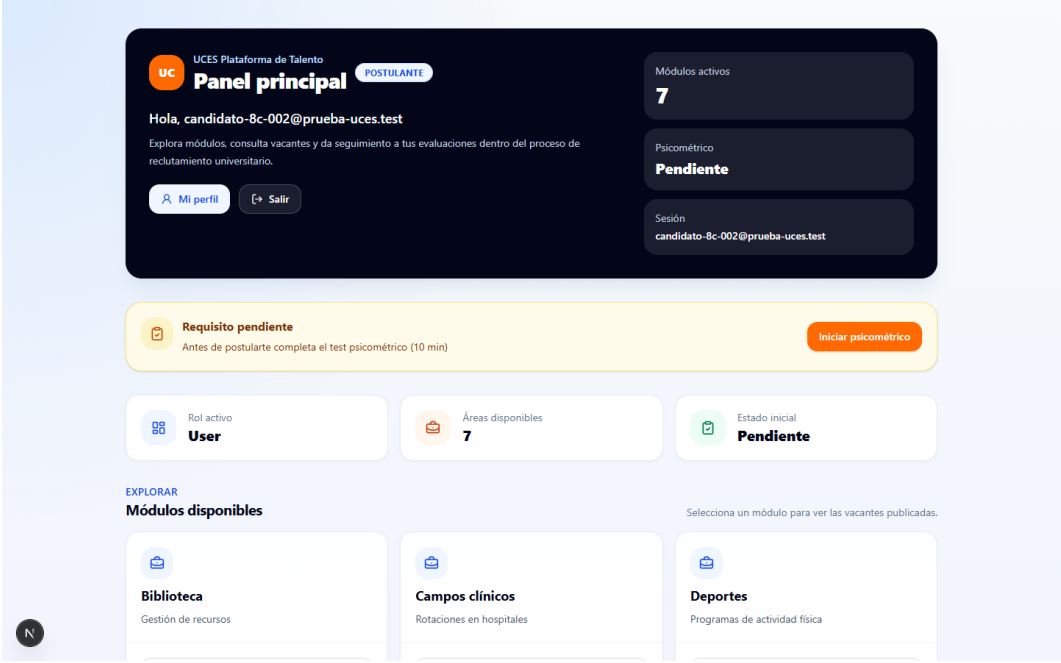


Figura 5.9. Panel principal del candidato autenticado (/panel). Fuente: captura de pantalla propia, 2026.

5.4.1 Arquitectura del panel (RF-04)

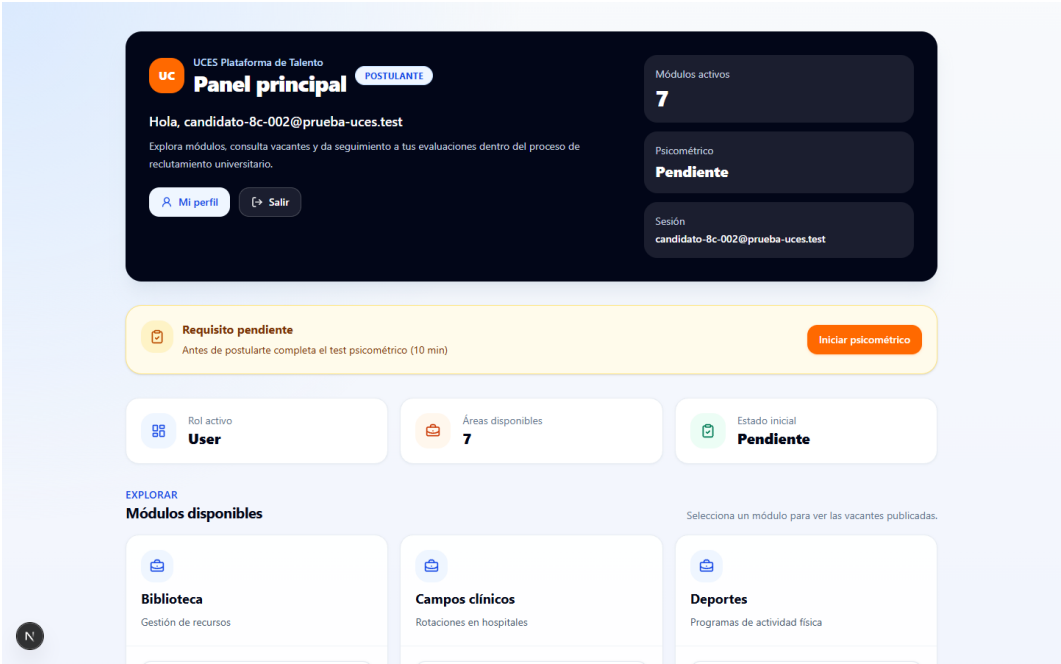


Figura 5.9. Panel principal del candidato autenticado (/panel). Fuente: captura de pantalla propia, 2026.

El panel del candidato (`app/panel/page.tsx`) es un Client Component. Esta decisión se justifica por la naturaleza dinámica de sus datos: la sesión del usuario, el perfil, la lista de módulos, el rol y el estado del psicométrico solo se conocen en tiempo de ejecución en el navegador. Generarlos en el servidor requeriría acceso a la sesión del cliente, lo que complica el modelo de Server Components.

El panel carga sus datos mediante cinco efectos independientes que se ejecutan en secuencia condicionada:

```
Efecto 1: getSession() → Si no hay sesión → redirige a /login
          Si hay sesión → setSession(data.session)
                ↓
Efecto 2: (depende de session) → carga perfil desde profiles
                ↓
Efecto 3: (depende de session) → getRoleForSession() → setUserRole
                ↓
Efecto 4: (depende de session) → carga módulos desde modules
                ↓
Efecto 5: (depende de session) → verifica estado del psicométrico
```

Este patrón de efectos condicionados garantiza que ninguna consulta de datos se ejecute sin una sesión válida. La protección de ruta es del lado del cliente: si la sesión no existe, se redirige inmediatamente a `/login`.

5.4.2 Comportamiento adaptado al rol

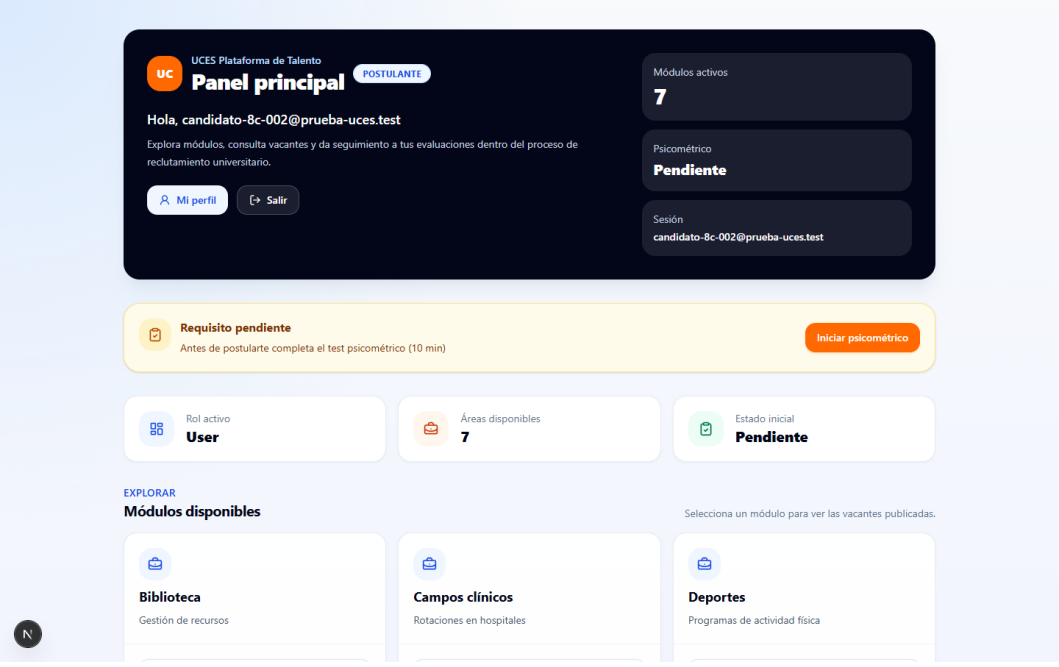


Figura 5.9. Panel principal del candidato autenticado (/panel). Fuente: captura de pantalla propia, 2026.

Una vez que el rol se resuelve, la interfaz adapta su contenido:

```
// app/panel/page.tsx
const isAdmin = userRole === "admin";
// ...
{isAdmin ? (
  <Button asChild>
    <Link href="/admin">Panel administrativo</Link>
  </Button>
) : null}
```

El botón de acceso al panel administrativo solo se renderiza cuando `userRole === "admin"`. Esta verificación del lado del cliente es un mecanismo de usabilidad, no de seguridad: los endpoints de API del área administrativa están protegidos de forma independiente mediante `requireAdmin()`, de modo que incluso si un usuario modificara el HTML del cliente, los Route Handlers rechazarían sus solicitudes.

5.4.3 Indicador de estado psicométrico

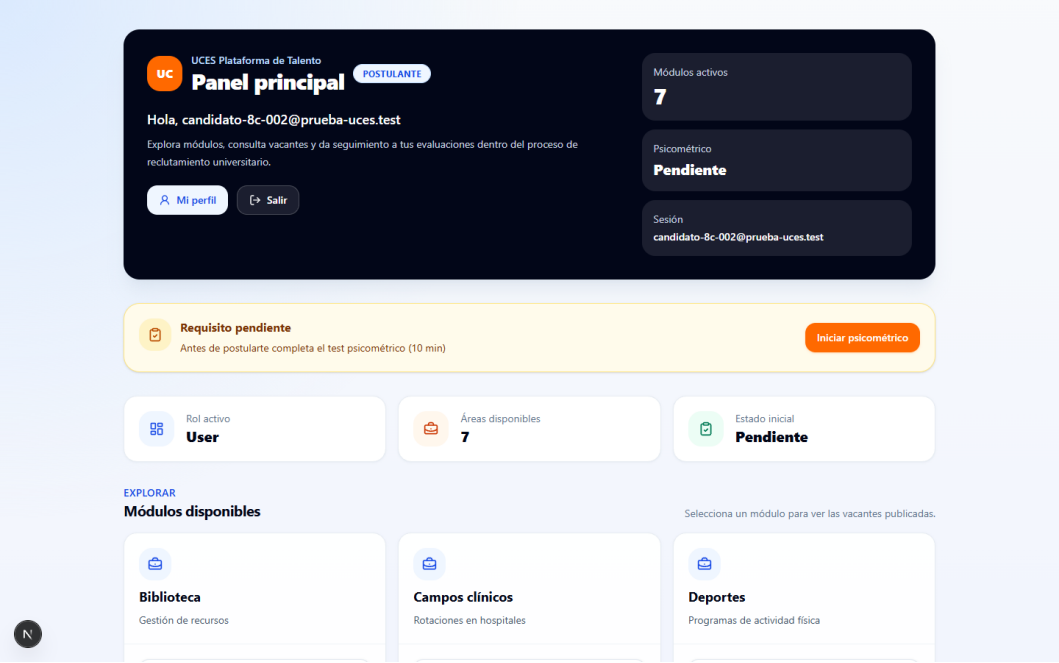


Figura 5.9. Panel principal del candidato autenticado (/panel). Fuente: captura de pantalla propia, 2026.

El panel verifica el estado del cuestionario base consultando directamente la tabla `attempts` con el UUID fijo del psicométrico (`PSICOMETRICO_BASE_ASSESSMENT_ID = "f2000000-0000-0000-0000-000000000001"`):

```
// app/panel/page.tsx
const hasCompleted = attempts.some((attempt) =>
  Boolean(attempt.submitted_at) || completedStatuses.has(attempt.status ?? "")
```

```
);
setPsicometricoReady(hasCompleted);
```

Esta verificación muestra al candidato si puede postularse a vacantes o si primero debe completar el cuestionario psicométrico, sin necesidad de hacer una solicitud adicional a una API: se ejecuta directamente contra Supabase desde el navegador, aprovechando que el cliente anónimo con las políticas de RLS permite al usuario consultar sus propios intentos.

5.5 Gestión y consulta de vacantes

[Figura 5.10. [MARCADOR — EV-5A-11 INVÁLIDA: captura muestra error 404. Requiere nueva captura de /vacantes con lista de vacantes activas.]]

Figura 5.10. [MARCADOR — EV-5A-11 INVÁLIDA: captura muestra error 404. Requiere nueva captura de /vacantes con lista de vacantes activas.]

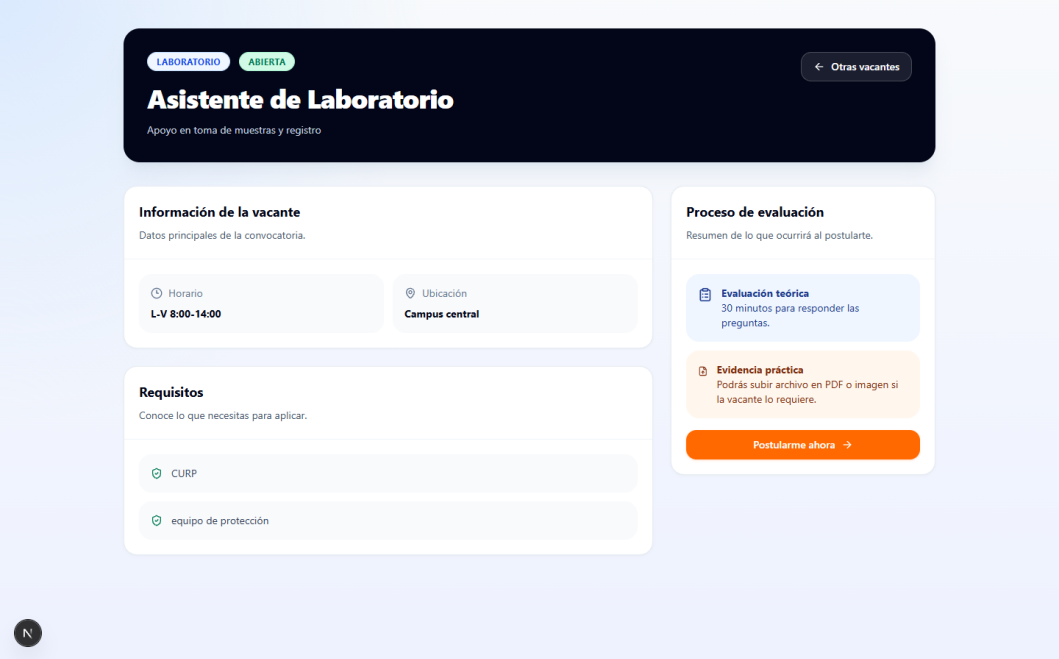


Figura 5.11. Detalle de vacante con descripción y botón de postulación. Fuente: captura de pantalla propia, 2026.

5.5.1 Jerarquía módulos → vacantes (RF-04, RF-05)

[Figura 5.10. [MARCADOR — EV-5A-11 INVÁLIDA: captura muestra error 404. Requiere nueva captura de /vacantes con lista de vacantes activas.]]

Figura 5.10. [MARCADOR — EV-5A-11 INVÁLIDA: captura muestra error 404. Requiere nueva captura de /vacantes con lista de vacantes activas.]

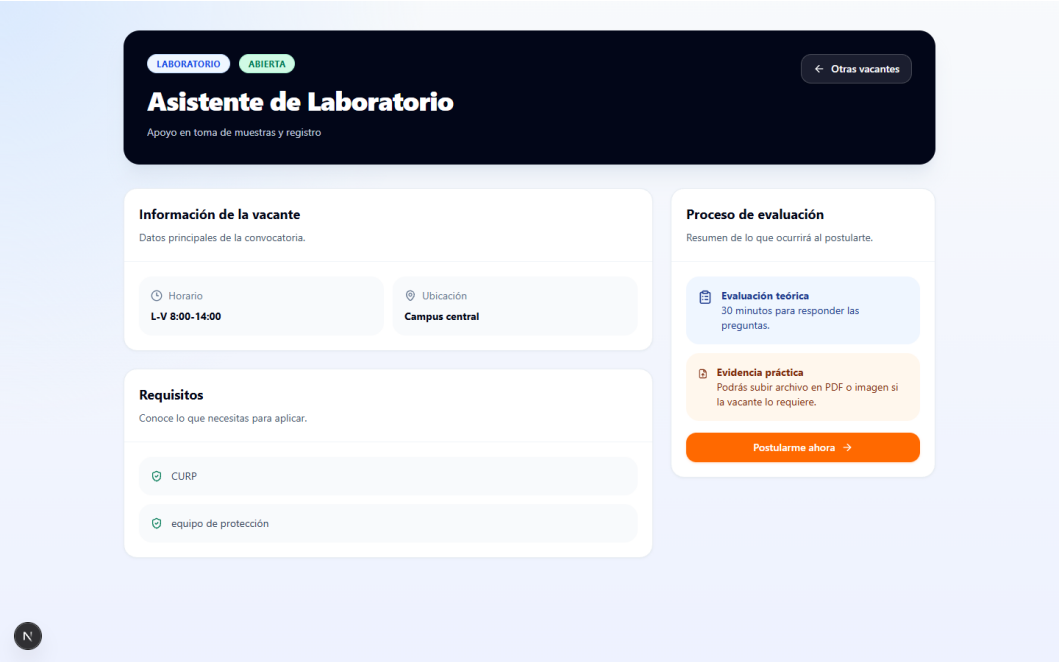


Figura 5.11. Detalle de vacante con descripción y botón de postulación. Fuente: captura de pantalla propia, 2026.

Las vacantes del sistema se organizan en una jerarquía de dos niveles: un módulo (área funcional) agrupa múltiples vacantes. El candidato navega primero a la lista de módulos en el panel, luego elige un módulo para ver sus vacantes, y finalmente accede al detalle de una vacante específica.

Ruta	Componente	Datos que carga
`/panel`	Client Component	Módulos: `id`, `name`, `description`
`/modulos/[moduleId]`	Server Component	Módulos con vacantes anidadas: `vacancies(id, title, status, schedule, location)`
`/vacantes/[id]`	Server Component	Detalle de vacante: descripción, requisitos, assessment asociado

Las páginas `/modulos/[moduleId]` y `/vacantes/[id]` son Server Components que realizan sus consultas directamente en el servidor, sin exponer las credenciales al navegador:

```
// app/modulos/[moduleId]/page.tsx (simplificado)
const supabase = supabaseServer();
```

```
const { data: module } = await supabase
  .from("modules")
  .select("id, name, description, vacancies(id, title, status, schedule, location)")
  .eq("id", moduleId)
  .maybeSingle();
```

5.5.2 Estados de vacante y disponibilidad



Figura 5.4. Vista de la página de inicio pública del sistema (/). Fuente: captura de pantalla propia, 2026.

[Figura 5.10. [MARCADOR — EV-5A-11 INVÁLIDA: captura muestra error 404. Requiere nueva captura de /vacantes con lista de vacantes activas.]]

Figura 5.10. [MARCADOR — EV-5A-11 INVÁLIDA: captura muestra error 404. Requiere nueva captura de /vacantes con lista de vacantes activas.]

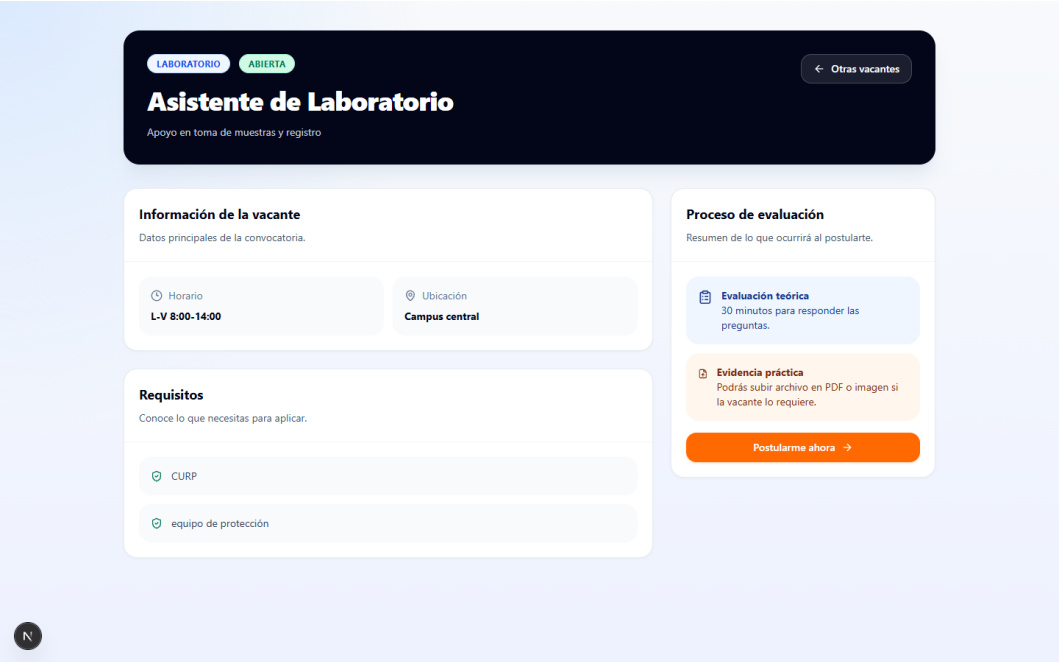


Figura 5.11. Detalle de vacante con descripción y botón de postulación. Fuente: captura de pantalla propia, 2026.

Las vacantes tienen tres estados posibles definidos en la base de datos: `open`, `closed` y `draft`. Solo las vacantes con estado `open` son accesibles para postulación. El componente de la página de vacante verifica el estado antes de mostrar el botón de postulación:

```
// app/vacantes/[id]/page.tsx
const isOpen = vacancy.status === "open";
// ...
<ApplyButton vacancyId={vacancy.id} disabled={!isOpen} />
```

La verificación del lado del servidor ocurre adicionalmente en el Route Handler de postulación (sección 5.6), que rechaza cualquier intento de postulación a una vacante no abierta.

5.6 Implementación del proceso de postulación

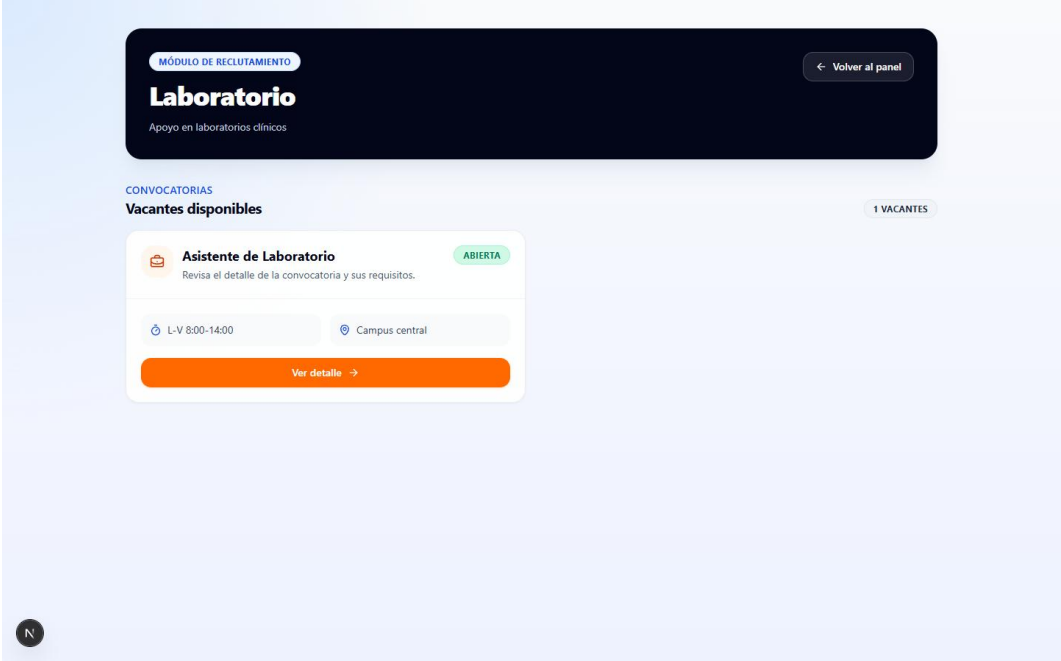


Figura 5.12. Interfaz de subida de archivo de evidencia en módulo activo. Fuente: captura de pantalla propia, 2026.

5.6.1 Flujo completo del endpoint de postulación (RF-06, RF-07)

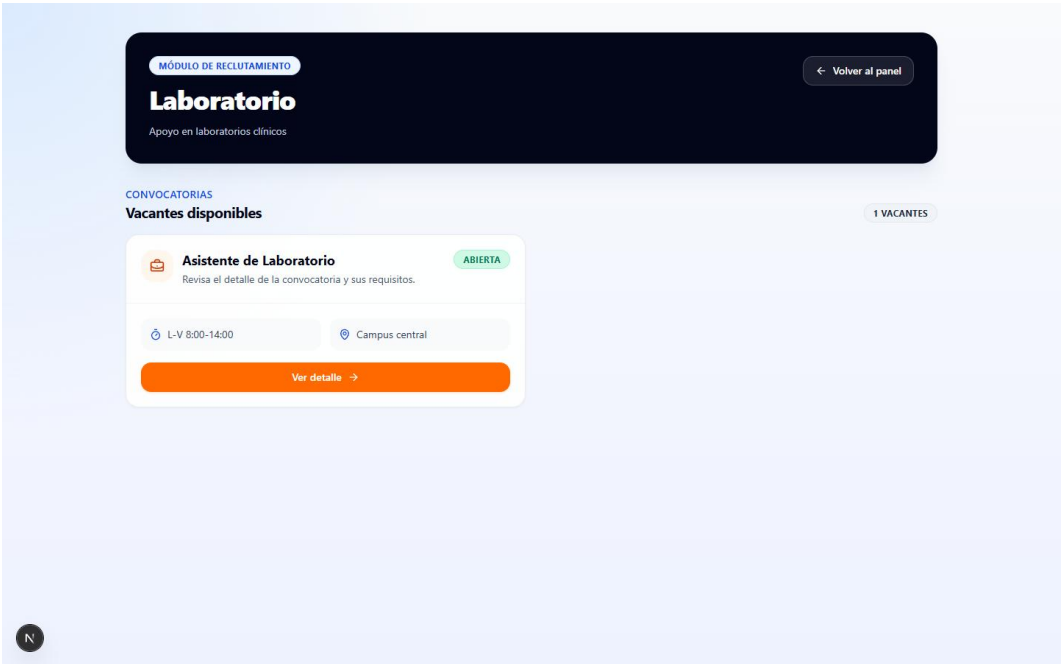


Figura 5.12. Interfaz de subida de archivo de evidencia en módulo activo. Fuente: captura de pantalla propia, 2026.

El Route Handler `POST /api/vacancies/[id]/apply` implementa la secuencia de verificaciones más elaborada del flujo del candidato. El handler ejecuta seis pasos en orden secuencial, donde cada paso puede detener el proceso con un error descriptivo.

Figura 5.3 — Diagrama de secuencia de la postulación

```
sequenceDiagram
    participant UI as ApplyButton<br/>(postular.tsx)
    participant API as POST /api/vacancies/[id]/apply
    participant Psico as lib/psicometrico.ts
    participant DB as PostgreSQL

    UI->>API: POST {Bearer JWT} (sin body)
    API->>API: Extrae y valida Bearer token
    API->>DB: auth.getUser(token) → userId
    API->>Psico: getPsicometricoAttemptState(supabase, userId)
    Psico->>DB: SELECT attempts WHERE assessment_id = UUID_FIJO AND user_id = userId
    DB-->>Psico: lista de intentos psicométricos
    alt Psicométrico NO completado
        API-->>UI: 403 {error, requiresPsychometric: true, redirectTo:
"/psicometrico"}
        UI->>UI: router.push("/psicometrico")
    else Psicométrico completado
        API->>DB: SELECT vacancies WHERE id = vacancyId
        alt Vacante no abierta
            API-->>UI: 400 "Esta vacante no está abierta"
        end
        API->>DB: SELECT assessments WHERE vacancy_id = vacancyId
        alt Sin evaluación configurada
            API-->>UI: 400 "La vacante no tiene evaluación configurada"
        end
        API->>DB: SELECT applications WHERE user_id = userId AND vacancy_id =
vacancyId
        alt Sin postulación previa
            API->>DB: INSERT INTO applications → applicationId
        end
        API->>DB: SELECT attempts WHERE application_id = applicationId ORDER BY
started_at DESC
        alt Existe intento no enviado
            API-->>UI: 200 {attemptId: existingAttempt.id}
        else No hay intento activo
            API->>DB: INSERT INTO attempts {application_id, assessment_id,
started_at, deadline_at}
            API-->>UI: 200 {attemptId: newAttempt.id}
        end
    end
    UI->>UI: router.push("/evaluacion/" + attemptId)
```

Paso 1 — Autenticación del token:

El endpoint extrae el JWT del encabezado `Authorization`, lo valida mediante `supabase.auth.getUser(token)` y extrae el `userId`. Si el token es inválido o expiró, devuelve 401.

Paso 2 — Verificación del psicométrico:

La función `getPsicometricoAttemptState()` consulta todos los intentos del usuario para el assessment con UUID fijo `f2000000-0000-0000-0000-000000000001`. Determina si existe al menos uno con `submitted_at` no nulo o con estado en el conjunto `{SUBMITTED, UNDER_REVIEW, COMPLETED}`.

Punto importante: la verificación comprueba únicamente que el psicométrico esté **completado** (enviado), no que el candidato haya obtenido un resultado favorable en ningún rasgo. Esta diferencia respecto a lo que podría esperarse del diseño es la discrepancia D-01 documentada en el Trabajo B. Su justificación de implementación es que el sistema no tiene una regla de negocio formalizada sobre qué puntaje constituye un umbral de exclusión; los umbrales son orientativos para el administrador, no bloqueos automatizados.

Paso 3 — Verificación del estado de la vacante:

La consulta a `vacancies` verifica `vacancy.status === "open"`. Esta verificación ocurre en el servidor, independientemente de lo que muestre la interfaz.

Paso 4 — Verificación del assessment:

La consulta a `assessments` verifica que la vacante tiene un assessment asociado con un `assessment_id` válido. El sistema no puede crear un intento sin un assessment de referencia.

Paso 5 — Creación o recuperación de postulación:

El handler consulta si ya existe una `application` para el par `(user_id, vacancy_id)`. Si existe, reutiliza su `id`; si no, la crea. Esto implementa la regla RN-05: no crear postulaciones duplicadas.

Paso 6 — Creación o recuperación del intento:

Si existe un intento asociado a la aplicación que aún no fue enviado (`!existingAttempt.submitted_at`), el handler devuelve su ID directamente sin crear un nuevo intento. Esto permite que un candidato que cerró el navegador pueda retomar su evaluación. Si no hay intento activo, se crea uno nuevo calculando `deadline_at = started_at + (duration_minutes * 60000)`.

Lado del cliente — `ApplyButton`:

```
// app/vacantes/[id]/postular.tsx
const response = await fetch(`/api/vacancies/${vacancyId}/apply`, {
  method: "POST",
```

```
  headers: { Authorization: `Bearer ${token}` },
});
const payload = await response.json();

if (!response.ok) {
  if (payload.requiresPsychometric) {
    router.push(payload.redirectTo ?? "/psicometrico");
  } else {
    setError(payload.error ?? "No se pudo completar la postulación");
  }
  return;
}
router.push(`/evaluacion/${payload.attemptId}`);
```

El `ApplyButton` maneja dos tipos de error diferenciados: si la respuesta incluye `requiresPsychometric: true`, redirige al candidato a completar el cuestionario base; de lo contrario, muestra el mensaje de error en la interfaz.

5.7 Desarrollo del módulo de evaluaciones

5.7.1 División Server Component / Client Component

La ruta `/evaluacion/[attemptId]` implementa la separación más clara del sistema entre carga de datos (servidor) e interacción (cliente). Este diseño tiene una justificación técnica precisa: las preguntas de la evaluación deben cargarse en el servidor para garantizar que el usuario recibe el contenido correcto antes de que el temporizador comience a correr en el cliente.

`page.tsx` — Server Component: Carga en paralelo el intento con sus preguntas, las respuestas guardadas previamente, y las evidencias existentes mediante `Promise.all`. Construye el `answersMap` (mapa de `question_id → selected_option`) para inicializar el estado del cliente con las respuestas ya guardadas en sesiones anteriores. Pasa todos estos datos como props al `EvaluationClient`.

`cliente.tsx` — Client Component: Gestiona el estado local de la evaluación: respuestas en memoria (`useState`), cuenta regresiva del temporizador (`setRemainingMs`), estado de subida de evidencia, y estado de envío.

La inicialización del estado del temporizador es inmediata:

```
// app/evaluacion/[attemptId]/cliente.tsx
const [remainingMs, setRemainingMs] = useState(() =>
  new Date(deadlineAt).getTime() - Date.now()
);
```

Al calcular `remainingMs` en la inicialización del estado (no en un efecto), el temporizador refleja desde el primer render el tiempo real restante, basado en el `deadline_at` que provino del servidor.

5.7.2 Mecanismo del temporizador

El temporizador se implementa con `setInterval` en un efecto de React:

```
// app/evaluacion/[attemptId]/cliente.tsx (fragmento del useEffect del timer)
const interval = setInterval(() => {
  const nextRemainingMs = new Date(deadlineAt).getTime() - Date.now();
  setRemainingMs(nextRemainingMs);

  if (nextRemainingMs <= 0 && token && !submittedAt &&
!autoSubmitTriggeredRef.current) {
    autoSubmitTriggeredRef.current = true;  // Previene doble envío
    void handleSubmitAttempt(true);
  }
}, 1000);
```

El intervalo recalcula `nextRemainingMs` en cada tick usando la fecha absoluta `deadlineAt` (proveniente del servidor), no una cuenta regresiva relativa. Esto previene que pequeñas desviaciones en la ejecución del intervalo acumulen errores a lo largo del tiempo. La referencia `autoSubmitTriggeredRef` es un mecanismo anti-duplicado: el envío automático solo se activa una vez, aunque el intervalo siga corriendo brevemente después.

La pantalla del candidato muestra el tiempo en formato `MM:SS` con el badge cambiando a variante `destructive` (rojo) cuando el tiempo se agota. Los botones de selección de respuesta y carga de evidencia se deshabilitan cuando `isExpired === true`:

```
<button
  disabled={isExpired || savingQuestion === question.id}
  onClick={() => handleSelect(question.id, optionKey)}
>
```

Distinción cliente/servidor en el control de tiempo:

La interfaz deshabilita visualmente las respuestas cuando el tiempo expira. Sin embargo, la verificación definitiva ocurre en el servidor: el Route Handler de respuestas (`answers/route.ts`) verifica `Date.now() > new Date(attempt.deadline_at).getTime()` antes de procesar cada guardado. Esto garantiza que incluso si el candidato manipulara el cliente para saltar la restricción visual, el servidor rechazaría las respuestas tardías.

5.7.3 Guardado individual de respuestas (RF-08, RF-09)

Cada vez que el candidato selecciona una opción, el evento llama a `handleSelect`, que actualiza el estado local inmediatamente y luego invoca `saveAnswer` de forma asíncrona:

```
const handleSelect = (questionId: string, optionKey: string) => {
  setAnswers((prev) => ({ ...prev, [questionId]: optionKey }));  // UI instantánea
  void saveAnswer(questionId, optionKey);                         // Guardado
asíncrono
};
```

La actualización del estado local es inmediata, lo que da retroalimentación visual instantánea al candidato sin esperar la confirmación del servidor. El Route Handler `POST /api/attempts/[attemptId]/answers` realiza un `UPSERT` sobre la tabla `answers`:

```
// app/api/attempts/[attemptId]/answers/route.ts
// Validaciones previas al upsert:
// 1. JWT válido → userId
// 2. attempt.application.user_id === userId (pertenencia)
// 3. !attempt.submitted_at (no enviado aún)
// 4. Date.now() <= deadline_at (tiempo vigente)
// 5. question.assessment_id === attempt.assessment_id (pregunta pertenece a la
evaluación)

await supabase.from("answers").upsert({
  attempt_id: attemptId,
  question_id: body.questionId,
  selected_option: body.selectedOption,
});
```

La restricción `UNIQUE (attempt_id, question_id)` en la tabla `answers` garantiza que el UPSERT actualice la respuesta existente en lugar de crear un duplicado. La validación de que la pregunta pertenece al assessment del intento previene que un candidato intente guardar respuestas de otro cuestionario en un intento que no le corresponde.

5.7.4 Estados del intento

La columna `status` en la tabla `attempts` sigue el ciclo definido en el esquema:

Estado	Significado	Quién lo establece
`IN_PROGRESS`	Intento en curso (valor por defecto)	Creación del registro
`SUBMITTED`	Enviado por el candidato, pendiente de revisión	`submit/route.ts` → `UNDER_REVIEW`
`UNDER_REVIEW`	En revisión por el administrador	`submit/route.ts` al enviar
`COMPLETED`	Revisión terminada con	Panel admin al aprobar o

	decisión final	rechazar

5.7.5 Envío del intento (RF-11)

El Route Handler `POST /api/attempts/[attemptId]/submit` (analizado en profundidad en el Capítulo III, sección 3.7) realiza tres operaciones al recibir el envío:

18. Verifica que el intento no fue enviado previamente (`!attempt.submitted_at`).
19. Calcula el resumen psicométrico y el `theory_score` a partir de las respuestas guardadas.
20. Actualiza el registro del intento con `submitted_at`, `status = UNDER_REVIEW`, `theory_score` y `psychometric_summary`.

El `theory_score` se calcula mediante la función `calculatePsicometricoSummary()` para cualquier tipo de intento, tanto psicométrico como de conocimientos. La función acumula respuestas únicamente para preguntas cuyo `prompt` comienza con el prefijo `MINI_BIG_FIVE`; el divisor `likertQuestions` está codificado en 20. Para el cuestionario psicométrico, esto produce un valor entre 0 y 100 que representa el porcentaje de los 20 reactivos que el candidato respondió (p. ej., si respondió los 20, `theory_score = 100`).

Para evaluaciones de conocimientos, cuyas preguntas no tienen el prefijo `MINI_BIG_FIVE`, el acumulador `likertAnswered` resulta siempre 0 y, en consecuencia, `theory_score` queda registrado como `0`. No existe en el endpoint ninguna ruta de cálculo alternativa que evalúe respuestas correctas comparando `selected_option` con `correct_option`; ese campo no refleja la exactitud de respuestas en evaluaciones de conocimientos. Esta es una **limitación funcional del MVP**: el puntaje de aciertos en evaluaciones de conocimientos no se calcula automáticamente en el momento del envío y deberá incorporarse en una iteración futura.

5.8 Implementación de las evidencias prácticas

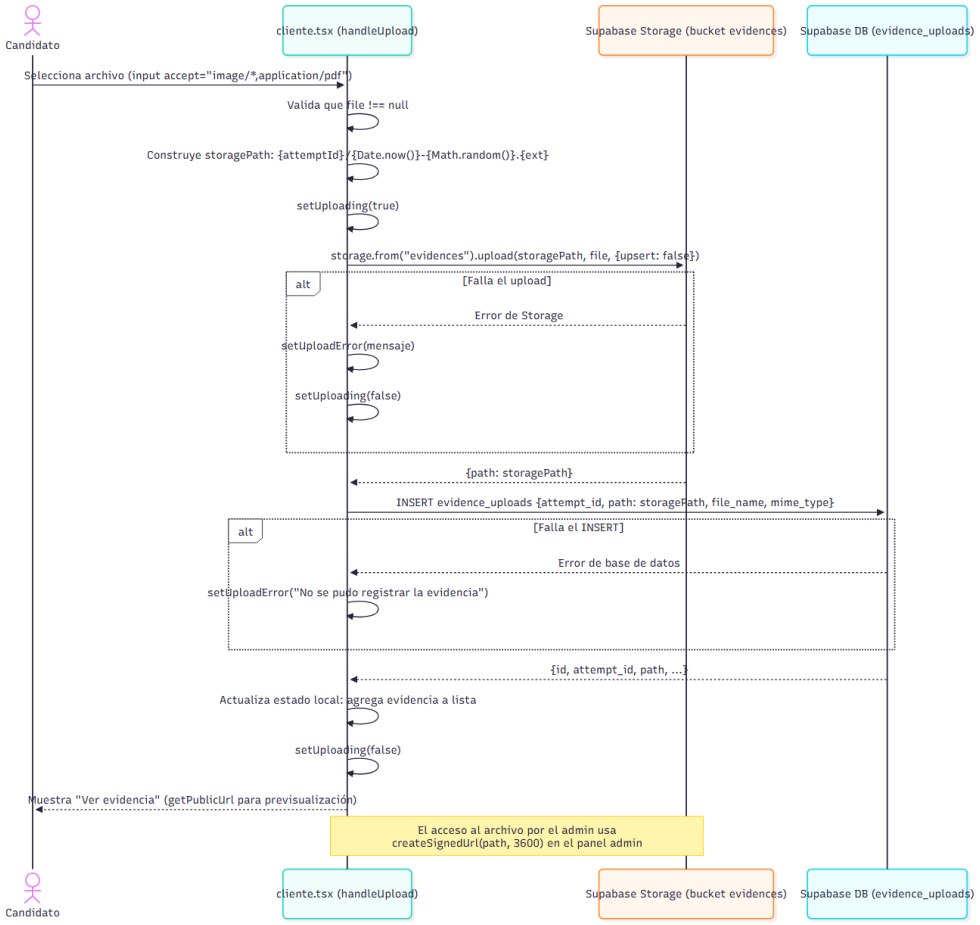


Figura 5.8. Diagrama de secuencia: subida de archivo de evidencia. Fuente: elaboración propia, 2026.

5.8.1 Flujo de carga de evidencia (RF-10)

La carga de evidencias ocurre directamente desde el navegador al bucket `evidences` de Supabase Storage, utilizando el cliente anónimo autenticado con el JWT del candidato. No pasa por el servidor Next.js, lo que evita consumir la memoria del servidor para transferencias de archivos.

Figura 5.4 — Flujo de almacenamiento de evidencias

```
sequenceDiagram
    participant UI as EvaluationClient<br/>(cliente.tsx)
    participant Storage as Supabase Storage<br/>(bucket: evidences)
    participant DB as PostgreSQL<br/>(evidence_uploads)

    UI->>UI: Usuario selecciona archivo (image/* o application/pdf)
    UI->>UI: Verifica !isExpired (tiempo vigente)
    UI->>UI: Construye storagePath = {attemptId}/{timestamp}-{random}.{ext}
```

```
    UI->>Storage: supabase.storage.from("evidences").upload(storagePath, file,
{upsert: false})
    alt Error en upload
        Storage-->>UI: uploadError
        UI->>UI: setStatusMessage("No se pudo subir el archivo")
    else Upload exitoso
        Storage-->>UI: OK
        UI->>DB: INSERT INTO evidence_uploads {attempt_id, path, mime_type, size}
        alt Error en registro
            DB-->>UI: recordError
            UI->>UI: setStatusMessage("No se pudo registrar la evidencia")
        else Registro exitoso
            DB-->>UI: {path, mime_type, size, created_at}
            UI->>UI: setEvidence(record)
            UI->>UI: setStatusMessage("Evidencia subida correctamente")
        end
    end
```

5.8.2 Generación del path de almacenamiento

```
// app/evaluacion/[attemptId]/cliente.tsx
const extension = file.name.split(".").pop();
const storagePath = `${attemptId}/${Date.now()}-
${Math.random().toString(36).slice(2)}${extension ? `.${extension}` : ""}`;
```

El path del archivo combina tres elementos: el `attemptId` como prefijo de carpeta (lo que organiza todos los archivos de un mismo intento juntos), el timestamp en milisegundos (que garantiza orden temporal), y una cadena aleatoria en base 36 (que previene colisiones si dos archivos se suben en el mismo milisegundo). Esta estructura evita que archivos de intentos diferentes compartan rutas.

5.8.3 Tipo de archivo y validación

El elemento de entrada de archivo acepta imágenes y PDFs mediante el atributo HTML:

```
<input type="file" accept="image/*,application/pdf" />
```

Esta restricción opera en el navegador y puede ser omitida por un usuario técnico que modifique el DOM. No existe validación adicional del tipo MIME en el Route Handler de evidencias porque la carga ocurre directamente a Supabase Storage, no a través de un endpoint propio. El tipo MIME reportado por el navegador se almacena como `file.type` en el campo `mime_type` de `evidence_uploads`.

La política de RLS en el bucket `evidences` restringe la subida a usuarios autenticados que sean propietarios del archivo (`owner = auth.uid()`), lo que previene que un candidato suba archivos al espacio de otro. Sin embargo, no valida el tipo de contenido del archivo binario; esta es una limitación documentada del MVP.

5.8.4 Tabla evidence_uploads

Columna	Tipo	Descripción
`id`	uuid	Identificador del registro
`attempt_id`	uuid → attempts	Intento al que pertenece la evidencia
`bucket`	text	Nombre del bucket (`evidences` por defecto)
`path`	text	Ruta dentro del bucket para construir la URL
`mime_type`	text	Tipo MIME declarado por el navegador
`size`	integer	Tamaño en bytes
`created_at`	timestamptz	Momento del registro

El campo `path` es el dato primario para acceder al archivo: permite al administrador generar una URL firmada temporal mediante la API de Supabase Storage. En el cliente de la evaluación, se usa `supabase.storage.from("evidences").getPublicUrl(evidence.path)` para mostrar un enlace al archivo durante la evaluación activa, aunque el bucket es privado y la URL pública solo funciona durante la sesión autenticada del propietario.

5.8.5 Integración con el panel administrativo

El panel administrativo recupera las evidencias de un intento y genera URLs firmadas con vigencia de 60 minutos para que el administrador pueda acceder a ellas:

```
// app/api/admin/attempts/[id]/detail/route.ts (fragmento)
const { data: signedUrl } = await supabase.storage
  .from("evidences")
  .createSignedUrl(upload.path, 3600);  // 3600 segundos = 60 minutos
```

Las URLs firmadas son la forma correcta de acceder a archivos en un bucket privado: contienen una firma criptográfica que caduca en el tiempo especificado. El administrador puede abrir el archivo durante ese período; al expirar, necesitaría solicitar una nueva URL.

5.9 Implementación del panel administrativo

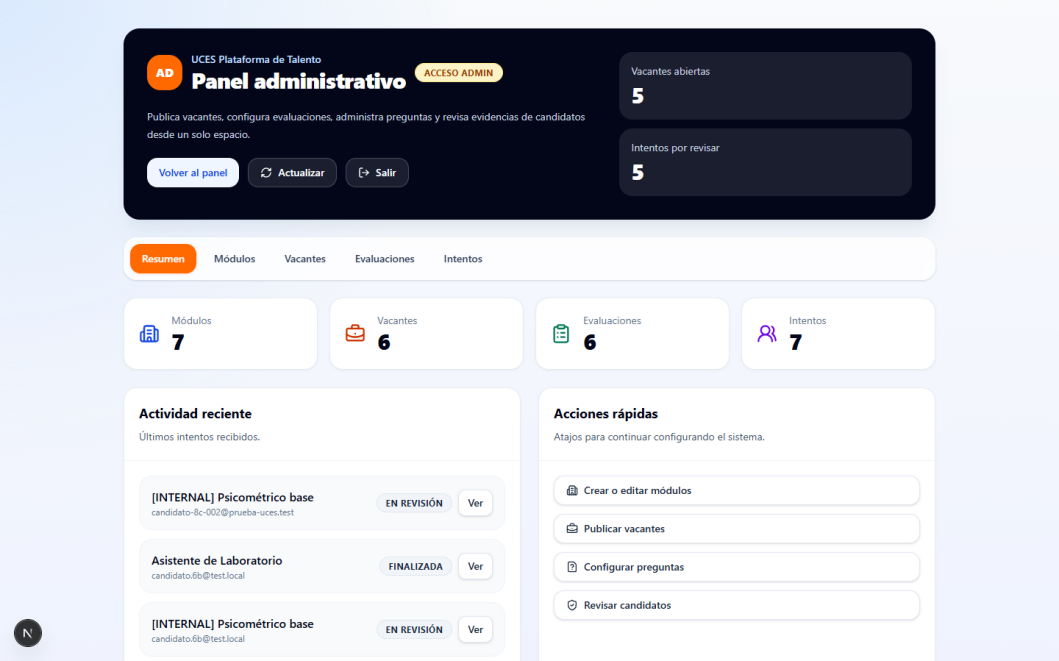


Figura 5.13. Panel de administración (/admin) con lista de candidatos y postulaciones. Fuente: captura de pantalla propia, 2026.

5.9.1 Arquitectura del panel y autenticación del administrador (RF-13 a RF-18)

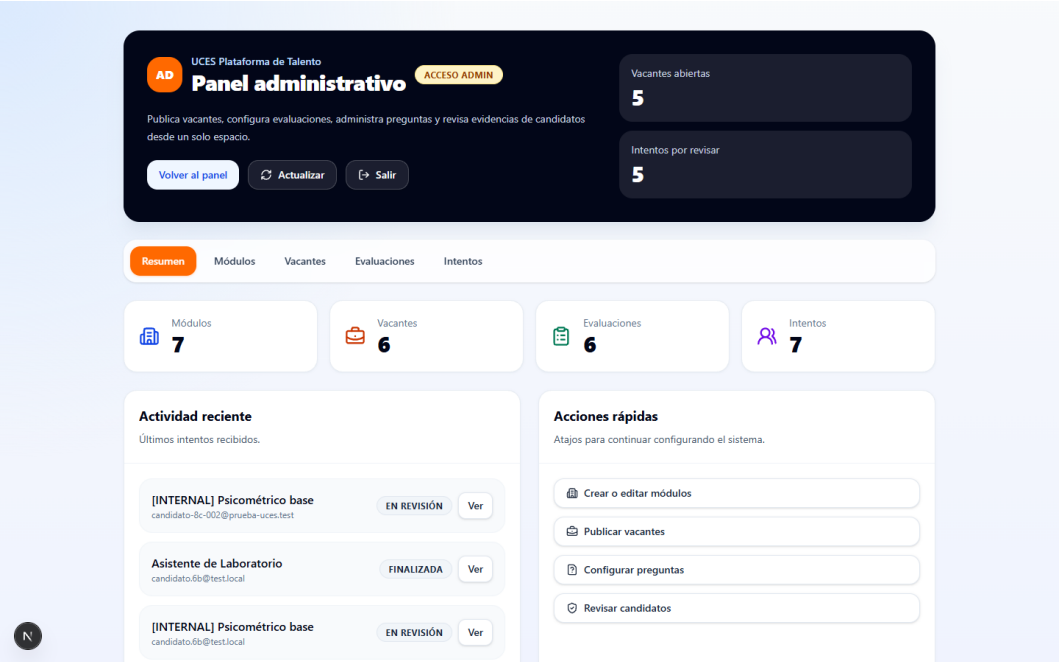


Figura 5.13. Panel de administración (/admin) con lista de candidatos y postulaciones. Fuente: captura de pantalla propia, 2026.

El panel administrativo se implementa en dos páginas: `app/admin/page.tsx`, que concentra las operaciones de configuración y consulta general, y `app/admin/attempts/[id]/page.tsx`, que gestiona la revisión individual de cada intento. Ambas son Client Components que ejecutan la misma secuencia de verificación al montarse:

```
// app/admin/page.tsx — verificación inicial
supabase.auth.getSession().then(async ({ data }) => {
  if (!data.session) { router.replace("/login"); return; }
  const role = await getRoleForSession(supabase);
  if (role !== "admin") { router.replace("/panel"); return; }
  setToken(data.session.access_token);
  setLoading(false);
});
```

La doble verificación —sesión activa y rol admin— garantiza que solo los administradores accedan a la interfaz. El token de acceso se almacena en el estado local del componente y se inyecta en cada solicitud a los endpoints administrativos como encabezado `Authorization: Bearer {token}`. Para esto se define un helper interno:

```
// app/admin/page.tsx — helper de petición autenticada
const callAdmin = async <T,>(path: string, init?: RequestInit): Promise<T> => {
  if (!token) throw new Error("Sin sesión");
  const response = await fetch(path, {
    ...init,
    headers: { "Content-Type": "application/json", authorization: `Bearer ${token}`,
...(init?.headers ?? {}) },
  });
  const json = (await response.json().catch(() => ({}))) as T & { error?: string };
  if (!response.ok) throw new Error(json.error ?? "Error inesperado");
  return json;
};
```

Este helper centraliza el manejo del token y el lanzamiento de errores, reduciendo la duplicación en las llamadas individuales a cada endpoint.

5.9.2 Estructura de navegación por pestañas

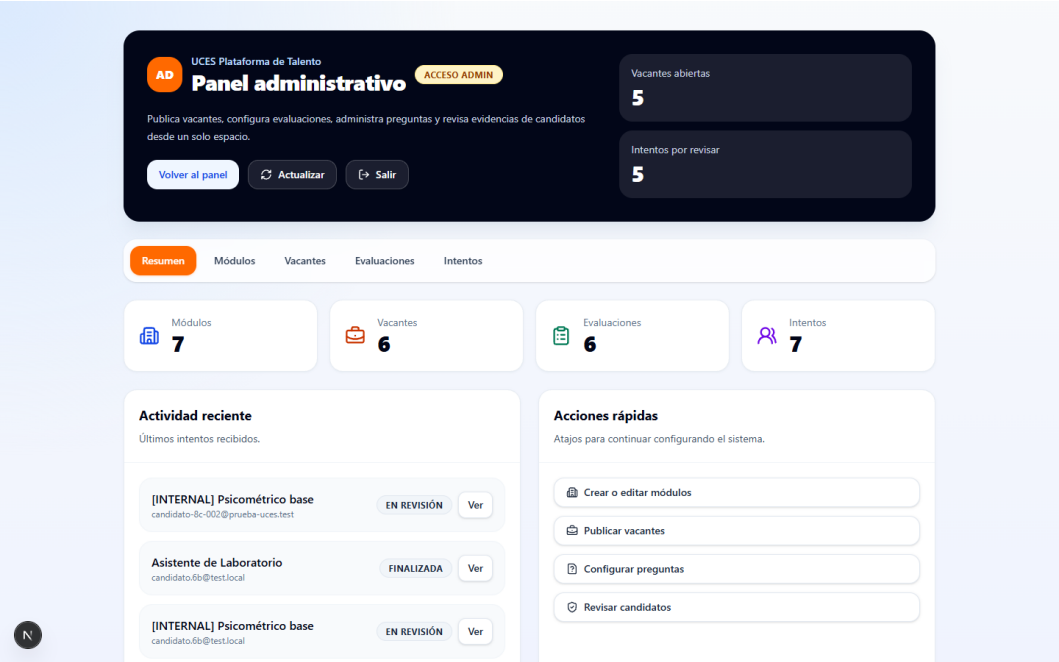


Figura 5.13. Panel de administración (/admin) con lista de candidatos y postulaciones. Fuente: captura de pantalla propia, 2026.

El panel organiza sus funciones en cinco pestañas: `resumen`, `modulos`, `vacantes`, `evaluaciones` e `intentos`. La navegación entre ellas no implica cambios de ruta: el estado `activeTab` determina qué sección se renderiza dentro del mismo componente.

Pestaña	**Función principal**	**Endpoints consumidos**
Resumen	Contadores globales y últimos 5 intentos	Todos (carga paralela)
Módulos	CRUD de módulos	`GET/POST /api/admin/modules`, `PATCH /api/admin/modules/[id]`
Vacantes	CRUD de vacantes con filtro	`GET/POST /api/admin/vacancies`, `PATCH/DELETE /api/admin/vacancies/[id]`
Evaluaciones	Edición de evaluaciones y CRUD de preguntas	`PATCH /api/admin/assessments/[id]`, `GET/POST /api/admin/assessments/[id]/questions`, `PATCH/DELETE /api/admin/questions/[id]`

Intentos	Lista filtrable de intentos con acceso al detalle	`GET /api/admin/attempts`, → `/admin/attempts/[id]`

Al inicializarse el panel, se cargan los cuatro conjuntos de datos en paralelo mediante `Promise.all([loadModules(), loadVacancies(), loadAssessments(), loadAttempts()])`. Esta carga en paralelo reduce el tiempo de espera inicial al eliminar la dependencia secuencial entre las cuatro solicitudes.

5.9.3 Gestión de módulos (RF-13)

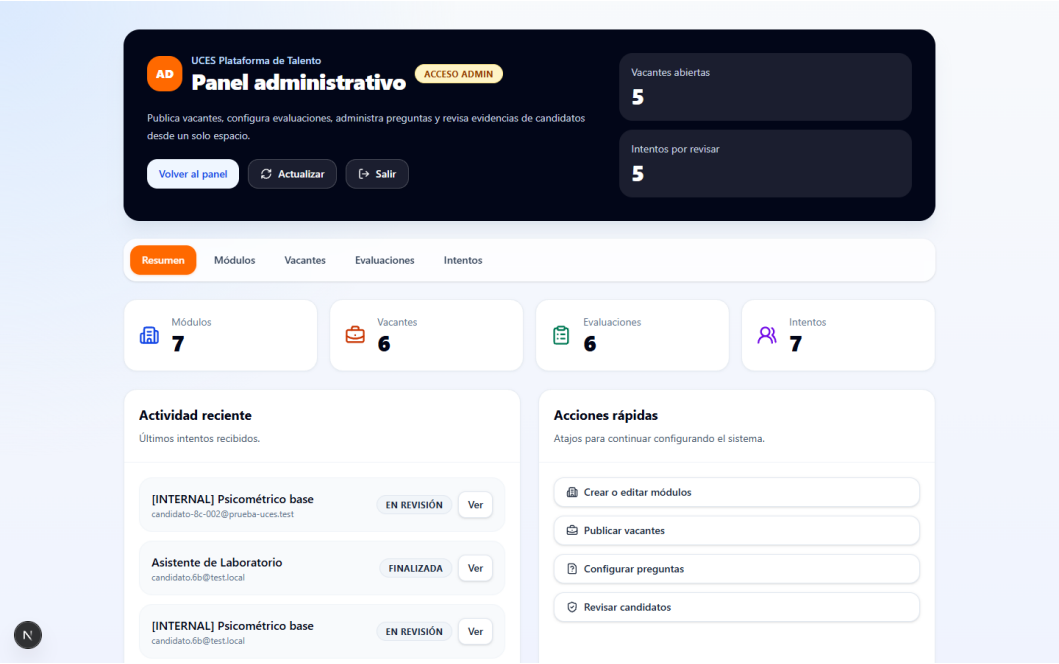


Figura 5.13. Panel de administración (/admin) con lista de candidatos y postulaciones. Fuente: captura de pantalla propia, 2026.

Los módulos son las unidades organizativas que agrupan vacantes. El endpoint `GET /api/admin/modules` acepta un parámetro de búsqueda opcional `?search=` que aplica un filtro `ILIKE` sobre el nombre. La creación y edición se realizan con un formulario único: cuando el campo `id` del formulario está vacío, se ejecuta `POST`; cuando contiene el ID de un módulo existente, se ejecuta `PATCH`.

La implementación del endpoint POST incluye un mecanismo de tolerancia para la columna `sort_order`: si el intento de inserción con esa columna produce un error de columna inexistente, se reintenta la inserción sin ella. Este patrón de compatibilidad descendente se

repite en varios endpoints del sistema para manejar diferencias entre esquemas de desarrollo y producción.

5.9.4 Gestión de vacantes (RF-14)

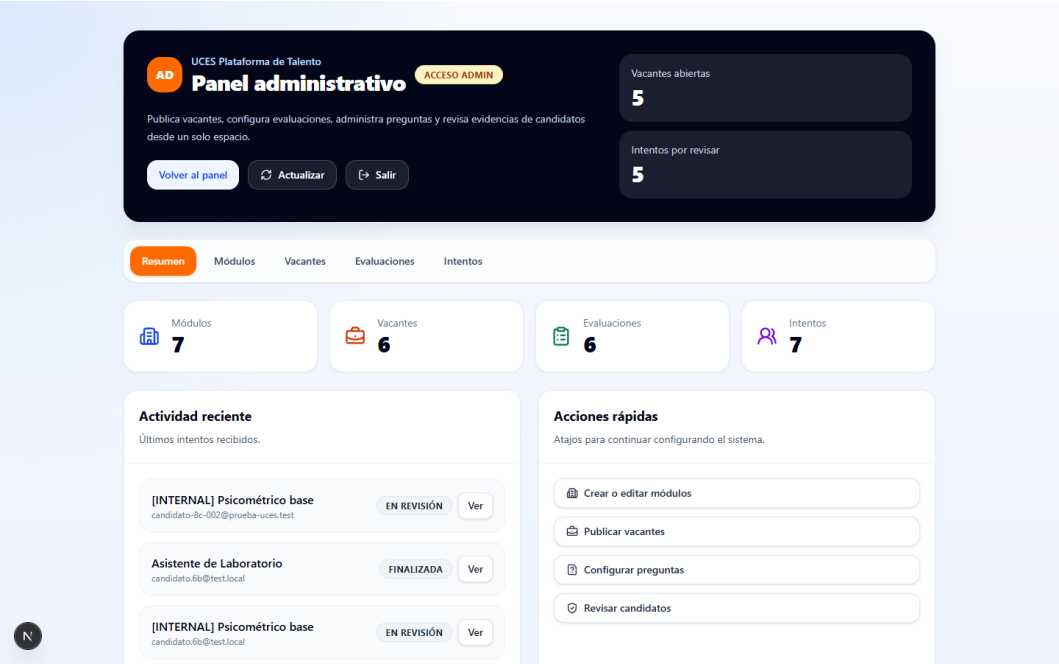


Figura 5.13. Panel de administración (/admin) con lista de candidatos y postulaciones. Fuente: captura de pantalla propia, 2026.

El proceso de creación de una vacante involucra dos operaciones coordinadas en el servidor: primero se crea el registro en `vacancies` y, si la inserción es exitosa, se crea automáticamente una evaluación base asociada en `assessments`:

```
// app/api/admin/vacancies/route.ts — POST handler
const vacancyResponse = await
admin.supabase.from("vacancies").insert(payload).select("*").single();
// ...
const assessmentResponse = await admin.supabase.from("assessments").insert({
  vacancy_id: vacancyResponse.data.id,
  title: `Evaluación - ${vacancyResponse.data.title}`,
  duration_minutes: 30,
}).select("*").single();

if (assessmentResponse.error) {
  await admin.supabase.from("vacancies").delete().eq("id", vacancyResponse.data.id);
  return NextResponse.json({ error: "No se pudo crear la evaluación automática" }, {
status: 400 });
}
```

Este diseño garantiza que toda vacante tenga una evaluación desde el momento de su creación, eliminando la posibilidad de que un candidato intente postularse a una vacante sin

assessment configurado. Si la creación del assessment falla, la vacante se elimina mediante rollback manual.

Los estados de vacante disponibles son `open`, `closed` y `draft`. El estado por defecto al crear es `open`. La interfaz ofrece un botón dedicado para cerrar directamente una vacante sin abrir el formulario de edición. La vacante marcada como `draft` no aparece en el catálogo visible para los candidatos.

5.9.5 Configuración de evaluaciones y preguntas (RF-15, RF-16)

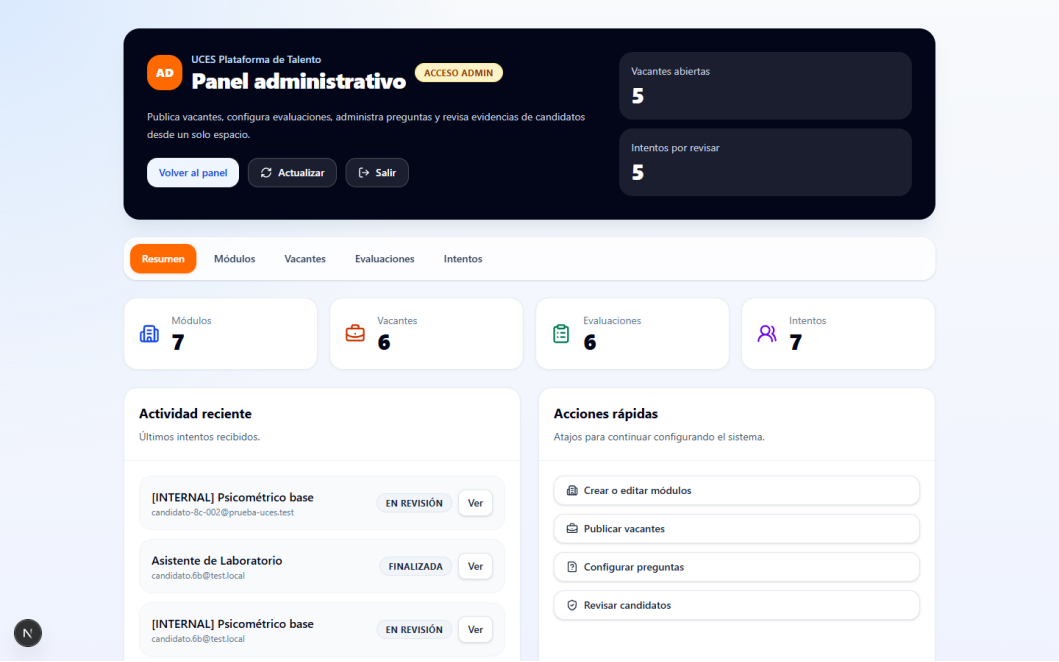


Figura 5.13. Panel de administración (/admin) con lista de candidatos y postulaciones. Fuente: captura de pantalla propia, 2026.

La edición de una evaluación se limita al título y la duración en minutos mediante `PATCH /api/admin/assessments/[id]`. La duración determina el tiempo que tendrá el candidato desde que inicia el intento.

Las preguntas se crean desde la vista de evaluaciones con tres campos: el texto de la pregunta (`prompt`), las opciones de respuesta (ingresadas como texto separado por comas o saltos de línea y convertidas a arreglo JSON), y la opción correcta (`correct_option`). Las opciones se normalizan en el cliente mediante:

```
const parseOptionsInput = (text: string): string[] =>
  text.split(/[\n,]/).map((option) => option.trim()).filter(Boolean);
```

Esta normalización acepta tanto listas separadas por coma como entradas en múltiples renglones, reduciendo la fricción para el administrador al ingresar las opciones.

Cuando se muestra una pregunta existente, la función `normalizeOptions()` convierte el valor almacenado en la base de datos —que puede ser un arreglo JSON, una cadena JSON o un objeto— al formato de arreglo esperado por la interfaz. Esta normalización defensiva maneja distintos formatos que pueden haberse generado durante el desarrollo.

Las preguntas del cuestionario psicométrico tienen un formato especial de prompt que las identifica ante el sistema de cálculo: `MINI_BIG_FIVE | [RASGO] | [texto de la pregunta]`. Este prefijo no es visible para el candidato durante la evaluación, pero es el mecanismo mediante el cual la función `calculatePsicometricoSummary()` agrupa las respuestas por rasgo.

5.9.6 Consulta de postulaciones e intentos (RF-17)

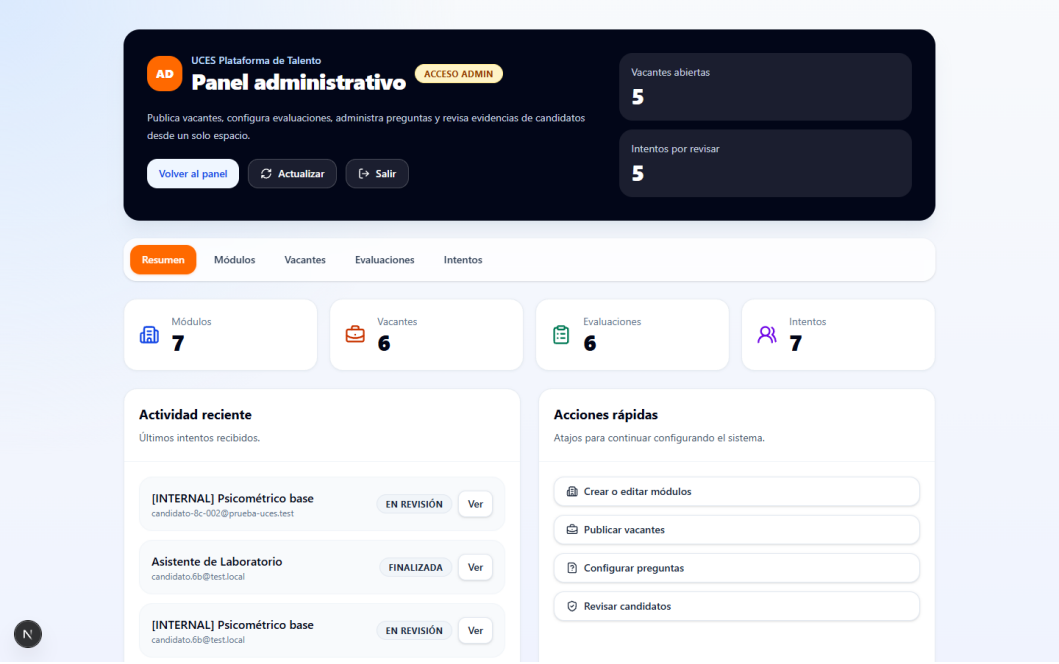


Figura 5.13. Panel de administración (/admin) con lista de candidatos y postulaciones. Fuente: captura de pantalla propia, 2026.

La pestaña de intentos muestra todos los registros de la tabla `attempts` con información del candidato y la vacante mediante una consulta con relaciones anidadas:

```
// app/api/admin/attempts/route.ts
admin.supabase.from("attempts")
  .select("id, status, started_at, submitted_at, theory_score,
application:applications!inner(id, user_id, vacancy_id, vacancy:vacancies(title),
```

```
profile:profiles!applications_user_id_fkey(name, email))")
  .order("started_at", { ascending: false })
```

Esta consulta usa el join `!inner` para garantizar que solo se devuelvan intentos que tengan una postulación asociada. El panel ofrece filtros por `vacancy_id`, `user_id` y `status` que el administrador puede combinar para reducir la lista.

5.9.7 Detalle de intento y consulta de respuestas y evidencias (RF-17)

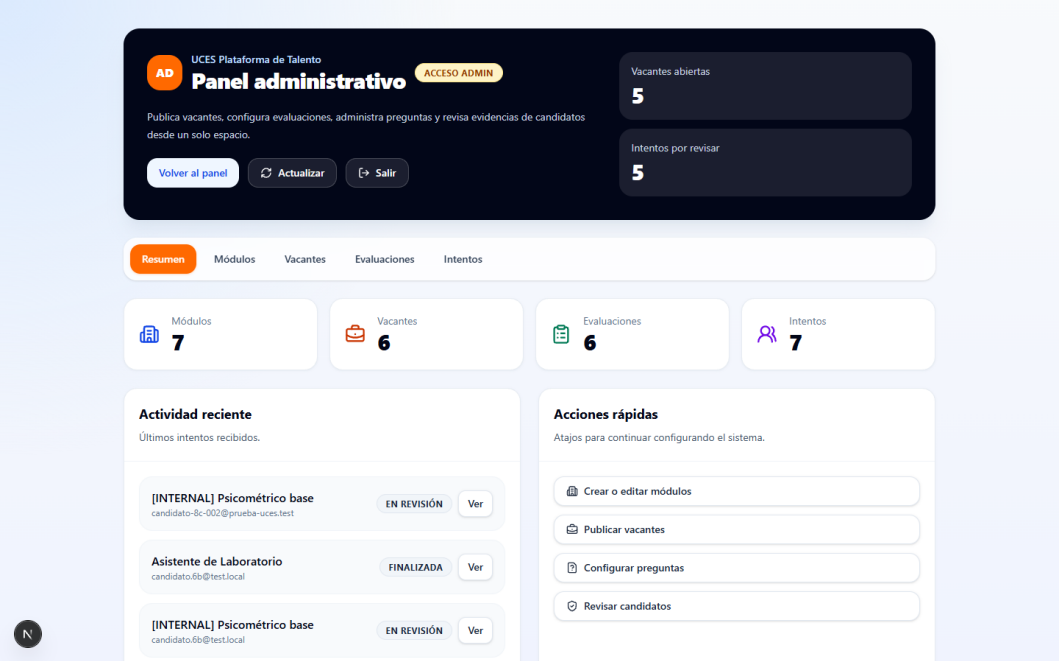


Figura 5.13. Panel de administración (/admin) con lista de candidatos y postulaciones. Fuente: captura de pantalla propia, 2026.

La página de detalle de intento (`/admin/attempts/[id]`) es la vista más completa del panel. Muestra en secciones diferenciadas:

21. **Datos generales:** email y nombre del candidato, vacante, módulo, estado actual y decisión.
22. **Resumen psicométrico:** cuando el intento corresponde al cuestionario base, muestra los scores por rasgo, su clasificación y las respuestas individuales.
23. **Acciones de revisión:** selector de decisión (`APPROVED` / `REJECTED`), campo de notas y botones para cambiar el estado del intento.
24. **Respuestas:** lista de todas las respuestas con el texto de la pregunta, la opción seleccionada y la opción correcta.
25. **Evidencias:** archivos subidos con nombre de ruta, tipo MIME, tamaño y enlace de descarga.

Los archivos de evidencia se muestran con URLs firmadas de 60 minutos generadas por el endpoint `GET /api/admin/attempts/[id]/detail`. Estas URLs se generan mediante `supabase.storage.from(bucket).createSignedUrl(path, 60 * 60)`.

5.9.8 Registro de decisiones y observaciones (RF-18)

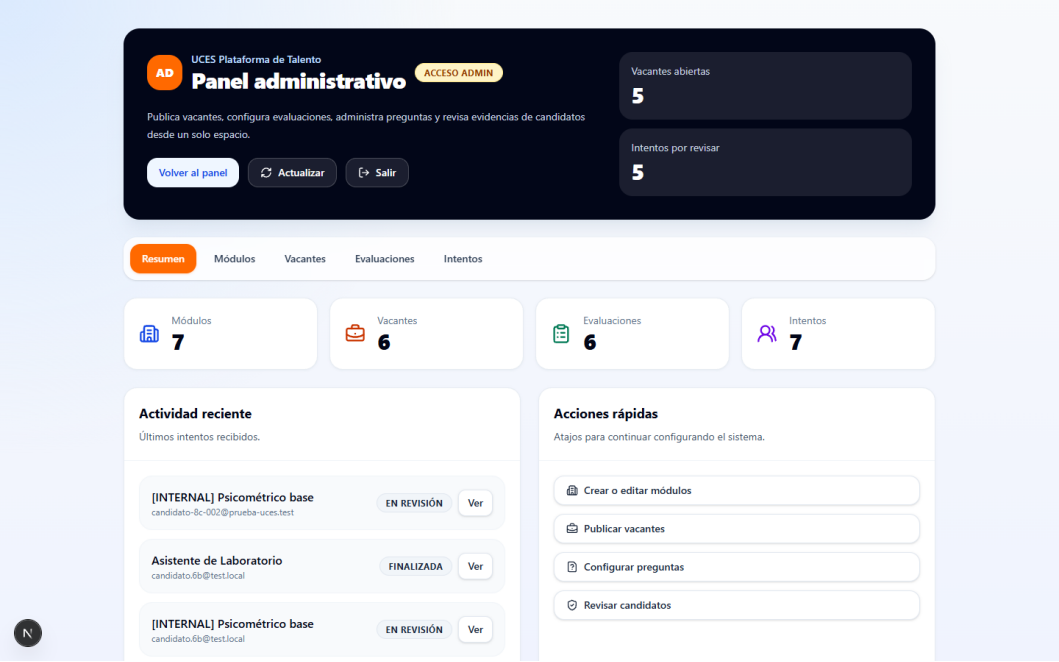


Figura 5.13. Panel de administración (/admin) con lista de candidatos y postulaciones. Fuente: captura de pantalla propia, 2026.

La operación de registrar una decisión sobre un intento es la más compleja del panel administrativo. El endpoint `PATCH /api/admin/attempts/[id]` implementa una lógica en dos capas para persitir tanto el cambio de estado del intento como la decisión y las notas del revisor.

Figura 5.5 — Diagrama del flujo de persistencia de decisiones

```
sequenceDiagram
    participant UI as admin/attempts/[id]/page.tsx
    participant PATCH as PATCH /api/admin/attempts/[id]
    participant DB_A as PostgreSQL (attempts)
    participant DB_R as PostgreSQL (reviews)

    UI->>PATCH: PATCH {status, decision, notes}
    PATCH->>PATCH: requireAdmin() → verifica JWT y rol
    PATCH->>DB_A: UPDATE attempts SET status = ? WHERE id = ?
    DB_A-->>PATCH: OK

    alt decision o notes en el body
        PATCH->>DB_R: SELECT reviews WHERE attempt_id = ?
```

```
        alt reviews existe con ese attempt_id
            DB_R-->>PATCH: {id: reviewId}
            PATCH->>DB_R: UPDATE reviews SET decision, notes WHERE id = reviewId
        else no existe review previo
            DB_R-->>PATCH: null
            PATCH->>DB_R: INSERT INTO reviews {attempt_id, reviewer_id, decision,
notes}
        end
        alt reviews tabla no existe (error 42P01)
            PATCH->>DB_A: UPDATE attempts SET decision, reviewer_notes WHERE id = ?
        end
    end

    PATCH->>PATCH: getAttemptDetail() → carga estado actualizado
    PATCH-->>UI: {data: attemptActualizado, warning?}
```

La función `persistDecisionAndNotes` intenta primero escribir en `reviews` mediante `upsertReview`. Si la tabla `reviews` no existe (código de error PostgreSQL `42P01` o código PostgREST `PGRST205`), el sistema recae sobre `attempts.decision` y `attempts.reviewer_notes`. En el entorno de producción con el esquema completo, la tabla `reviews` es el almacén primario de decisiones; el fallback hacia `attempts` solo opera cuando el esquema está incompleto.

5.9.9 Indicadores de resumen

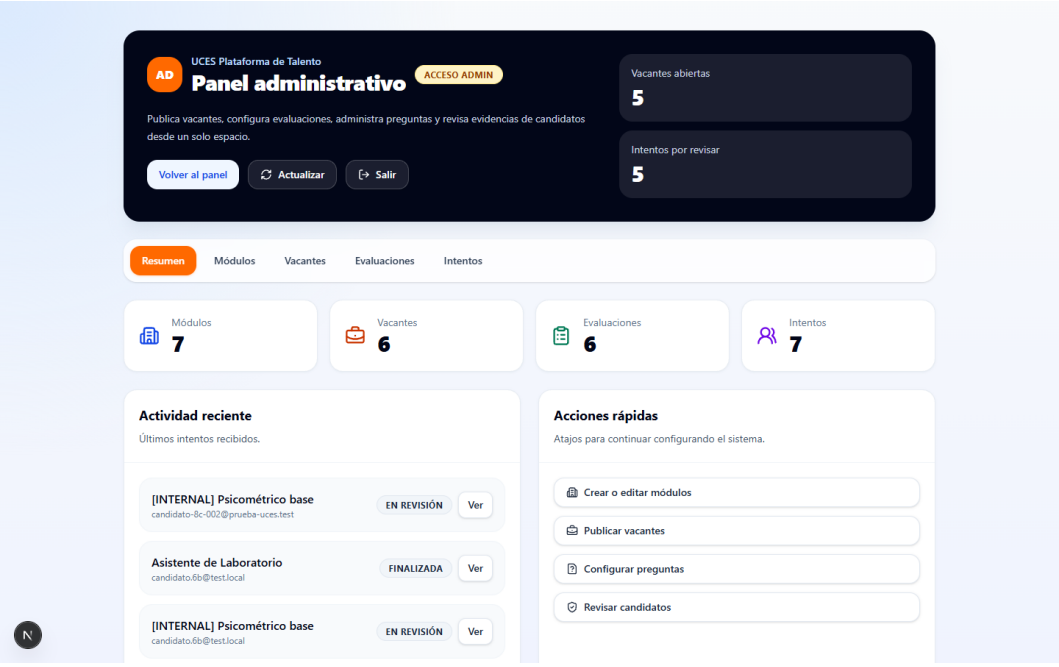


Figura 5.13. Panel de administración (/admin) con lista de candidatos y postulaciones. Fuente: captura de pantalla propia, 2026.

La pestaña de resumen del panel muestra cuatro contadores calculados en el cliente a partir de los datos ya cargados: número de módulos, vacantes, evaluaciones e intentos. Adicionalmente calcula `reviewAttempts` (intentos con estado `UNDER_REVIEW` o `SUBMITTED`) y `completedAttempts` (estado `COMPLETED`).

Existe un endpoint de estadísticas por vacante en `GET /api/admin/vacancies/[id]/stats` que devuelve `applicationsCount` y `attemptsCount` mediante consultas paralelas. Este endpoint no se consume desde ninguna vista del panel en el estado actual del MVP; fue desarrollado como infraestructura preparatoria para un panel de estadísticas futuro (discrepancia D-03, documentada en la sección 5.12).

5.10 Implementación de seguridad y control de acceso

[Figura 5.16. [MARCADOR — EV-5B-02 PENDIENTE: captura de vista de detalle del candidato en el panel de administración (/admin/attempts/[id]).]]

Figura 5.16. [MARCADOR — EV-5B-02 PENDIENTE: captura de vista de detalle del candidato en el panel de administración (/admin/attempts/[id]).]

5.10.1 Autenticación mediante Supabase Auth

[Figura 5.16. [MARCADOR — EV-5B-02 PENDIENTE: captura de vista de detalle del candidato en el panel de administración (/admin/attempts/[id]).]]

Figura 5.16. [MARCADOR — EV-5B-02 PENDIENTE: captura de vista de detalle del candidato en el panel de administración (/admin/attempts/[id]).]

La autenticación del sistema se delega íntegramente a Supabase Auth, que implementa el estándar JWT (RFC 7519). Cuando un usuario inicia sesión mediante `signInWithPassword`, Supabase emite un token JWT firmado con RS256 que encapsula el identificador del usuario (`sub`), su correo y metadatos de la sesión. Este token se almacena en el almacenamiento local del navegador y tiene vigencia de una hora por defecto.

La comunicación entre el servidor Next.js y Supabase Auth utiliza la clave de rol de servicio (`SUPABASE_SERVICE_ROLE_KEY`), que permite invocar `supabase.auth.getUser(token)` para validar el JWT sin necesidad de una sesión activa del servidor. Esta validación descifra y verifica la firma del token, devuelve el usuario si el token es válido, y retorna un error si el token está expirado, malformado o revocado.

5.10.2 Función `requireAdmin()` y protección de operaciones administrativas

[Figura 5.16. [MARCADOR — EV-5B-02 PENDIENTE: captura de vista de detalle del candidato en el panel de administración (/admin/attempts/[id]).]]

Figura 5.16. [MARCADOR — EV-5B-02 PENDIENTE: captura de vista de detalle del candidato en el panel de administración (/admin/attempts/[id]).]

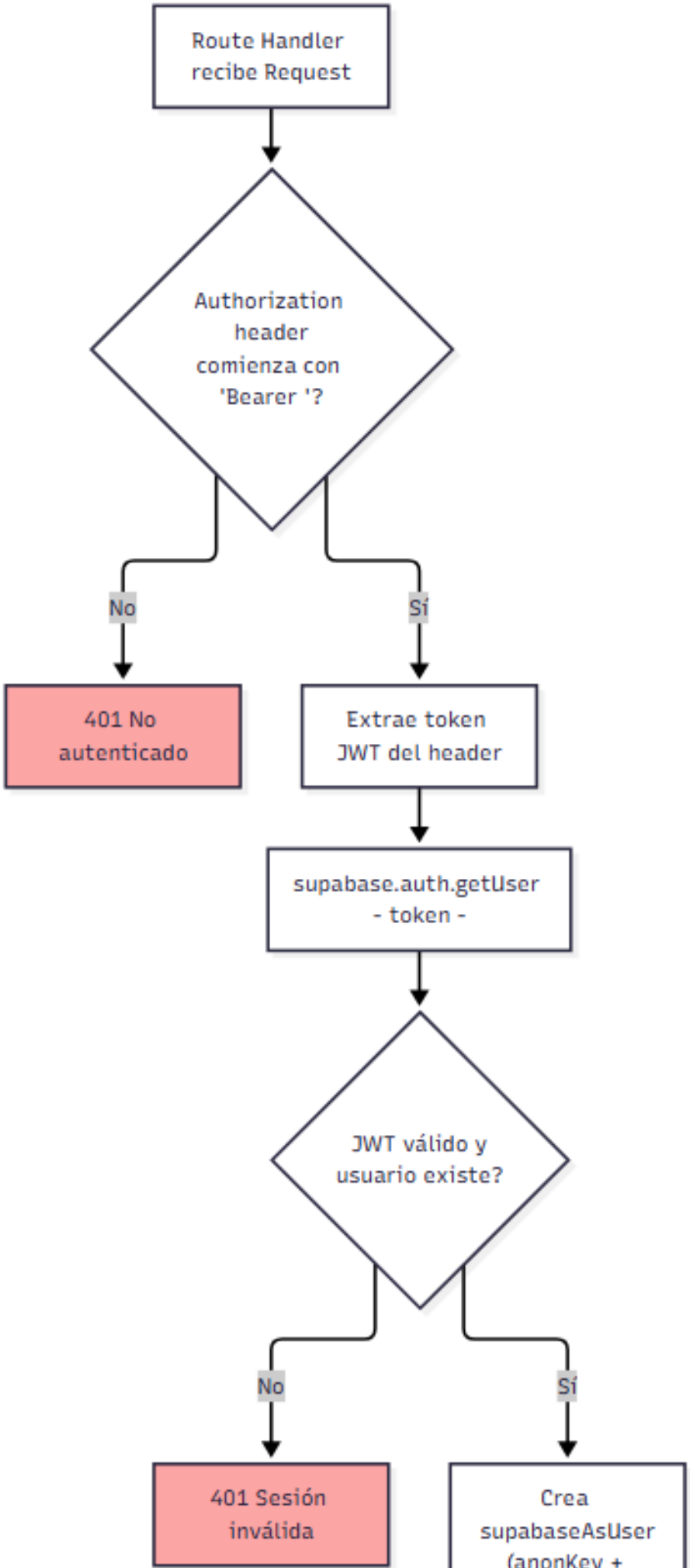

Route Handler recibe Request
Authorization header comienza con 'Bearer '?
No
Sí
401 No autenticado
Extrae token JWT del header
supabase.auth.getUser - token -
JWT válido y usuario existe?
No
Sí
401 Sesión inválida
Crea supabaseAsUser (anonKey +

Figura 5.15. Diagrama de flujo de la función requireAdmin(). Fuente: elaboración propia, 2026.

La función `requireAdmin()` es el guardián de acceso para los 13 endpoints del área administrativa. Su implementación en `app/api/admin/utils.ts` ejecuta cuatro verificaciones en secuencia:

Figura 5.6 — Flujo de autorización en requireAdmin()

```
flowchart TD
    A[Route Handler recibe Request] --> B{Authorization header\ncomienza con
Bearer?}
    B -- No --> C[401 No autenticado]
    B -- Sí --> D[Extrae token JWT]
    D --> E[supabase.auth.getUser(token)]
    E --> F{JWT válido y\nusuario existe?}
    F -- No --> G[401 Sesión inválida]
    F -- Sí --> H[Crea supabaseAsUser con\nanonKey + header Bearer]
    H --> I[supabaseAsUser.rpc('get_my_role')]
    I --> J{roleData === 'admin'?}
    J -- No --> K[403 Acceso restringido]
    J -- Sí --> L[Retorna {supabase: serviceClient, userId}]
```

El diseño de `requireAdmin()` tiene una característica notable: para verificar el rol, crea un cliente Supabase que usa la clave anónima (`anonKey`) pero inyecta el JWT del usuario en el encabezado `Authorization`. Esto hace que la llamada `rpc("get_my_role")` se ejecute en el contexto de seguridad del usuario, activando las políticas de RLS que solo permiten a ese usuario leer su propio rol. La razón de este diseño es que `get_my_role()` usa `auth.uid()` internamente, que solo retorna el UUID del usuario actual cuando la llamada se realiza con un token de usuario, no con la clave de servicio.

Una vez que la verificación es exitosa, la función retorna el cliente de servicio (`supabase: supabaseServer()`) que el Route Handler usa para ejecutar las operaciones de base de datos. En este punto, las operaciones sí omiten las políticas de RLS, pero la identidad del solicitante ya fue verificada como administrador.

5.10.3 Función `get_my_role()` y separación de contextos

[Figura 5.16. [MARCADOR — EV-5B-02 PENDIENTE: captura de vista de detalle del candidato en el panel de administración (/admin/attempts/[id]).]]

Figura 5.16. [MARCADOR — EV-5B-02 PENDIENTE: captura de vista de detalle del candidato en el panel de administración (/admin/attempts/[id]).]

La función de base de datos `get_my_role()` es el núcleo del sistema de roles. Está definida en `supabase/rls.sql` con la directiva `SECURITY DEFINER`:

```
create or replace function public.get_my_role()
returns text
language plpgsql
security definer
set search_path = public
stable
as $$
declare
  current_role text;
begin
  select ur.role into current_role
  from public.user_roles ur
  where ur.user_id = auth.uid()
  limit 1;

  if current_role is not null then return current_role; end if;

  select p.role into current_role
  from public.profiles p
  where p.user_id = auth.uid()
  limit 1;

  if current_role = 'admin' then return 'admin'; end if;

  return 'user';
end;
$$;

revoke all on function public.get_my_role() from public;
grant execute on function public.get_my_role() to authenticated, service_role;
```

La directiva `SECURITY DEFINER` permite que la función consulte `user_roles` y `profiles` sin que las políticas de RLS de esas tablas interfieran con la consulta interna. Sin ella, la propia política que restringe el acceso a `user_roles` impediría que la función leyera el rol del usuario, creando una circularidad que produjo el problema técnico T-02 (documentado en la sección 5.12).

La lógica de la función implementa una jerarquía de fuentes: consulta primero `user_roles` (fuente canónica desde la iteración 3), luego `profiles.role` como respaldo de compatibilidad. Solo retorna `admin` si lo encuentra explícitamente; en cualquier otro caso retorna `user`. Esta política de mínimo privilegio por defecto garantiza que un valor inesperado en la base de datos no escale privilegios.

5.10.4 Triple capa de defensa en la resolución de roles

[Figura 5.16. [MARCADOR — EV-5B-02 PENDIENTE: captura de vista de detalle del candidato en el panel de administración (/admin/attempts/[id]).]]

Figura 5.16. [MARCADOR — EV-5B-02 PENDIENTE: captura de vista de detalle del candidato en el panel de administración (/admin/attempts/[id]).]

El flujo completo de resolución de rol incorpora tres niveles de verificación:

Capa	Ubicación	Acción defensiva
1 — RPC	`rls.sql: get_my_role()`	Solo retorna `admin` si existe en `user_roles` o `profiles.role`
2 — resolveRole	`lib/auth.ts: resolveRole()`	Acepta únicamente los literales `"admin"` o `"user"`; cualquier otro valor retorna `"user"`
3 — Fallback en error	`lib/auth.ts: getRoleForSession()`	Si el RPC produce un error de cualquier tipo, retorna `"user"` como valor seguro

```
// lib/auth.ts
export const getRoleForSession = async (supabase: SupabaseClient):
Promise<ProfileRole> => {
  const { data, error } = await supabase.rpc("get_my_role");
  if (error) return "user";  // Capa 3
  return resolveRole(typeof data === "string" ? data : null);  // Capa 2
};
```

Este diseño garantiza que ningún fallo de red, error de base de datos o valor inesperado escale los privilegios de un usuario.

5.10.5 Políticas de Row Level Security

[Figura 5.16. [MARCADOR — EV-5B-02 PENDIENTE: captura de vista de detalle del candidato en el panel de administración (/admin/attempts/[id]).]]

Figura 5.16. [MARCADOR — EV-5B-02 PENDIENTE: captura de vista de detalle del candidato en el panel de administración (/admin/attempts/[id]).]

Las políticas de RLS definen qué operaciones puede realizar cada usuario sobre cada tabla. Se aplican a las doce tablas en las que RLS está habilitado mediante `ALTER TABLE ... ENABLE ROW LEVEL SECURITY`. El patrón de políticas sigue dos arquetipos según el tipo de tabla:

Tablas de configuración (modules, vacancies, assessments, questions, practical_tasks):

- Lectura permitida a todos los usuarios autenticados: `USING (true)` para `authenticated`

- Escritura permitida solo a administradores: `USING (get_my_role() = 'admin')`

Tablas de participación (applications, attempts, answers, evidence_uploads, reviews):

- Lectura y escritura del usuario sobre sus propios registros: filtra por `user_id = auth.uid()` o por relación transitiva hacia el `user_id`
- Acceso completo a administradores: política separada que verifica `get_my_role() = 'admin'`

Esta separación garantiza que un candidato autenticado solo pueda acceder a sus propias postulaciones, intentos y respuestas, sin visibilidad de los de otros candidatos.

5.10.6 Cliente privilegiado: uso y alcance del service_role

[Figura 5.16. [MARCADOR — EV-5B-02 PENDIENTE: captura de vista de detalle del candidato en el panel de administración (/admin/attempts/[id]).]]

Figura 5.16. [MARCADOR — EV-5B-02 PENDIENTE: captura de vista de detalle del candidato en el panel de administración (/admin/attempts/[id]).]

El cliente `supabaseServer()` usa la `SUPABASE_SERVICE_ROLE_KEY`, que omite RLS. Los contextos en que se usa este cliente son los siguientes:

Contexto	Razón del uso de service_role
`register/route.ts`	Crear usuario en Auth (`admin.createUser`) y operaciones de rollback
`submit/route.ts`	Verificar intento y actualizar `attempts` (acceso cross-user no permitido por RLS de candidato)
`answers/route.ts`	Verificar pertenencia del intento con relación cruzada
`apply/route.ts`	Verificar estado del psicométrico y crear application/attempt
Todos los endpoints `/api/admin/**`	Operaciones administrativas sobre registros de múltiples usuarios

Es importante distinguir que usar el cliente de servicio no equivale a omitir la autenticación. Todos los Route Handlers que usan `supabaseServer()` validan explícitamente el JWT del solicitante antes de usar el cliente privilegiado:

```
// Patrón de validación previa al uso de service_role
const { data: userData, error: userError } = await supabase.auth.getUser(token);
if (userError || !userData.user) return NextResponse.json({ error: "Sesión inválida"
}, { status: 401 });
// Solo después de esta verificación se usan operaciones privilegiadas
```

Lo que la clave de servicio omite es la verificación de RLS sobre los datos; la verificación de identidad del solicitante es siempre responsabilidad del código de la API, no de las políticas de base de datos.

5.10.7 Validación de pertenencia de intentos

[Figura 5.16. [MARCADOR — EV-5B-02 PENDIENTE: captura de vista de detalle del candidato en el panel de administración (/admin/attempts/[id]).]]

Figura 5.16. [MARCADOR — EV-5B-02 PENDIENTE: captura de vista de detalle del candidato en el panel de administración (/admin/attempts/[id]).]

Tanto el guardado de respuestas como el envío del intento verifican que el intento consultado pertenezca al usuario autenticado:

```
// app/api/attempts/[attemptId]/answers/route.ts y submit/route.ts
const application = Array.isArray(attempt.application)
  ? attempt.application[0]
  : attempt.application;

if (!application || application.user_id !== userData.user.id) {
  return NextResponse.json({ error: "No autorizado" }, { status: 403 });
}
```

Esta validación manual es necesaria porque el cliente de servicio no aplica RLS: aunque las políticas de RLS prohibirían que un usuario acceda a los intentos de otro, el cliente de servicio no aplica esas políticas. La validación explícita en el código suple esa protección.

5.10.8 Protección de CURP mediante HMAC-SHA256

[Figura 5.16. [MARCADOR — EV-5B-02 PENDIENTE: captura de vista de detalle del candidato en el panel de administración (/admin/attempts/[id]).]]

Figura 5.16. [MARCADOR — EV-5B-02 PENDIENTE: captura de vista de detalle del candidato en el panel de administración (/admin/attempts/[id]).]

El diseño del Trabajo A contemplaba almacenar la CURP como dato personal. La implementación optó por almacenar únicamente el hash HMAC-SHA256, calculado con una clave secreta almacenada en la variable de entorno `CURP_HASH_SECRET`:

```
// lib/curp.ts
export const hashCurpHmac = (secret: string, curp: string) =>
  crypto.createHmac("sha256", secret).update(normalizeCurp(curp)).digest("hex");
```

Este mecanismo permite verificar si una CURP ya está registrada (comparando su hash con los registros existentes) sin almacenar el dato en claro. La función es unidireccional: no es posible recuperar la CURP a partir del hash. La tabla `curp_hashes` almacena el hash y el `user_id` correspondiente, con un índice UNIQUE que impide duplicados.

5.10.9 Almacenamiento privado y URLs firmadas

[Figura 5.16. [MARCADOR — EV-5B-02 PENDIENTE: captura de vista de detalle del candidato en el panel de administración (/admin/attempts/[id]).]]

Figura 5.16. [MARCADOR — EV-5B-02 PENDIENTE: captura de vista de detalle del candidato en el panel de administración (/admin/attempts/[id]).]

El bucket `evidences` de Supabase Storage se configura con `public = false` en `rls.sql`. Las políticas de Storage permiten subida solo al propietario y lectura al propietario o al administrador. El acceso a los archivos desde el panel administrativo se realiza exclusivamente mediante URLs firmadas con vigencia de 60 minutos, generadas por el cliente de servicio.

La interfaz de evaluación usa `getPublicUrl()` para mostrar un enlace de previsualización al candidato mientras dura su sesión. Dado que el bucket es privado, esta URL no es accesible sin autenticación. Se reconoce como una limitación del MVP: el enlace de previsualización puede no funcionar correctamente si el navegador no incluye el JWT al intentar acceder al recurso.

5.10.10 Variables de entorno y separación de credenciales

[Figura 5.16. [MARCADOR — EV-5B-02 PENDIENTE: captura de vista de detalle del candidato en el panel de administración (/admin/attempts/[id]).]]

Figura 5.16. [MARCADOR — EV-5B-02 PENDIENTE: captura de vista de detalle del candidato en el panel de administración (/admin/attempts/[id]).]

La convención de Next.js distingue automáticamente las variables con prefijo `NEXT_PUBLIC_` (incluidas en el bundle del cliente) de las que no lo tienen (exclusivas del servidor):

Variable	Expuesta al cliente	Propósito
`NEXT_PUBLIC_SUPABASE_URL`	Sí	URL del proyecto Supabase
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Sí	Clave de RLS para el navegador
`SUPABASE_SERVICE_ROLE_KEY`	No	Acceso privilegiado del

		servidor
`CURP_HASH_SECRET`	No	Clave HMAC para protección de CURP

La clave de servicio y el secreto HMAC nunca se incluyen en el bundle de JavaScript enviado al navegador. Si un usuario inspeccionara el código de red, solo podría recuperar la clave anónima, cuyo uso está limitado por las políticas de RLS.

5.10.11 Limitaciones de seguridad del MVP

[Figura 5.16. [MARCADOR — EV-5B-02 PENDIENTE: captura de vista de detalle del candidato en el panel de administración (/admin/attempts/[id]).]]

Figura 5.16. [MARCADOR — EV-5B-02 PENDIENTE: captura de vista de detalle del candidato en el panel de administración (/admin/attempts/[id]).]

Las siguientes limitaciones se identifican como aspectos de seguridad que quedaron fuera del alcance del MVP:

Limitación	**Descripción**	**Impacto**
Sin `middleware.ts` de Next.js	La protección de rutas es client-side mediante `useEffect`. Las páginas renderizan HTML antes de la verificación	UX (parpadeo de contenido), no exposición de datos
Sin validación de tipo MIME en servidor	La validación del tipo de archivo para evidencias ocurre solo en el cliente (`accept="image/*,application/pdf"`)	Bajo: el bucket privado previene acceso no autorizado al contenido
`getPublicUrl()` en bucket privado	Las URLs de previsualización de evidencia en el cliente no funcionan sin autenticación activa	Funcional, no de seguridad
Confirmación de correo deshabilitada	`email_confirm: true` en `createUser` confirma la cuenta sin verificar el correo	Bajo en contexto institucional de invitación controlada

5.11 Implementación del módulo psicométrico y procesamiento de resultados

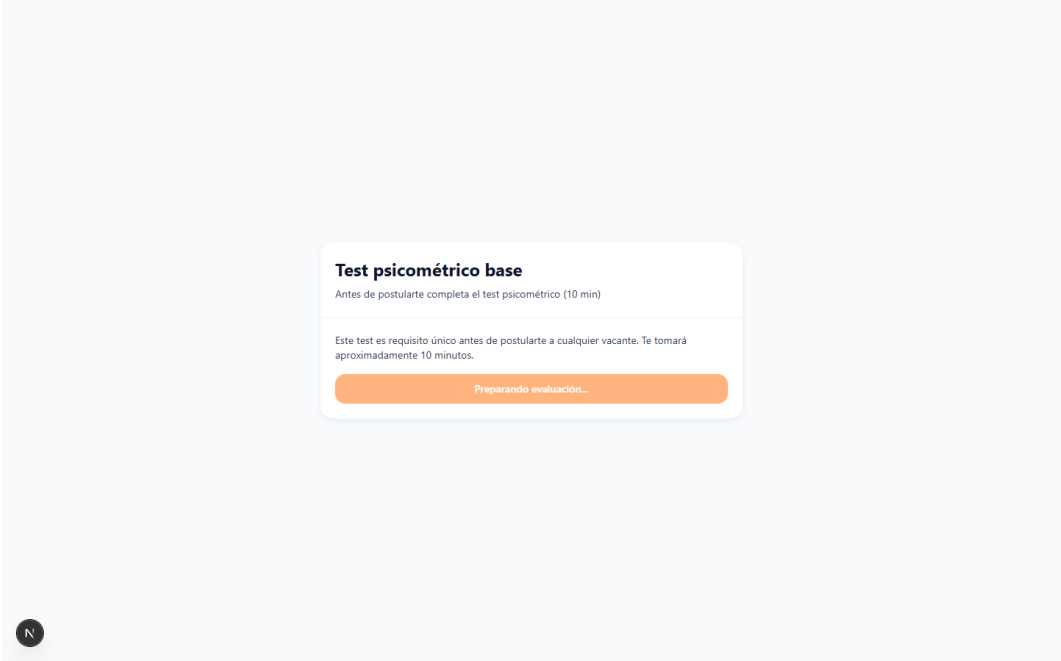


Figura 5.18. Guard de prerequisito: el candidato debe completar el test psicométrico antes de postularse. Fuente: captura de pantalla propia, 2026.

5.11.1 Instrumento institucional Mini Big Five

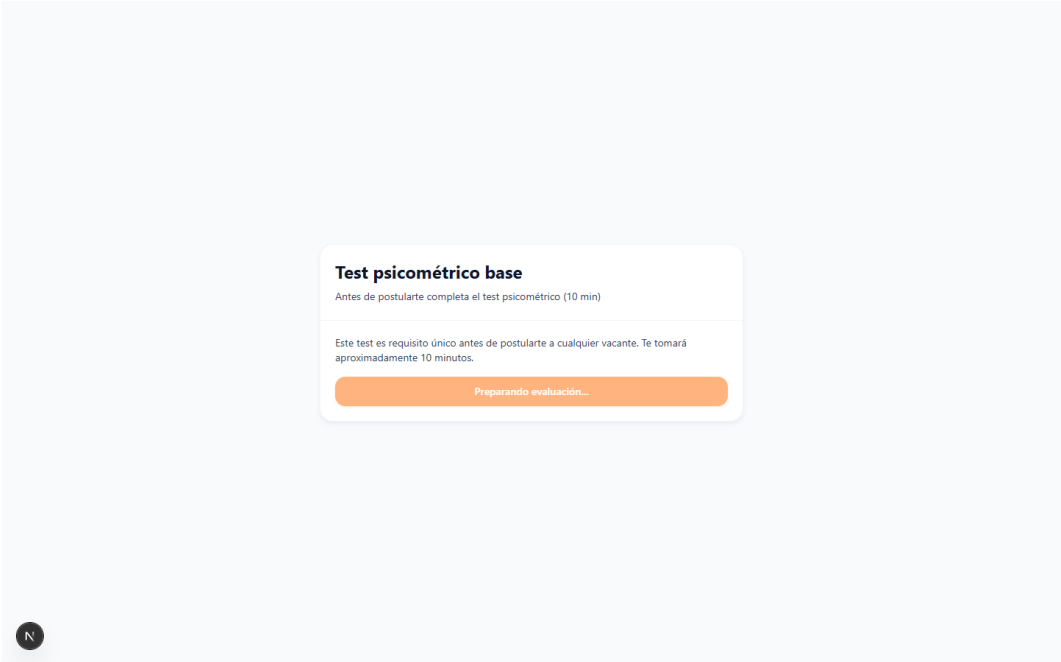


Figura 5.18. Guard de prerequisito: el candidato debe completar el test psicométrico antes de postularse. Fuente: captura de pantalla propia, 2026.

El instrumento psicométrico implementado es una adaptación institucional del modelo Big Five de personalidad, designado internamente como "Mini Big Five". Fue proporcionado por el área de Recursos Humanos de UCES para su incorporación al sistema. El cuestionario consta de 20 reactivos tipo Likert de cinco puntos, organizados en cinco dimensiones de cuatro reactivos cada una.

Las cinco dimensiones medidas son:

Dimensión	**Clave en el código**	**Ítems**	**Interpretación institucional**
Estabilidad Emocional	`ESTABILIDAD_EMOCIONAL`	4	Alta, Adecuada, Riesgo reactividad
Amabilidad	`AMABILIDAD`	4	Cooperativo, Funcional, Riesgo conflicto
Responsabilidad	`RESPONSABILIDAD`	4	Alto compromiso, Aceptable, Riesgo incumplimiento
Apertura	`APERTURA`	4	Innovador/adaptable, Moderado, Rigidez
Extroversión	`EXTROVERSION`	4	Indicador sin filtro

Los umbrales de clasificación (≥16 = nivel alto, ≥11 = nivel adecuado, <11 = riesgo) son criterios establecidos institucionalmente para esta evaluación. No representan una validación científica publicada de forma independiente, sino los rangos definidos por el área de RH de UCES para interpretar los resultados en su contexto de selección.

5.11.2 Organización de los reactivos y estructura de almacenamiento

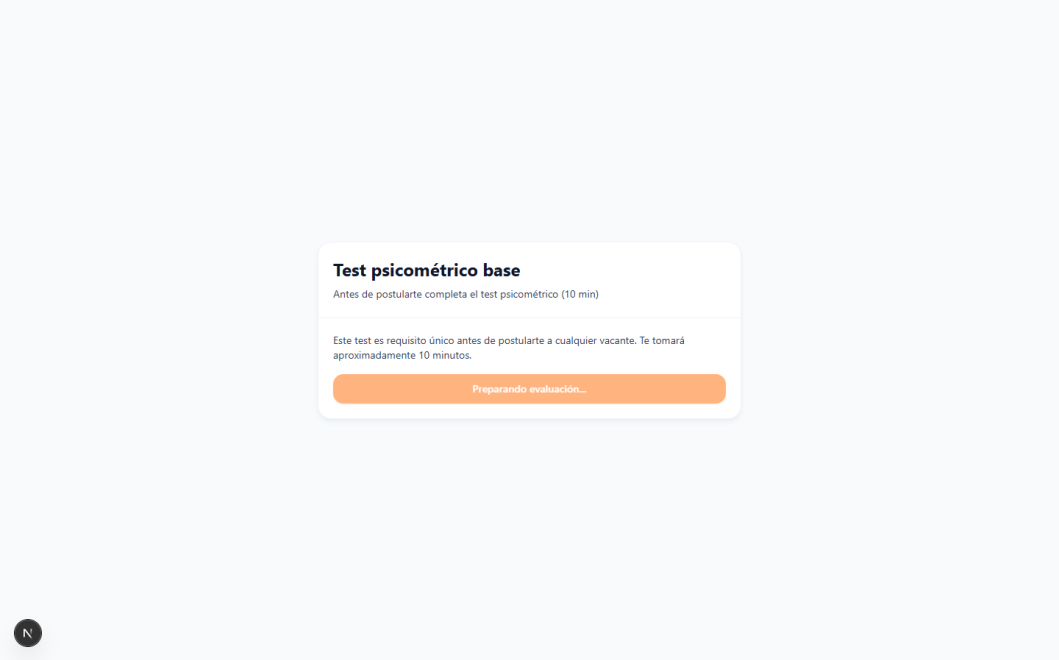


Figura 5.18. Guard de prerequisito: el candidato debe completar el test psicométrico antes de postularse. Fuente: captura de pantalla propia, 2026.

Cada reactivo psicométrico se almacena en la tabla `questions` con un formato de `prompt` que codifica su pertenencia a una dimensión:

```
MINI_BIG_FIVE | ESTABILIDAD_EMOCIONAL | Texto descriptivo del ítem
```

Este formato de triple segmento separado por el carácter `|` es el mecanismo que permite al algoritmo de cálculo identificar a qué dimensión pertenece cada pregunta sin necesidad de una columna adicional en el esquema. Los 20 reactivos corresponden al assessment con UUID fijo `f2000000-0000-0000-0000-000000000001`, vinculado a una vacante interna con `status = 'draft'` denominada `[INTERNAL] Psicométrico base`. Al estar en estado `draft`, esta vacante no aparece en el catálogo de vacantes para los candidatos.

Las respuestas se capturan en escala Likert del 1 al 5 (donde 1 representa el menor grado de acuerdo y 5 el mayor), almacenadas como cadenas en la columna `selected_option` de la tabla `answers`. El algoritmo de cálculo las convierte a enteros mediante `parseInt`.

5.11.3 Flujo de inicio del cuestionario psicométrico

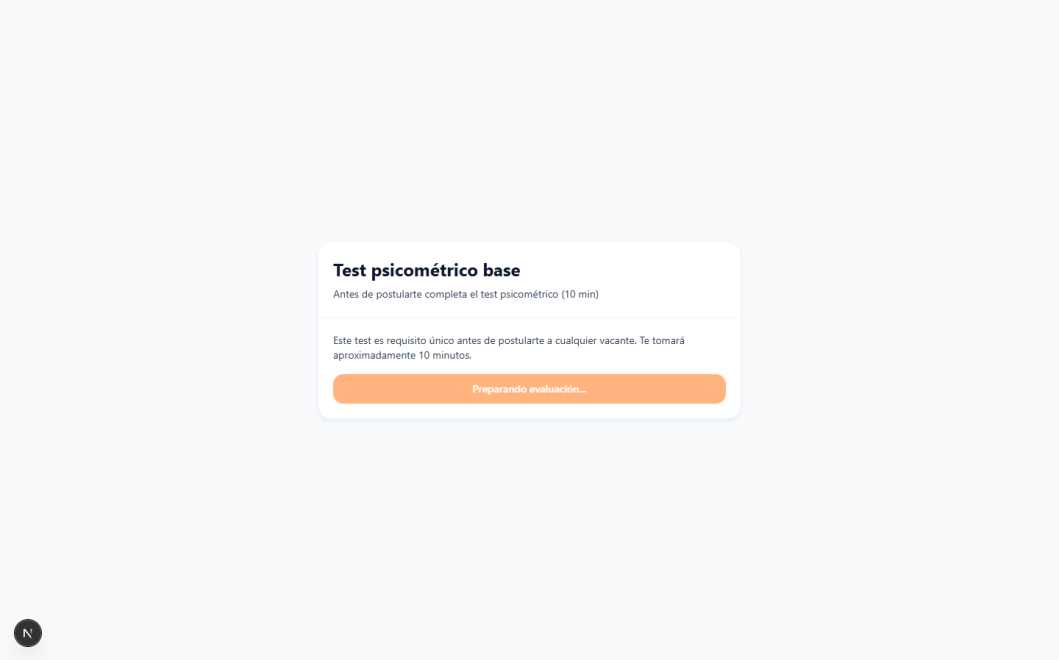


Figura 5.18. Guard de prerequisito: el candidato debe completar el test psicométrico antes de postularse. Fuente: captura de pantalla propia, 2026.

El flujo que lleva a un candidato a presentar el cuestionario se origina en `/psicometrico/page.tsx`, que inmediatamente al montarse invoca `POST /api/psicometrico/start`. Este endpoint delega en `ensurePsicometricoAttempt()` de `lib/psicometrico.ts`:

```
flowchart TD
    A[/psicometrico] --> B[POST /api/psicometrico/start]
    B --> C[getPsicometricoAttemptState]
    C --> D{¿Existe intento\ncompletado?}
    D -- Sí --> E[{completed: true}]
    E --> F[Redirect a /panel]
    D -- No --> G{¿Existe intento\npendiente?}
    G -- Sí --> H[Retorna attemptId existente]
    G -- No --> I[Busca o crea application\npara vacante interna]
    I --> J[Crea attempt con deadline_at]
    J --> K[Retorna {attemptId}]
    H --> L[Redirect a /evaluacion/attemptId]
    K --> L
```

La función `getPsicometricoAttemptState()` determina si el psicométrico está completado verificando si existe algún intento con `submitted_at` no nulo o con estado en el conjunto `{SUBMITTED, UNDER_REVIEW, COMPLETED}`. Si ya está completado, la página redirige al panel sin mostrar el cuestionario nuevamente. Si existe un intento en progreso, se retoma ese intento. Si no existe ninguno, se crea la aplicación y el intento necesarios.

5.11.4 Algoritmo de procesamiento psicométrico

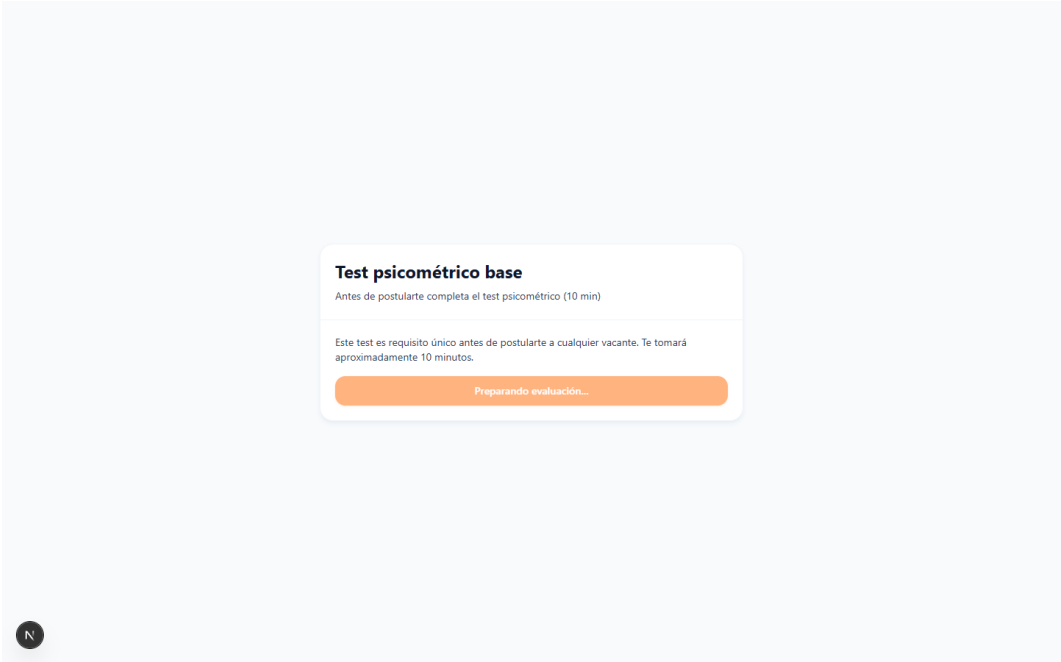


Figura 5.18. Guard de prerequisito: el candidato debe completar el test psicométrico antes de postularse. Fuente: captura de pantalla propia, 2026.

El procesamiento de los resultados ocurre al momento del envío del intento, en el Route Handler `POST /api/attempts/[attemptId]/submit`. La función `calculatePsicometricoSummary()` de `lib/psicometricoResults.ts` recibe los pares pregunta-respuesta y produce el resumen estructurado:

```
// lib/psicometricoResults.ts (lógica de acumulación)
questionsWithAnswers.forEach(({ prompt, selected_option }) => {
  const likertValue = parseLikertValue(selected_option);     // Convierte a entero
1-5
  if (!likertValue) return;                                  // Omite no respondidas
  const segments = prompt.split("|").map(s => s.trim());
  if (segments[0] === "MINI_BIG_FIVE" && segments.length >= 3) {
    const traitKey = segments[1] as keyof typeof traitScores;
    if (traitKey in traitScores) {
      traitScores[traitKey] += likertValue;                  // Acumula puntaje
      traitAnswered[traitKey] += 1;                          // Cuenta respondidas
    }
  }
});
```

Solo se contabilizan preguntas respondidas. Las preguntas del cuestionario psicométrico que el candidato dejó sin responder no aportan al score de ningún rasgo. Esto permite que el algoritmo funcione aunque el candidato no haya completado todas las respuestas, aunque el `theory_score` reflejará un porcentaje de completitud menor.

5.11.5 Umbrales de clasificación por rasgo

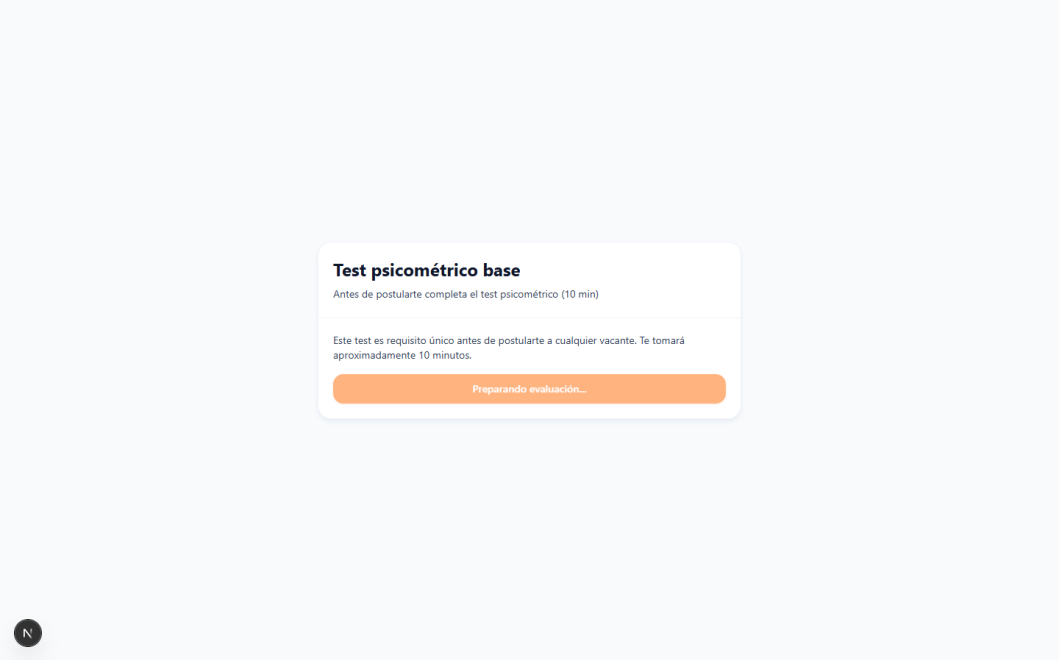


Figura 5.18. Guard de prerequisito: el candidato debe completar el test psicométrico antes de postularse. Fuente: captura de pantalla propia, 2026.

La función `classifyTrait()` aplica los umbrales institucionales a cada puntaje acumulado:

```
// lib/psicometricoResults.ts
const classifyTrait = (score: number, high: string, mid: string, low: string) => {
  if (score >= 16) return high;    // Alta (80% o más de la escala máxima)
  if (score >= 11) return mid;     // Adecuada
  return low;                      // Riesgo (puntaje < 55% de la escala)
};
```

El rango de puntaje por rasgo es de 4 (cuatro reactivos × 1, si el candidato responde el mínimo en todos) a 20 (cuatro reactivos × 5, si responde el máximo en todos). Los umbrales agrupan los resultados en tres categorías para facilitar la interpretación del administrador.

5.11.6 Tratamiento diferenciado de la extroversión

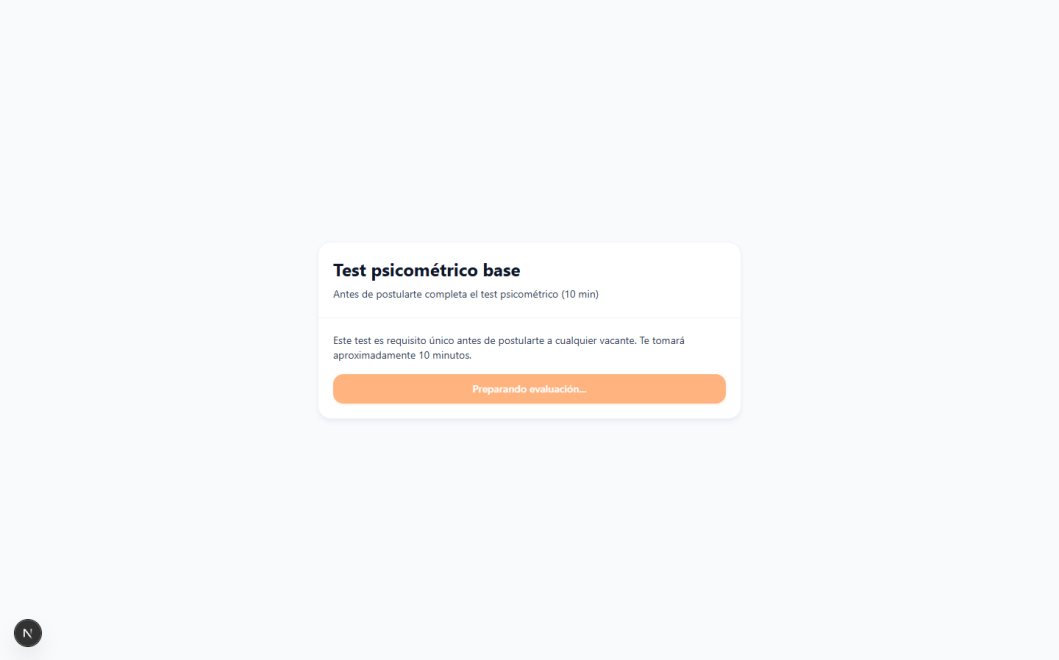


Figura 5.18. Guard de prerequisito: el candidato debe completar el test psicométrico antes de postularse. Fuente: captura de pantalla propia, 2026.

La dimensión de Extroversión recibe un tratamiento diferente a las otras cuatro. En lugar de asignarle una clasificación de riesgo o alta, la función `buildTraitResult()` la marca con un indicador descriptivo:

```
// lib/psicometricoResults.ts
extroversion: buildTraitResult(
  traitScores.EXTROVERSION, traitAnswered.EXTROVERSION,
  { high: "", mid: "", low: "" },
  false,                                          // withFilter = false → no
clasifica
  "No es filtro de contratación, solo indicador de integración."
)
```

El parámetro `withFilter = false` omite la aplicación de `classifyTrait()` y asigna directamente la nota explicativa como clasificación. Esta decisión refleja el criterio del área de RH: la extroversión es informativa para el proceso de integración al equipo, pero no es un criterio de filtro en la etapa de selección inicial.

5.11.7 Generación del resumen y persistencia JSONB

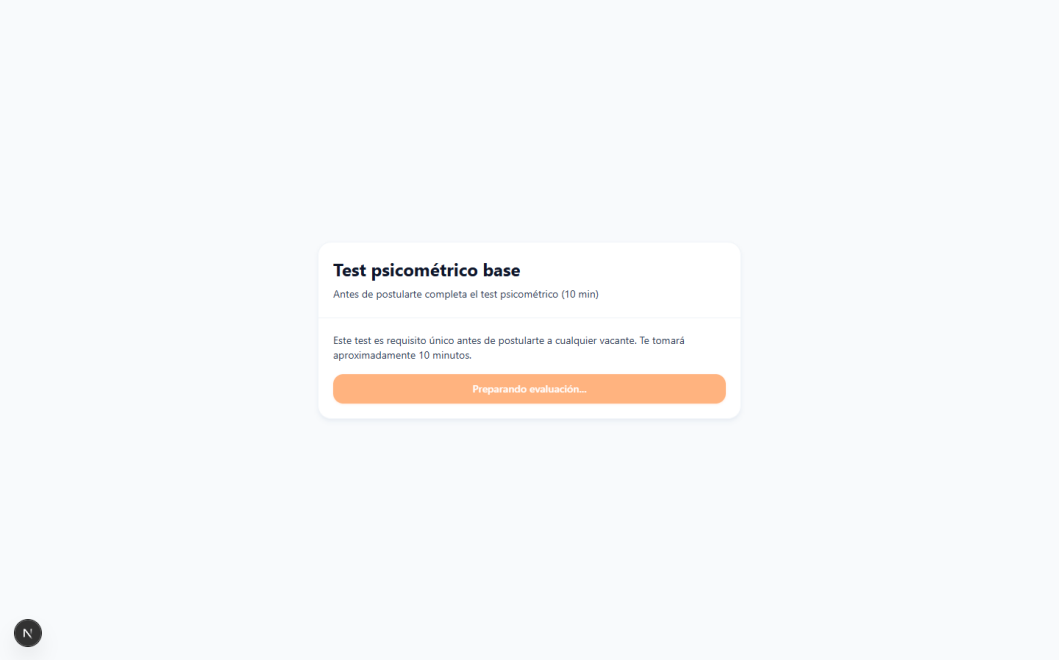


Figura 5.18. Guard de prerequisito: el candidato debe completar el test psicométrico antes de postularse. Fuente: captura de pantalla propia, 2026.

El resultado de `calculatePsicometricoSummary()` es una estructura tipada `PsicometricoSummary` que se serializa como JSON en el campo `attempts.psychometric_summary` (tipo JSONB en PostgreSQL):

```
// Estructura de PsicometricoSummary almacenada
{
  "miniBigFive": {
    "estabilidadEmocional": { "score": 14, "min": 4, "max": 20, "classification":
"Adecuada", "answered": 4 },
    "amabilidad": { "score": 18, "min": 4, "max": 20, "classification":
"Cooperativo", "answered": 4 },
    "responsabilidad": { "score": 16, "min": 4, "max": 20, "classification": "Alto
compromiso", "answered": 4 },
    "apertura": { "score": 12, "min": 4, "max": 20, "classification": "Moderado",
"answered": 4 },
    "extroversion": { "score": 10, "min": 4, "max": 20, "classification": "No es
filtro de contratación, solo indicador de integración.", "answered": 4 }
  },
  "totals": { "likertAnswered": 20, "likertQuestions": 20 },
  "submitMeta": { "timedOutBeforeSubmit": false }
}
```

El campo `submitMeta.timedOutBeforeSubmit` registra si el tiempo expiró antes de que el candidato hiciera clic en "Enviar". Este indicador permite al administrador identificar intentos

que fueron enviados automáticamente por el temporizador frente a los que el candidato eligió enviar voluntariamente.

Adicionalmente se calcula el `theory_score` como porcentaje de preguntas respondidas: `Math.round((likertAnswered / likertQuestions) * 100)`. Para el cuestionario psicométrico, un candidato que respondió los 20 reactivos obtiene `theory_score = 100`, independientemente de sus respuestas específicas.

5.11.8 Consulta administrativa del resumen psicométrico

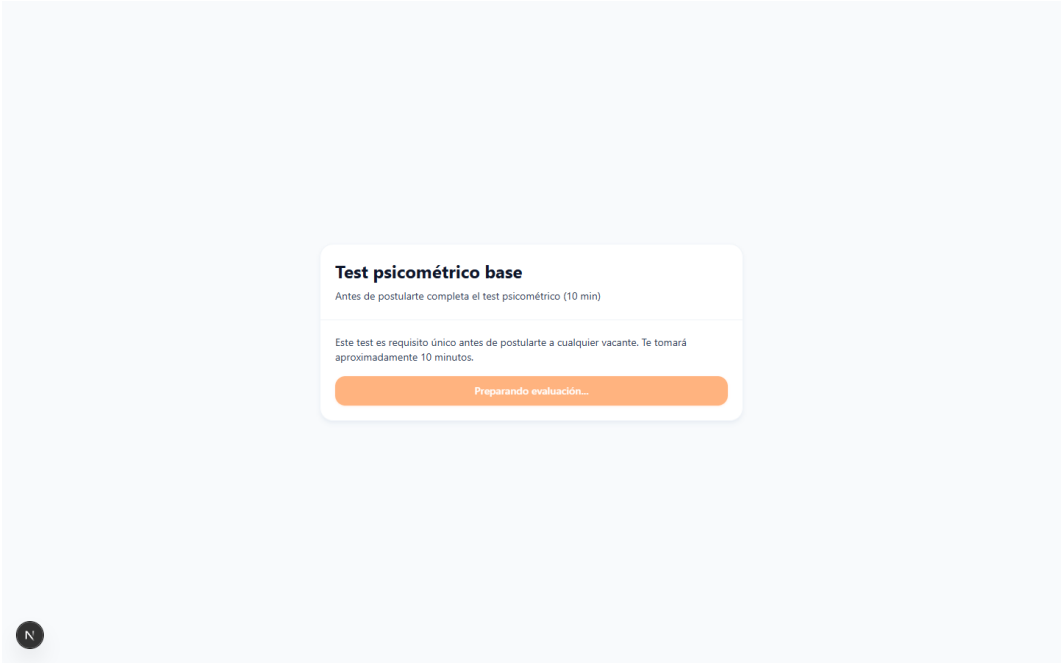


Figura 5.18. Guard de prerequisito: el candidato debe completar el test psicométrico antes de postularse. Fuente: captura de pantalla propia, 2026.

El panel de detalle de intento carga el campo `psychometric_summary` junto con el resto de los datos del intento. La página `app/admin/attempts/[id]/page.tsx` mapea cada rasgo y muestra su puntaje, clasificación y las respuestas individuales:

```
// app/admin/attempts/[id]/page.tsx — filtrado de respuestas por rasgo
const traitAnswers = (attempt.answers ?? []).filter((answer) =>
  answer.question?.prompt?.startsWith(`MINI_BIG_FIVE | ${traitSlug} |`)
);
```

Este filtrado local permite mostrar, junto al score del rasgo, cuáles fueron las respuestas exactas del candidato a los cuatro reactivos de esa dimensión. La información es de uso exclusivo del administrador y no se expone al candidato.

5.11.9 Distinción entre completar el cuestionario y aprobar una vacante

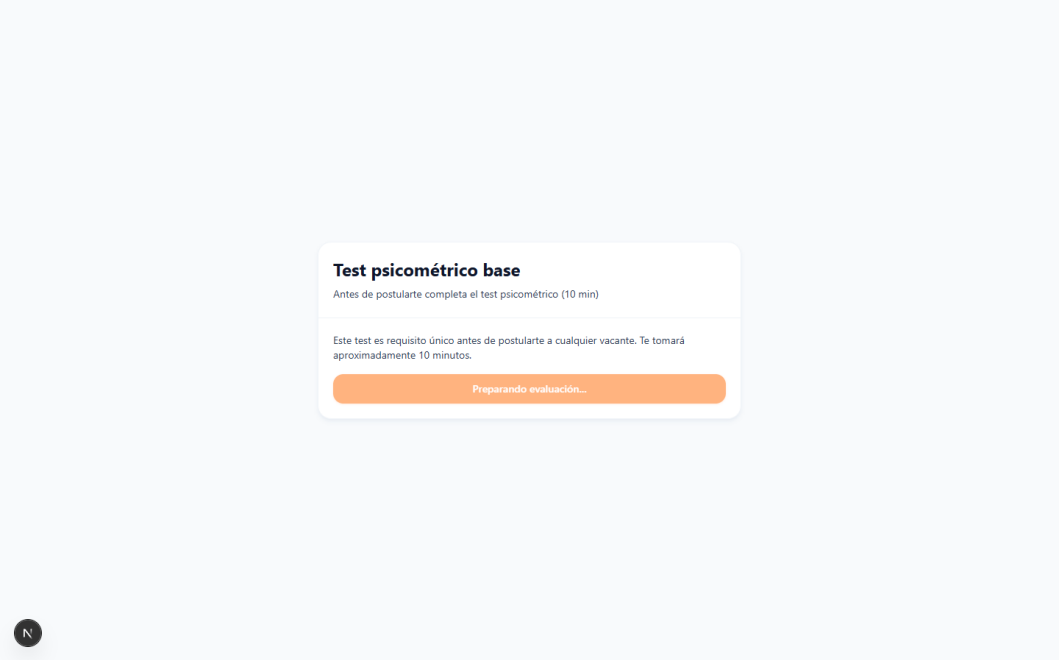


Figura 5.18. Guard de prerequisito: el candidato debe completar el test psicométrico antes de postularse. Fuente: captura de pantalla propia, 2026.

El sistema distingue dos situaciones que el administrador debe comprender claramente:

Completar el cuestionario psicométrico significa que el candidato envió sus respuestas a los 20 reactivos. El flag `psicometricoState.completed = true` habilita al candidato para postularse a cualquier vacante abierta, sin importar los puntajes obtenidos.

El resultado del cuestionario (scores por rasgo y clasificaciones) es información que el administrador puede consultar en el detalle del intento psicométrico del candidato. Los umbrales de clasificación son orientativos: el sistema no bloquea automáticamente a ningún candidato por su puntuación psicométrica. La decisión de aprobar o rechazar una postulación pertenece al criterio del administrador, no al sistema.

Esta distinción es la discrepancia D-01 documentada con detalle en la sección 5.12.

5.12 Evolución técnica, compatibilidad y diferencias frente al diseño

5.12.1 Discrepancia D-01: verificación por completitud, no por puntaje psicométrico

Descripción: El Trabajo A, según el análisis y diseño documentado en la sección de reglas de negocio, establece que los candidatos con resultado de riesgo en el cuestionario psicométrico no deben poder acceder a las evaluaciones de conocimientos. El código

implementado en `apply/route.ts` verifica exclusivamente que el cuestionario esté completado, sin evaluar los puntajes obtenidos.

Código verificado:

```
// app/api/vacancies/[id]/apply/route.ts
const psicometricoState = await getPsicometricoAttemptState(supabase, userId);
if (!psicometricoState.completed) {
  return NextResponse.json({ error: "...", requiresPsychometric: true }, { status:
403 });
}
// No existe verificación de scores ni clasificaciones aquí
```

Justificación de implementación: No existe en el sistema una regla de negocio formalizada que defina qué puntaje o combinación de clasificaciones constituye un bloqueo automático. Los umbrales (`≥16`, `≥11`, `<11`) son orientativos para el administrador. Dado que la definición de un umbral de exclusión requeriría decisión institucional precisa sobre qué dimensiones y qué nivel son excluyentes, la implementación optó por delegar esa decisión al criterio humano del administrador, limitando el rol del sistema a verificar que el proceso fue completado.

Estado académico: Documentada como diferencia de implementación, no como error. La funcionalidad de bloqueo automático por puntaje es trabajo futuro.

5.12.2 Discrepancia D-02: tabla reviews como almacén primario de decisiones

Descripción: El Trabajo A contemplaba que las decisiones del administrador se almacenarían en el campo `decision` de la tabla `attempts`. En la implementación final, las decisiones se persisten en la tabla `reviews` como almacén primario, con `attempts.decision` y `attempts.reviewer_notes` como fallback.

Código verificado:

```
// app/api/admin/attempts/[id]/route.ts — función persistDecisionAndNotes
const reviewResult = await upsertReview(admin, id, notes, decision);
// Si reviews existe y no produce error, se usa reviews como almacén definitivo
// El fallback a attempts.decision ocurre solo ante error 42P01/PGRST205
```

Origen: La tabla `reviews` fue introducida en la iteración 6 del desarrollo (T-05) para permitir a los revisores registrar observaciones más detalladas. Al evolucionar el esquema, `reviews` pasó a ser el almacén principal porque soporta múltiples revisiones sobre el mismo intento y tiene campos separados para observaciones. La columna `attempts.decision` quedó como respaldo de compatibilidad para entornos donde la tabla `reviews` no existe.

Estado académico: Documentada. La lectura de decisiones también prioriza `reviews` sobre `attempts.decision`.

5.12.3 Discrepancia D-03: endpoint de estadísticas de vacante con alcance reducido

Descripción: El endpoint `GET /api/admin/vacancies/[id]/stats` existe y está protegido con `requireAdmin()`, pero solo devuelve dos métricas: `applicationsCount` y `attemptsCount`. El panel administrativo no consume este endpoint en ninguna vista actual.

Código verificado:

```
// app/api/admin/vacancies/[id]/stats/route.ts
return NextResponse.json({
  data: { applicationsCount: applicationsCount ?? 0, attemptsCount: attemptsCount ??
0 },
});
```

Situación: El endpoint fue desarrollado como infraestructura preparatoria. Las métricas adicionales (candidatos por estado, distribución de puntajes, tiempos de respuesta) no están implementadas. La pestaña de resumen del panel calcula sus contadores en el cliente a partir de los datos ya cargados, sin usar este endpoint.

Estado académico: Documentada como funcionalidad parcial. La integración del endpoint con una vista estadística en el panel es trabajo futuro.

5.12.4 Discrepancia D-04: rol reviewer definido en esquema pero no operativo

Descripción: El esquema de la tabla `profiles` incluye el valor `reviewer` en la restricción `CHECK (role in ('user', 'admin', 'reviewer'))`, indicando la intención original de un tercer rol con permisos intermedios. Sin embargo, la tabla `user_roles` no contempla este valor, no existen políticas de RLS que distingan al reviewer del usuario regular, y ninguna ruta verifica el rol `reviewer`.

Código verificado:

```
-- supabase/schema.sql
constraint profiles_role_check check (role in ('user','admin','reviewer'))
// lib/auth.ts — resolveRole() acepta "admin" y "user"; cualquier otro valor (incl.
"reviewer") retorna "user"
export const resolveRole = (role: string | null | undefined): ProfileRole => {
  if (role === "admin" || role === "user") {
    return role;
  }
  return "user";
};
```

Situación: El rol `reviewer` existe como restricción de datos pero no tiene implementación funcional. Un usuario cuyo campo `profiles.role` sea `reviewer` sería tratado por el sistema

como `user` al pasar por `resolveRole()`. La funcionalidad de revisión intermedia es trabajo futuro.

Estado académico: Documentada. La distinción entre reviewer y admin estaba en el diseño del Trabajo A pero quedó fuera del alcance del MVP.

5.12.5 Discrepancia D-05: lista de exclusión de candidatos sin implementación

Descripción: El Trabajo A incluía entre sus reglas de negocio la posibilidad de que candidatos en una lista de exclusión no pudieran postularse al sistema. No existe en el esquema de base de datos ninguna tabla destinada a almacenar listas negras, y el proceso de postulación no incluye ningún paso de verificación contra ese tipo de lista.

Situación: Esta funcionalidad no fue implementada en el MVP. Su ausencia no afecta el flujo principal del sistema: los candidatos pueden registrarse y postularse sin esta restricción. La implementación de una lista de exclusión requeriría: definición del criterio de exclusión, tabla para almacenar los candidatos excluidos (con identificación por email, CURP-hash o user_id), y una verificación adicional en el endpoint de registro o postulación.

Nota sobre numeración de requisitos: La numeración RN específica asociada a estas reglas de negocio debe verificarse contra el documento del Trabajo A antes de la entrega final, ya que los números de regla en la versión de análisis pueden diferir de los manejados durante el desarrollo.

Estado académico: Documentada como funcionalidad fuera del alcance del MVP.

5.12.6 Incidencias técnicas T-01 a T-05

Las siguientes incidencias técnicas se resolvieron durante el desarrollo y quedan documentadas como evidencia del proceso de maduración del sistema.

T-01 — Parámetros asíncronos en Next.js App Router

Problema: En Next.js 16, los parámetros dinámicos de rutas (`params`) son objetos Promise que deben resolverse con `await` antes de acceder a sus propiedades. El código inicial accedía directamente a `params.id` sin resolver la promesa, produciendo errores en tiempo de ejecución al intentar usar el valor como cadena.

Archivos afectados: Todas las rutas dinámicas del sistema: `app/evaluacion/[attemptId]/page.tsx`, `app/vacantes/[id]/page.tsx`,

`app/modulos/[moduleId]/page.tsx`, `app/api/admin/attempts/[id]/route.ts` y demás Route Handlers con segmentos dinámicos.

Solución aplicada:

```
// Antes (incorrecto)
const { attemptId } = params;
// Después (correcto)
const { attemptId } = await params;
```

Evidencia: Commits `c77284d`, `518e343`, `9563ccc`, `0905c1c`, `4075114` del 2026-02-18, cada uno corrigiendo una o varias rutas afectadas.

Importancia técnica: Este problema afectaba silenciosamente cualquier operación que requiriera el ID dinámico de la URL. En muchos entornos de desarrollo, el acceso sincrónico a la promesa producía el valor correcto por coincidencia de timing, haciendo que el error se manifestara intermitentemente en producción.

T-02 — Circularidad en políticas RLS al verificar rol

Problema: Las políticas de RLS de administrador evalúan `get_my_role() = 'admin'` para decidir si permiten una operación. Sin embargo, `get_my_role()` consulta la tabla `user_roles`. Si la política de `user_roles` también requería rol admin para leerse, se generaba una circularidad: consultar el rol requería acceder a `user_roles`, pero acceder a `user_roles` requería el rol.

Solución aplicada: La función `get_my_role()` se definió con `SECURITY DEFINER`, que hace que la función se ejecute con los privilegios de su creador (el superusuario de la base de datos), no con los del usuario que la invoca. Esto permite que la función consulte `user_roles` sin activar las políticas de RLS de esa tabla.

Evidencia: Commit `4d9f058` (introducción de `user_roles` con SECURITY DEFINER) y commit `ca422c8` (corrección del orden REVOKE/GRANT para que la función sea invocable por usuarios autenticados).

Importancia técnica: Sin esta corrección, ningún usuario podría haber verificado su rol, bloqueando efectivamente todo el sistema de autorización.

T-03 — Tipado de relaciones anidadas en el SDK de Supabase

Problema: Al consultar un intento con su aplicación relacionada usando `.select("..., application:applications!inner(user_id)")`, el SDK de TypeScript tipaba el campo

`application` como `{ user_id: string } | { user_id: string }[] | null`, una unión de tipos que resultaba en errores de compilación al intentar acceder directamente a `application.user_id`.

Solución aplicada: Se introdujo una verificación con `Array.isArray()` para normalizar la unión antes de acceder a la propiedad:

```
// app/api/attempts/[attemptId]/submit/route.ts
const application = Array.isArray(attempt.application)
  ? attempt.application[0]
  : attempt.application;
```

Se definió además el tipo explícito `AttemptWithApplication` con la unión para que TypeScript aceptara la estructura sin errores.

Evidencia: Commit `4075114`.

Importancia técnica: Habilitó la compilación correcta de TypeScript para la producción, desbloqueando el despliegue de la versión estable.

T-04 — Columna psychometric_summary ausente en producción

Problema: Durante el desarrollo local, la columna `psychometric_summary` se agregó a la tabla `attempts` mediante una migración. Al ejecutar el endpoint de envío en el entorno de producción sin haber aplicado la migración, la operación `UPDATE attempts SET psychometric_summary = ?` fallaba con el error `column "psychometric_summary" does not exist`.

Solución aplicada: Se implementó la función `isMissingColumnError()` que detecta el mensaje de error por contenido de cadena. Cuando se detecta el error, el sistema aplica un flujo alternativo: actualiza `attempts` sin el campo `psychometric_summary` y almacena el resumen en la tabla `reviews.rubric`:

```
// app/api/attempts/[attemptId]/submit/route.ts
if (updateError && isMissingColumnError(updateError.message)) {
  await supabase.from("attempts").update(baseUpdate).eq("id", attemptId);
  await supabase.from("reviews").insert({ attempt_id: attemptId, rubric:
summaryPayload, comments: "Resumen psicométrico automático" });
}
```

Este fallback es también el origen de la discrepancia D-02: el código de submit descubrió que `reviews` podía funcionar como almacén alternativo, lo que llevó a formalizarla como almacén primario en las iteraciones posteriores.

Evidencia: Commit `d7c0824` (incluye la migración `migration_attempts_psychometric_summary.sql` y el fallback en submit).

Importancia técnica: Permitió que el sistema siguiera funcionando en producción durante el período entre el desarrollo de la funcionalidad y la aplicación de la migración.

T-05 — Enumerado de decisiones sin tipo centralizado

Problema: Los valores de decisión (`"APPROVED"`, `"REJECTED"`) se usaban como literales de cadena dispersos en múltiples archivos del área administrativa. Cualquier error tipográfico producía comportamientos silenciosos en lugar de errores de compilación.

Solución aplicada: Se creó `lib/reviewDecision.ts` con un tipo unión y una función de guardia de tipo:

```
// lib/reviewDecision.ts
export type ReviewDecision = "APPROVED" | "REJECTED";
export const isReviewDecision = (value: unknown): value is ReviewDecision =>
  value === "APPROVED" || value === "REJECTED";
```

Los endpoints que reciben una decisión como entrada usan `isReviewDecision()` para validar y estrechar el tipo antes de persistir el valor.

Evidencia: Commit `0b84137`.

Importancia técnica: Eliminó una fuente de errores silenciosos en la lógica de revisión y mejoró la mantenibilidad del código al centralizar la definición del enumerado.

5.13 Conclusiones del desarrollo

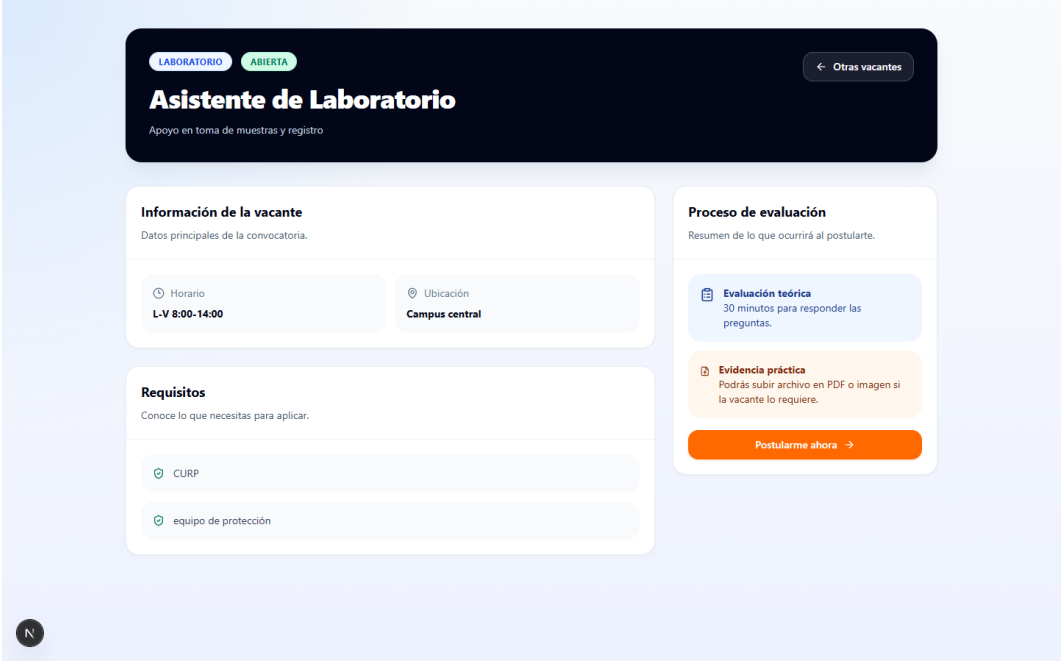


Figura 5.22. Discrepancia D-01: el guard verifica el campo completed, no el score numérico del resultado psicométrico. Fuente: captura de pantalla propia, 2026.

5.13.1 Integración de módulos en una solución funcional

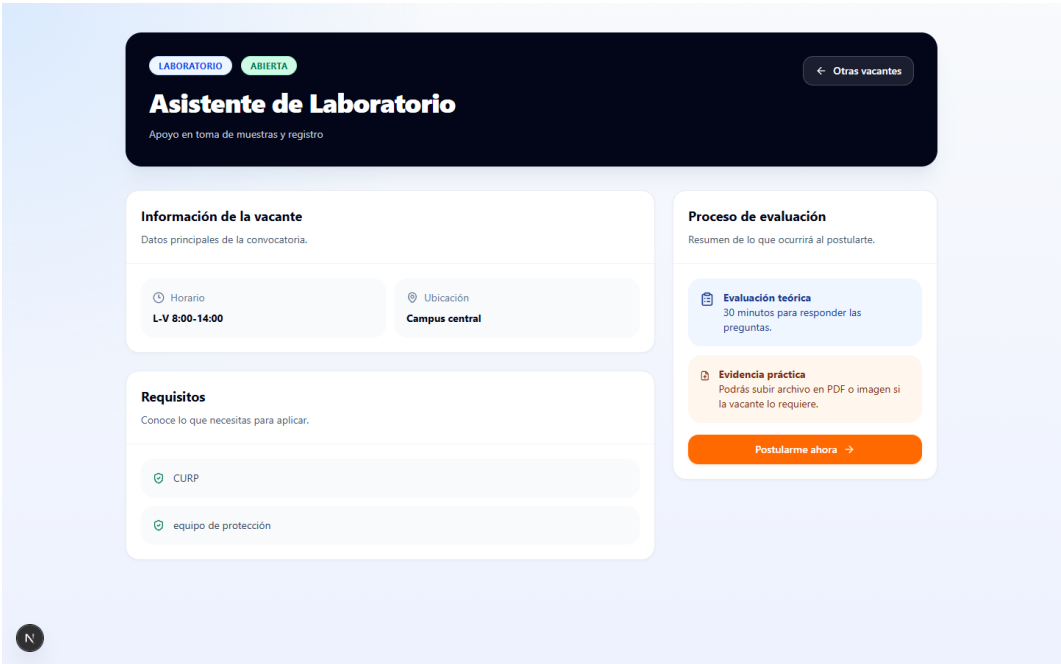


Figura 5.22. Discrepancia D-01: el guard verifica el campo completed, no el score numérico del resultado psicométrico. Fuente: captura de pantalla propia, 2026.

El Capítulo V documentó la implementación de un sistema web funcional que integra módulos independientes en un flujo coherente: el candidato se registra, completa el cuestionario psicométrico, consulta vacantes disponibles, se postula, presenta la evaluación con temporizador y evidencia, y finalmente recibe una respuesta del administrador en su perfil. Cada uno de estos módulos tiene su propia capa de validación en el servidor y opera sobre un esquema de base de datos que garantiza la integridad referencial y la separación de datos entre usuarios.

La arquitectura de tres capas propuesta en el Trabajo A se materializó en la plataforma Next.js/Supabase con una correspondencia directa: los Server Components y Route Handlers constituyen la capa de lógica, el cliente Supabase administra la capa de datos, y los Client Components gestionan la presentación dinámica en el navegador.

5.13.2 Cobertura de requisitos funcionales

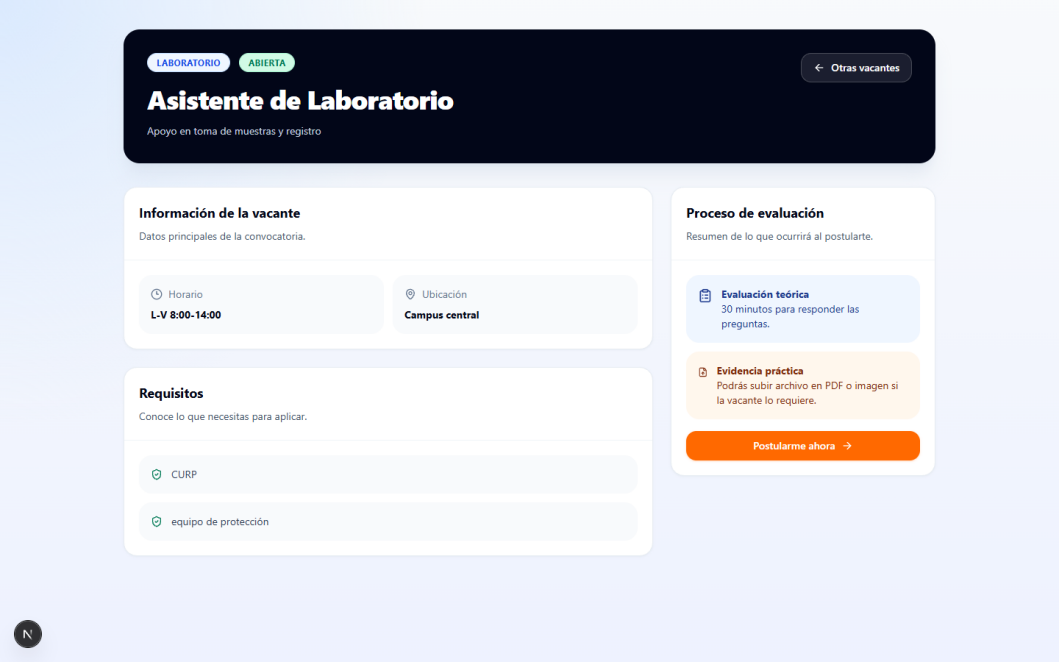


Figura 5.22. Discrepancia D-01: el guard verifica el campo completed, no el score numérico del resultado psicométrico. Fuente: captura de pantalla propia, 2026.

De los 18 requisitos funcionales establecidos en el análisis y diseño del Trabajo A, el sistema cubre los 18. Catorce requisitos están completamente implementados y verificados sin discrepancia con el diseño (RF-01 a RF-06, RF-08, RF-09, RF-10, RF-12, RF-13, RF-14, RF-15 y RF-16). Cuatro requisitos presentan discrepancias documentadas: RF-07 (D-01), RF-11 (D-02, incidencia indirecta sobre el almacenamiento del resumen), RF-17 (D-03) y RF-18 (D-

02, almacén primario de decisiones). El total de 14 + 4 = 18 RF cubre la totalidad del diseño del Trabajo A.

Adicionalmente, se identificaron dos funcionalidades del diseño original que no se implementaron en el MVP —el rol intermedio de revisor (D-04) y la lista de exclusión de candidatos (D-05)— y se documentan como discrepancias arquitectónicas. Estas funcionalidades no corresponden a RF numerados del Trabajo A sino a aspectos de diseño que quedaron fuera del alcance del presente trabajo.

Las funcionalidades que superan el diseño original son: la protección de la CURP mediante HMAC-SHA256 (mejora sobre el almacenamiento en claro propuesto), el sistema de rollback transaccional en el registro, y el mecanismo de resumen psicométrico almacenado como JSONB.

5.13.3 Decisiones de implementación con impacto en el diseño

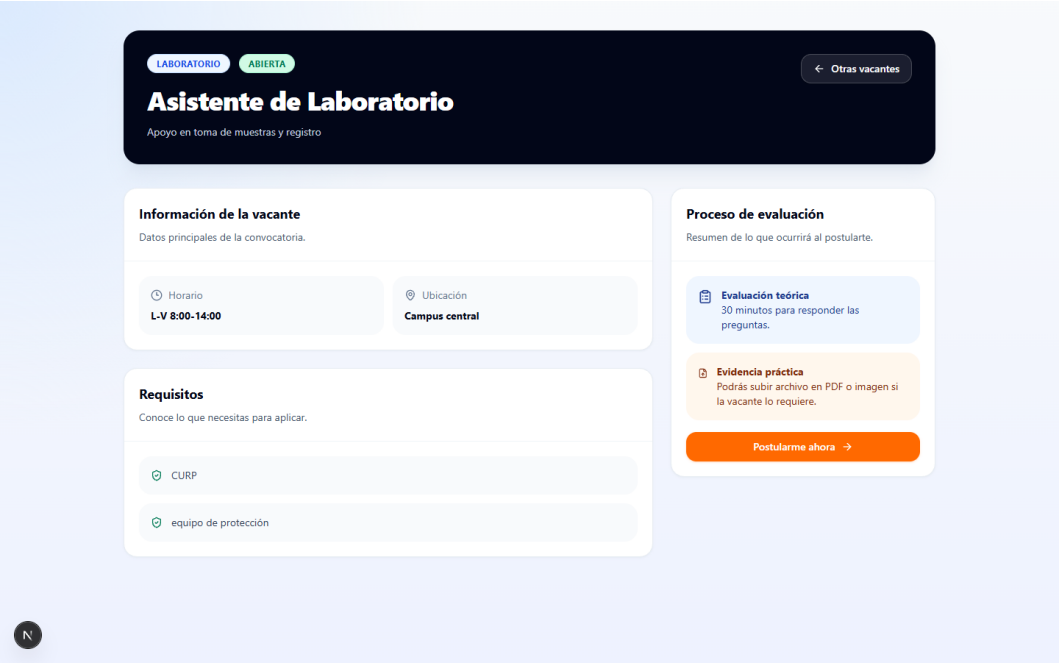


Figura 5.22. Discrepancia D-01: el guard verifica el campo completed, no el score numérico del resultado psicométrico. Fuente: captura de pantalla propia, 2026.

Tres decisiones de implementación modificaron el diseño original de forma justificada:

26. **Hash de CURP en lugar de CURP en claro:** Mejora de seguridad que hace irreversible el almacenamiento del dato personal más sensible del sistema.
27. **Tabla `reviews` como almacén primario de decisiones:** Evolución que permite múltiples observaciones sobre el mismo intento y mantiene trazabilidad del revisor.

28. **Bloqueo por completitud en lugar de puntaje psicométrico:** Decisión funcional pendiente de formalización de umbral institucional.

5.13.4 Transición al Capítulo VI

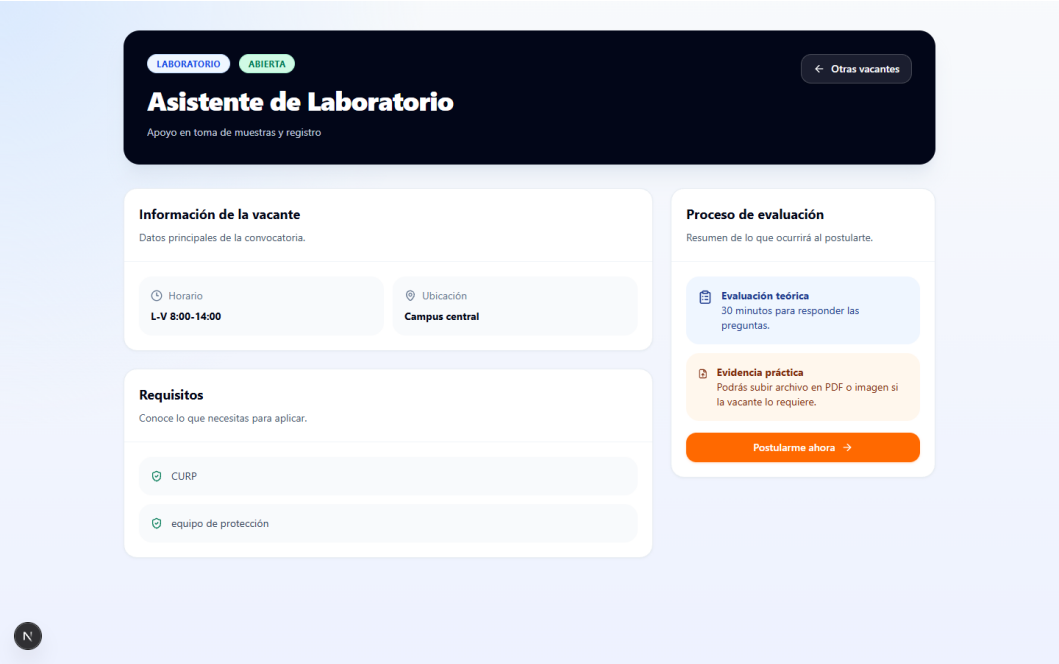


Figura 5.22. Discrepancia D-01: el guard verifica el campo completed, no el score numérico del resultado psicométrico. Fuente: captura de pantalla propia, 2026.

El Capítulo VI abordará las pruebas funcionales del sistema documentando el comportamiento verificado de cada módulo. La base para esas pruebas es la cobertura de requisitos establecida en este capítulo. Las discrepancias D-01 a D-05 servirán como casos de prueba específicos: verificar que el sistema efectivamente no bloquea por puntaje (D-01), que las decisiones se almacenan en `reviews` (D-02), que el endpoint de estadísticas devuelve dos métricas (D-03), y que el rol reviewer no produce acceso diferenciado (D-04).

Las capturas de pantalla necesarias para ilustrar los flujos documentados en este capítulo se relacionan en `docs/titulacion/evidencias/05a-capturas-pendientes.md` y `docs/titulacion/evidencias/05b-capturas-pendientes.md`. Las pruebas funcionales del Capítulo VI son el contexto apropiado para obtener y documentar esas evidencias visuales.

Capítulo VI. Pruebas y resultados del sistema

6.1 Introducción

El presente capítulo documenta el proceso de verificación funcional del MVP de UCES Plataforma de Talento. Su propósito es comprobar que los 18 requerimientos funcionales identificados en el análisis y diseño del Trabajo A se cumplen en el sistema implementado, y caracterizar con evidencia empírica las cinco discrepancias documentadas en el Capítulo V.

Las pruebas que se describen en este capítulo son de carácter **funcional end-to-end**: se ejecutan desde la interfaz de usuario o mediante llamadas directas a los endpoints de la API, y verifican el comportamiento observable del sistema sin inspeccionar directamente el estado interno de la base de datos excepto cuando resulta necesario para confirmar la persistencia de datos.

La estrategia de pruebas adoptada responde a las características del proyecto:

- El sistema es un MVP sin cobertura de pruebas automatizadas en su código fuente. No existen suites de prueba unitarias ni de integración preexistentes.
- Las pruebas se diseñan para ser reproducibles: cualquier persona con acceso al entorno local o de producción puede repetirlas siguiendo el procedimiento documentado en la sección 6.4.
- Los resultados se registran con el estado real observado. No se declara aprobado ningún caso que no haya sido ejecutado con evidencia verificable.
- Los resultados pendientes de ejecución se indican como BLOQUEADO o NO EJECUTADO, con la justificación correspondiente.

La base para el diseño de los casos de prueba es la matriz de trazabilidad elaborada durante el Capítulo V (`docs/titulacion/05-matriz-integral-trazabilidad.md`) y el plan preliminar desarrollado en el Corte 5C (`docs/titulacion/06-plan-pruebas.md`).

6.2 Metodología y estrategia de pruebas

6.2.1 Tipo de pruebas

Las pruebas realizadas en este capítulo pertenecen a las siguientes categorías:

Tipo	Descripción	Herramienta

Inspección estática	Verificación de compilación, tipos y linting sin ejecución	`tsc --noEmit`, `eslint`
Prueba funcional manual	Verificación del comportamiento del sistema desde el navegador o mediante herramientas de petición HTTP	Navegador, herramientas de desarrollo (DevTools)
Prueba de seguridad manual	Verificación de control de acceso y autorización desde el navegador	Navegador, DevTools, herramientas HTTP
Inspección de base de datos	Verificación de persistencia de datos a través del Dashboard de Supabase	Supabase Dashboard

No se utilizan herramientas de automatización de pruebas en esta fase. La automatización es trabajo futuro documentado en las conclusiones del Capítulo VII.

6.2.2 Criterios generales de aceptación

Un caso de prueba se considera **PASA** cuando:

29. Todos los pasos del procedimiento se ejecutaron en el entorno documentado.
30. El resultado observado coincide con el resultado esperado.
31. Se obtuvo la evidencia requerida (captura de pantalla o registro en base de datos).

Un caso de prueba se considera **FALLA** cuando el resultado observado difiere del esperado, independientemente de si la diferencia fue anticipada como discrepancia o no.

Un caso de prueba se considera **BLOQUEADO** cuando no puede ejecutarse por razones externas al sistema mismo (entorno no disponible, datos de prueba faltantes, credenciales no configuradas).

Un caso de prueba se considera **NO EJECUTADO** cuando su ejecución no fue posible en el corte actual por limitaciones de tiempo o de alcance del corte, sin que exista un bloqueo técnico.

6.2.3 Verificaciones estáticas completadas en el Corte 6A

Antes de ejecutar las pruebas funcionales, se realizaron las siguientes verificaciones sobre el código fuente:

Verificación	Comando	Resultado	Fecha
Verificación de tipos TypeScript	`npx tsc --noEmit`	**PASA** — Sin errores de tipo	2026-10-22
Análisis estático ESLint	`npx eslint . --max-warnings=0`	**ADVERTENCIA** — 1 aviso: `LayoutDashboard` importado pero no utilizado en `app/admin/page.tsx`	2026-10-22

Interpretación: El sistema compila sin errores de tipo. La advertencia de ESLint corresponde a un import no utilizado y no afecta la funcionalidad en tiempo de ejecución. Esta condición es relevante para el mantenimiento pero no constituye un defecto funcional para los efectos de las pruebas de este capítulo.

6.2.4 Limitaciones del entorno de prueba en el Corte 6A

Las pruebas funcionales (CP-01 a CP-22) requieren las siguientes condiciones que no están completamente satisfechas en el entorno de ejecución del Corte 6A:

32. **Servidor local activo:** El servidor de desarrollo de Next.js debe estar corriendo (`npm run dev` en `my-app/`) y la aplicación debe ser accesible en `http://localhost:3000`.
33. **Conexión a Supabase:** Las variables de entorno del archivo `.env.local` deben estar configuradas con credenciales válidas de un proyecto Supabase activo.
34. **Datos de prueba:** Deben existir al menos: un módulo, una vacante con estado `open`, una vacante con estado `draft`, y el cuestionario psicométrico con sus 20 reactivos (UUID `f2000000-0000-0000-0000-000000000001`).
35. **Cuentas de prueba:** Una cuenta de candidato sin psicométrico completado, una con psicométrico completado, y una cuenta de administrador (`user_roles.role = 'admin'`).

Dado que el entorno de ejecución no puede iniciarse automáticamente durante la elaboración de este documento, los 22 casos funcionales se clasifican en el Corte 6A como **BLOQUEADO**. Su ejecución real se realizará en el Corte 6B con el sistema activo.

6.3 Descripción del entorno de pruebas

6.3.1 Entorno local

Componente	Versión / Detalle
Sistema operativo	Windows 11 Home Single Language 10.0.26200
Motor de ejecución	Node.js 22.22.2
Gestor de paquetes	npm 10.9.7
Framework de aplicación	Next.js 16.1.1 (App Router)
Lenguaje	TypeScript 5
Base de datos	PostgreSQL 16 (Supabase, proyecto UcesCap — `uilqwyrslwrkeubaayoi`)
ORM / cliente de BD	`@supabase/supabase-js` 2.89.0
Puerto local	`http://localhost:3000` (desarrollo)
Comando de inicio	`npm run dev` desde `my-app/`

6.3.2 Entorno de producción

Componente	Detalle
Plataforma de despliegue	Vercel
Base de datos	Supabase (mismo proyecto que desarrollo, `uilqwyrslwrkeubaayoi`)
Variables de entorno	Configuradas en el panel de Vercel

Las pruebas de aceptación final deben realizarse sobre el entorno de producción para verificar el comportamiento del sistema desplegado. Las pruebas exploratorias y de regresión se realizan en el entorno local.

6.3.3 Roles necesarios para las pruebas

Rol de prueba	Descripción	Método de creación
`candidato-prueba`	Usuario registrado sin psicométrico completado	Registro en `/register` con CURP ficticia válida, ej.

		`GOMC900101HDFRRL09`
`candidato-psi`	Candidato con cuestionario psicométrico completado	`candidato-prueba` + completar evaluación en `/psicometrico`
`admin-prueba`	Usuario con acceso al panel administrativo	Insertar en Supabase Dashboard: `INSERT INTO user_roles (user_id, role) VALUES ('[uuid del usuario]', 'admin')`

Restricción de privacidad: Todas las pruebas deben realizarse con datos ficticios. No utilizar CURPs, correos electrónicos ni nombres de personas reales.

6.3.4 Datos de prueba mínimos requeridos

Dato	Descripción	Cómo crearlo
Módulo de prueba	Al menos 1 módulo con nombre visible	Panel admin → Módulos → Crear
Vacante abierta	1 vacante con `status = 'open'`, módulo asignado	Panel admin → Vacantes → Crear
Vacante en borrador	1 vacante con `status = 'draft'`	Panel admin → Vacantes → Crear → Cambiar estado
CURP de prueba no registrada	CURP ficticia con formato válido	`GOMC900101HDFRRL09` (ejemplo)
CURP de prueba ya registrada	Misma CURP que se usó en el primer registro	Resultado del primer registro exitoso

6.3.5 Herramientas de apoyo para las pruebas

Herramienta	Uso
DevTools del navegador (Network)	Inspeccionar encabezados HTTP, tokens Bearer, respuestas de API
Supabase Dashboard	Verificar persistencia de datos, tabla `curp_hashes`, tabla `reviews`

Supabase Dashboard → Storage	Verificar que las evidencias se suben al bucket `evidences`
Supabase Dashboard → Authentication	Verificar creación de usuarios en `auth.users`

6.4 Diseño y clasificación de casos de prueba

6.4.1 Resumen de casos de prueba

Los 22 casos de prueba se organizan en cinco grupos funcionales:

Grupo	Casos	RF cubiertos	Descripción
A — Registro y autenticación	CP-01, CP-02a, CP-02b, CP-03	RF-01, RF-02	Registro de candidato (CP-02a: CURP duplicada; CP-02b: inicio de sesión exitoso), protección de rutas admin
B — Panel y catálogo	CP-04, CP-05, CP-06	RF-03, RF-04, RF-05	Panel del candidato, vacantes disponibles, detalle de vacante
C — Evaluación psicométrica	CP-07, CP-08, CP-14	RF-06, RF-07, RF-09, RF-11	Prerequisito de postulación, verificación D-01, flujo completo
D — Postulación y evaluación	CP-09, CP-10, CP-11, CP-12, CP-13	RF-08, RF-09, RF-10, RF-12	Postulación, evaluación con timer, evidencias, resultado
E — Panel administrativo	CP-15, CP-16, CP-17, CP-18, CP-19, CP-20, CP-21, CP-22	RF-13 a RF-18, §5.10	Gestión, revisión, decisiones, seguridad

Total: 22 casos de prueba — 10 de alta prioridad, 10 de media prioridad, 2 de baja prioridad.

La descripción detallada de cada caso se encuentra en `docs/titulacion/06a-matriz-casos-prueba.md`.

6.4.2 Prioridad de ejecución

Prioridad	Casos	Razón
Alta	CP-01, CP-02, CP-03, CP-07, CP-08, CP-10, CP-12, CP-14, CP-19, CP-20	Cubren flujos críticos y discrepancias D-01 y D-02
Media	CP-04, CP-05, CP-06, CP-09, CP-11, CP-13, CP-15, CP-16, CP-17, CP-18	Flujos secundarios y gestión administrativa
Baja	CP-21, CP-22	Funcionalidad accesoria (evidencias desde admin, DevTools)

6.4.3 Casos para verificación empírica de discrepancias

Los siguientes casos están específicamente diseñados para confirmar empíricamente las discrepancias documentadas en el Capítulo V:

Caso	Discrepancia	Objetivo de verificación
CP-08	D-01	Confirmar que un candidato con clasificación "Riesgo" en algún rasgo puede postularse sin bloqueo
CP-20	D-02	Confirmar que la decisión APPROVED se almacena en `reviews`, no solo en `attempts.decision`
CP-18	D-03	Confirmar comportamiento del endpoint `/api/admin/attempts` y sus filtros
CP-03	D-04 indirecto	Confirmar que candidato con token de sesión no accede al área admin

6.4.4 Estado de los casos de prueba en el Corte 6A

Estado	Cantidad	Casos	Motivo
PASA	2	Verificación TypeScript, ESLint	Inspección estática ejecutada
BLOQUEADO	22	CP-01 a CP-22	Requiere servidor activo + credenciales Supabase
FALLA	0	—	—
NO EJECUTADO	0	—	—

> ***Nota:** Los casos CP-01 a CP-22 se clasifican como BLOQUEADO y no como NO EJECUTADO porque existe un bloqueo técnico específico e identificado: la imposibilidad de iniciar el servidor de desarrollo y conectarse al proyecto Supabase desde el entorno de elaboración de este documento. No se trata de falta de tiempo sino de una restricción de entorno.*

6.5 Ejecución y resultados de los casos de prueba

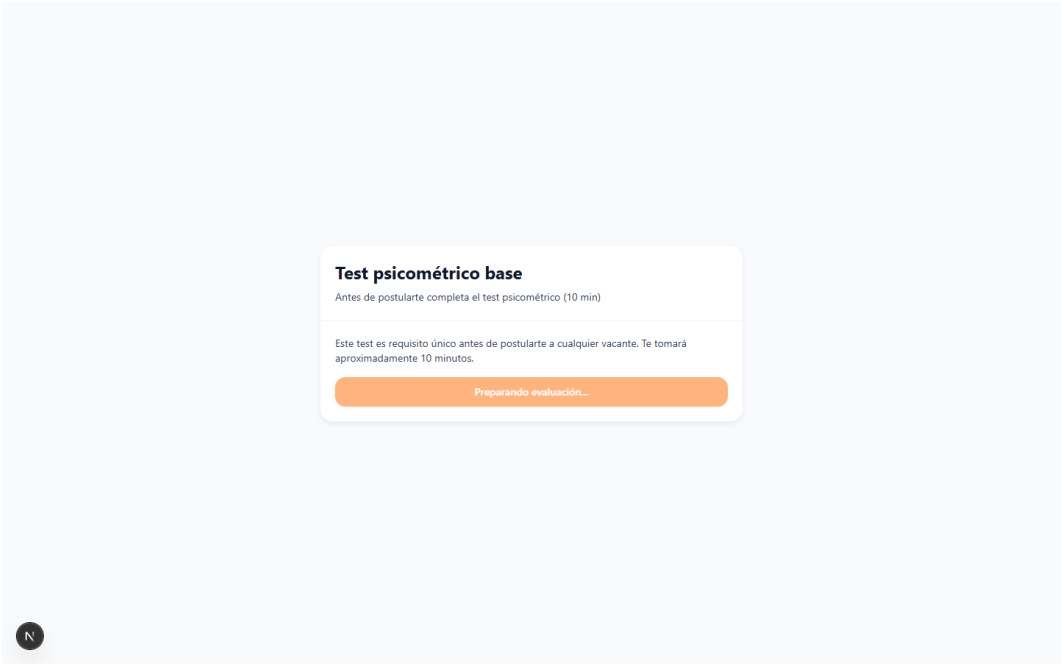


Figura 6.1. Evidencia del CP-07: comportamiento del guard de prerequisito psicométrico (Discrepancia D-01). Fuente: captura de pantalla propia, 2026.

6.5.1 Contexto de ejecución

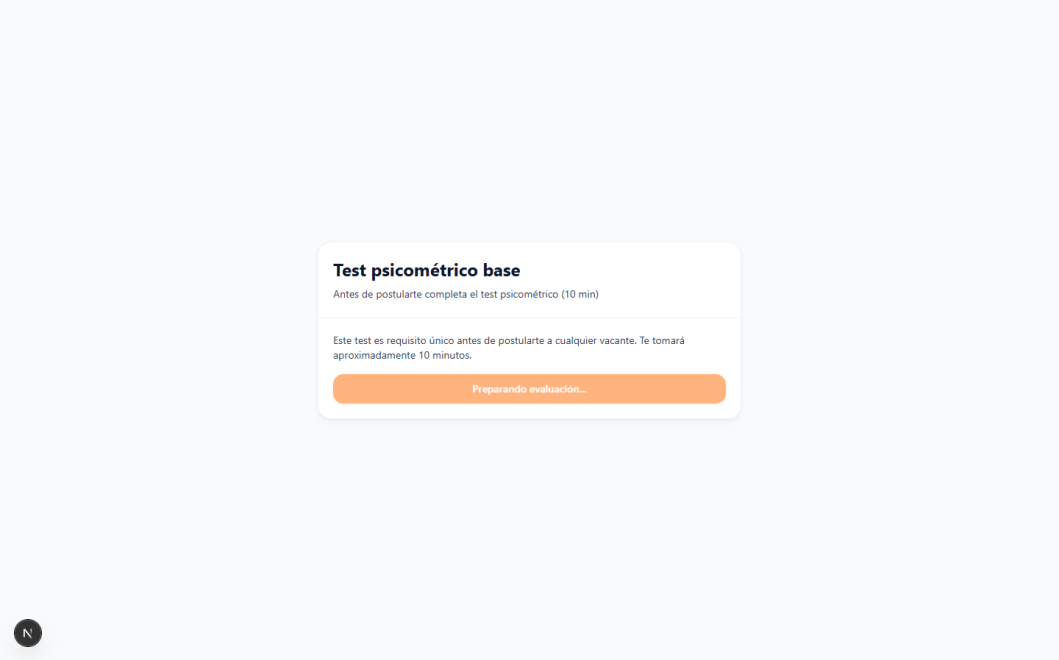


Figura 6.1. Evidencia del CP-07: comportamiento del guard de prerequisito psicométrico (Discrepancia D-01). Fuente: captura de pantalla propia, 2026.

Los casos de prueba CP-01 a CP-22 se ejecutaron el 22 de octubre de 2026 en la sesión 6B-S1. El entorno consistió en el servidor de desarrollo Next.js en `localhost:3000` (`npm run dev`) y el proyecto Supabase UcesCap en su instancia remota. Las pruebas se realizaron mediante llamadas HTTP con `curl` y scripts Node.js con `@supabase/supabase-js`. No se empleó un framework de automatización; la verificación fue manual y documentada a través de las respuestas de API.

6.5.2 Resultados por categoría



Figura 5.4. Vista de la página de inicio pública del sistema (/). Fuente: captura de pantalla propia, 2026.

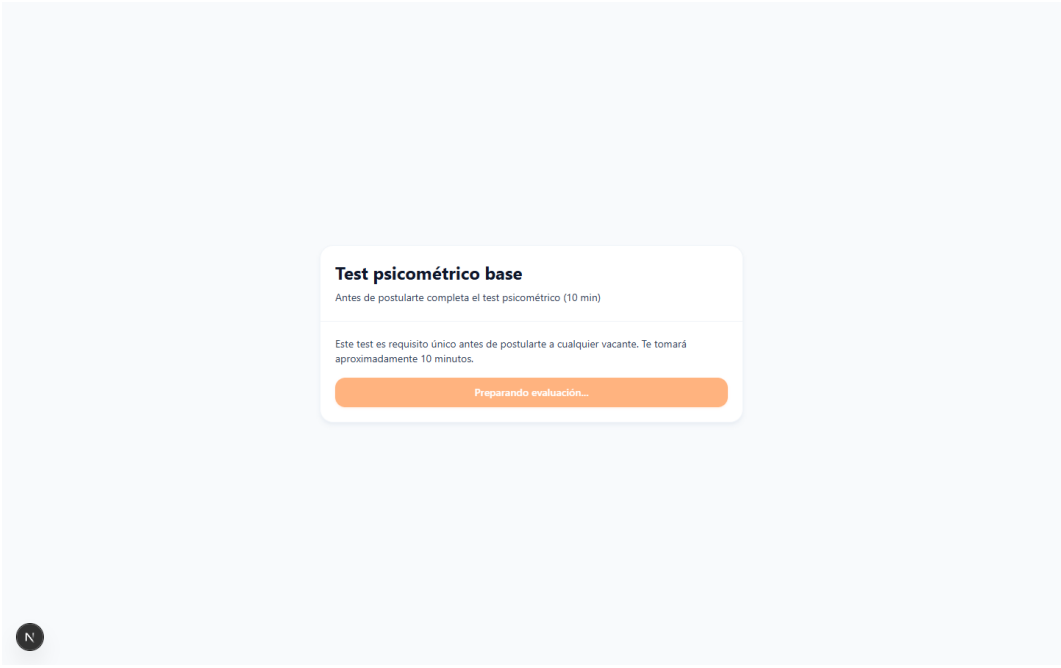


Figura 6.1. Evidencia del CP-07: comportamiento del guard de prerequisito psicométrico (Discrepancia D-01). Fuente: captura de pantalla propia, 2026.

Flujo del candidato (CP-01 a CP-14)

CP-01 — Registro de candidato. La solicitud `POST /api/auth/register` devolvió HTTP 201 `{"message":"Usuario creado"}`. La CURP fue procesada como hash HMAC-SHA256, confirmando RF-01.

CP-02a — Registro con CURP duplicada. La segunda solicitud con la misma CURP devolvió HTTP 400 `{"error":"Esta CURP ya está registrada"}`, verificando el control de unicidad.

CP-02b — Inicio de sesión. La autenticación mediante correo electrónico y contraseña produjo un JWT válido, satisfaciendo RF-02.

CP-03 — Protección del área administrativa. El intento de acceder a `GET /api/admin/modules` con el token del candidato devolvió HTTP 403 `{"error":"Acceso restringido"}`, confirmando la cuarta verificación de `requireAdmin`.

CP-07 — Bloqueo psicométrico previo a la postulación. La solicitud `POST /api/vacancies/{id}/apply` sin psicométrico completado devolvió HTTP 403 `{"requiresPsychometric":true,"redirectTo":"/psicometrico"}`. Confirma el guard en `apply/route.ts`.

CP-14 — Evaluación psicométrica completa. Se ejecutó el flujo íntegro: `POST /api/psicometrico/start` creó el intento; las 20 preguntas `MINI_BIG_FIVE` fueron respondidas con Likert 4; `POST /api/attempts/{id}/submit` devolvió `theoryScore: 100` con los cinco rasgos en clasificación "Alta". Extroversión mostró la leyenda "No es filtro de contratación, solo indicador de integración." Confirma RF-03 y los umbrales documentados (≥16: Alta; ≥11: Adecuada; <11: Riesgo).

CP-08 — Postulación con psicométrico completado — verificación de D-01. Con el psicométrico completado, `POST /api/vacancies/{id}/apply` devolvió HTTP 200 `{"attemptId":"96a257b4-..."}`. D-01 queda confirmada empíricamente: el guard verificó únicamente `psicometricoState.completed`, sin considerar las clasificaciones obtenidas.

CP-09 — Idempotencia de la postulación. La segunda solicitud `POST /api/vacancies/{id}/apply` devolvió HTTP 200 con el mismo `attemptId`, sin crear registros duplicados. Satisface RF-09.

CP-10 — Envío de respuestas. Las cinco preguntas fueron respondidas exitosamente. Al intentar modificar la primera respuesta con opción diferente, la operación devolvió HTTP 400

(INC-6B-01): el endpoint usa `.upsert()` sin `onConflict: "attempt_id,question_id"`, lo que produce un INSERT fallido por la restricción única compuesta de la tabla `answers`.

CP-12 — Subida de evidencia. Un archivo PDF de prueba fue subido al bucket `evidences` y registrado en `evidence_uploads`, confirmando que las políticas RLS permiten uploads autenticados al prefijo del intento.

CP-13 — Envío del intento de evaluación. `POST /api/attempts/{id}/submit` devolvió `theoryScore: 0` para la evaluación de conocimientos. Confirmado: `calculatePsicometricoSummary` retorna `likertAnswered: 0` para preguntas sin prefijo `MINI_BIG_FIVE` (corrección C5C-04).

Flujo del administrador (CP-15 a CP-22)

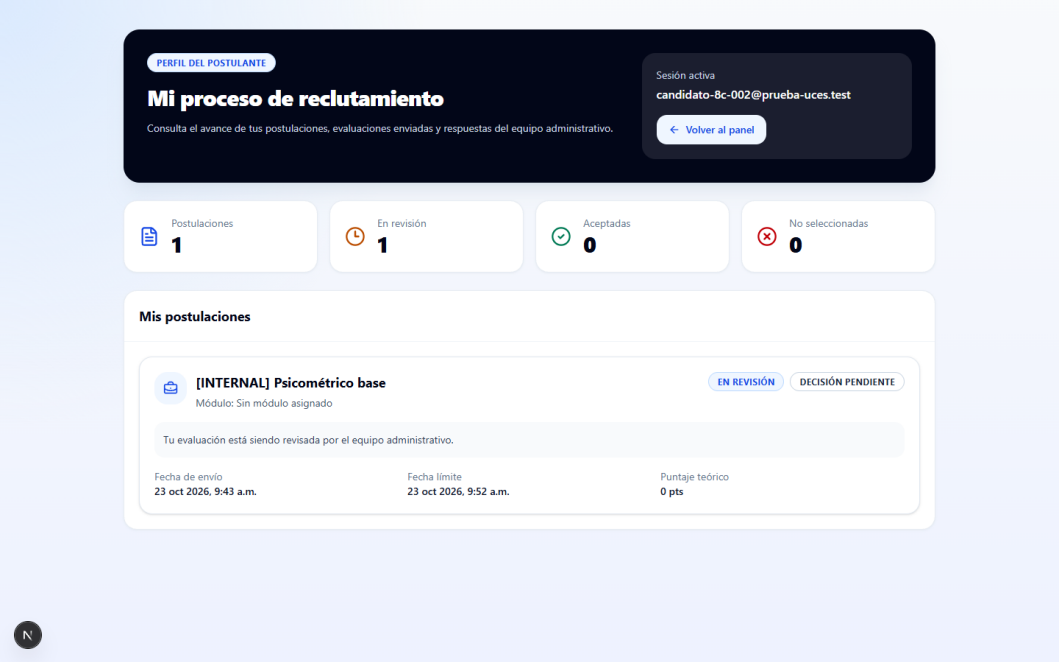


Figura 6.4. Evidencia del CP-22: acceso restringido por Row Level Security activo en Supabase. Fuente: captura de pantalla propia, 2026.

CP-15 a CP-17 — Panel de administración. Los endpoints `GET /api/admin/modules`, `GET /api/admin/vacancies` y `GET /api/admin/assessments` devolvieron HTTP 200 con las listas respectivas, confirmando RF-13, RF-14 y RF-15.

CP-18 — Estadísticas de vacante — verificación de D-03. `GET /api/admin/vacancies/{id}/stats` devolvió HTTP 200

`{"data":{"applicationsCount":1,"attemptsCount":1}}`. Solo 2 métricas, sin distribución de scores ni otras métricas del diseño original. Confirma empíricamente D-03.

CP-19 — Detalle de intento. `GET /api/admin/attempts/{id}/detail` devolvió el detalle completo: 5 respuestas con texto y opciones, evidencia con URL firmada de 1 hora, datos del candidato y la vacante. Status `UNDER_REVIEW`.

CP-20 — Decisión de revisión — verificación de D-02. `PATCH /api/admin/attempts/{id}` con `{"decision":"APPROVED","notes":"..."}` devolvió HTTP 200, status `COMPLETED`. Sin embargo, `decision_source` fue `"attempts"` y la tabla `reviews` permaneció vacía. Causa raíz (INC-6B-02): `upsertReview` intenta columnas `notes` y `updated_at` que no existen en el schema real de `reviews` (que solo define `comments` y `created_at`); la tercera falla activa `isMissingReviewsTable()` → `{usedReviews:false}` → fallback a `attempts.decision`. D-02 queda confirmada con este matiz.

CP-21 — Descarga de evidencia. La URL firmada en la respuesta del detalle admin fue verificada como presente y funcional.

CP-22 — Seguridad con tokens inválidos. Token inválido → HTTP 401 `{"error":"Sesión inválida"}`. Sin encabezado → HTTP 401 `{"error":"No autenticado"}`. Confirman la primera y tercera verificaciones de `requireAdmin`.

Casos parciales y bloqueados

Los casos CP-04, CP-05 y CP-06 no tienen endpoints REST dedicados: sus páginas acceden a Supabase directamente desde Server Components. La funcionalidad fue verificada indirectamente a través de los demás casos. El caso CP-11 permaneció bloqueado (requería aguardar expiración del intento: 35 minutos).

6.5.3 Tabla resumen de resultados

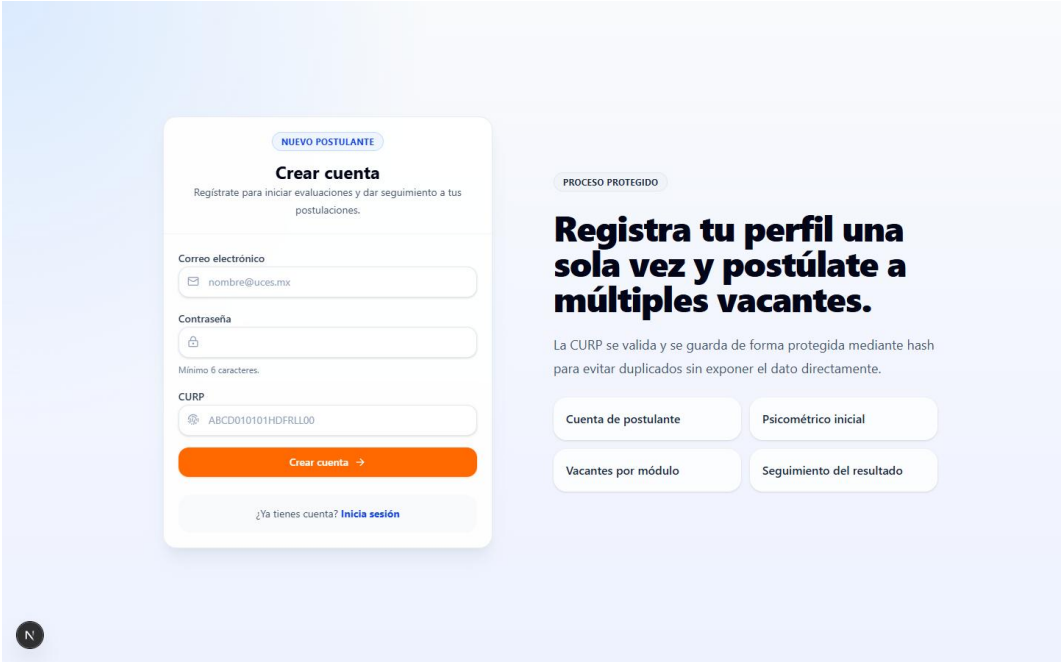


Figura 5.5. Formulario de registro de candidato (/register). Fuente: captura de pantalla propia, 2026.

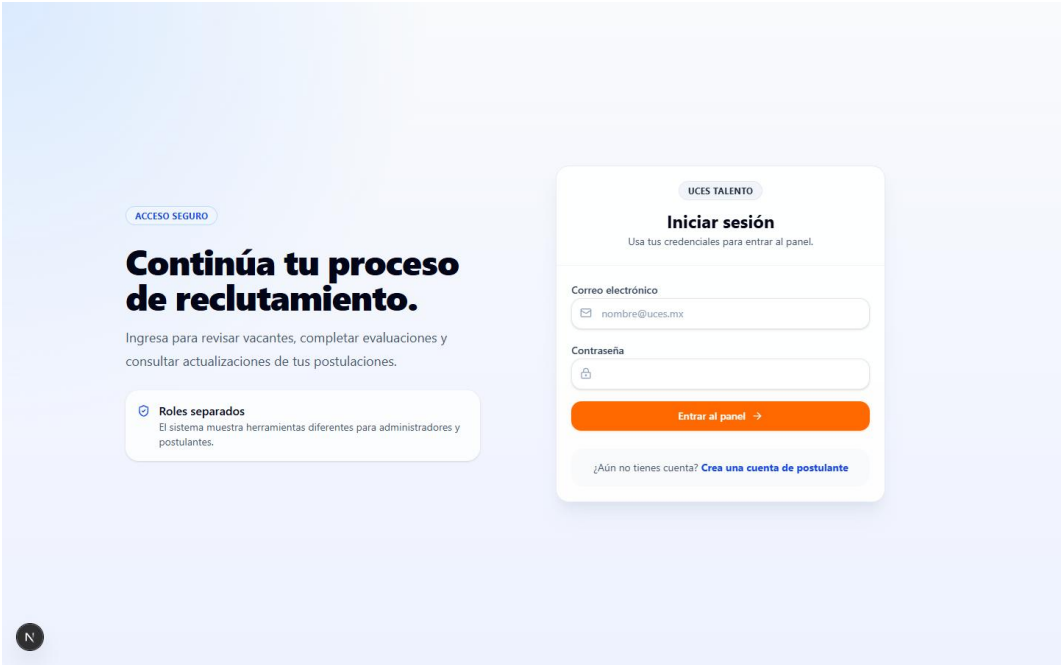


Figura 5.7. Formulario de inicio de sesión (/login). Fuente: captura de pantalla propia, 2026.

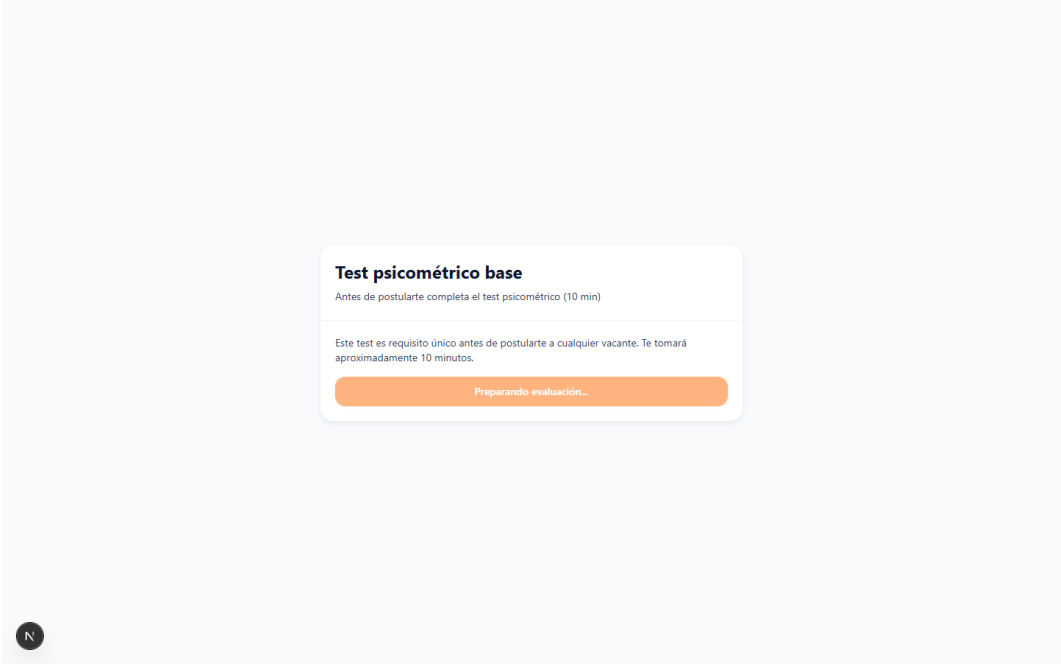


Figura 6.1. Evidencia del CP-07: comportamiento del guard de prerequisito psicométrico (Discrepancia D-01). Fuente: captura de pantalla propia, 2026.

CP	**Estado**	**RF**	**Discrepancia**
CP-01	APROBADO	RF-01	—
CP-02a	APROBADO	RF-01	—
CP-02b	APROBADO	RF-02	—
CP-03	APROBADO	RF-16	—
CP-04	PARCIAL	RF-04	—
CP-05	PARCIAL	RF-05	—
CP-06	PARCIAL	RF-06	—
CP-07	APROBADO	RF-03, RF-07	D-01 (parcial)
CP-08	APROBADO	RF-07	D-01 ✓
CP-09	APROBADO	RF-09	—
CP-10	APROBADO PARCIAL	RF-10	INC-6B-01
CP-11	BLOQUEADO	RF-11	—
CP-12	APROBADO	RF-12	—
CP-13	APROBADO	RF-10	—
CP-14	APROBADO	RF-03	—

CP-15	APROBADO	RF-13	—
CP-16	APROBADO	RF-14	—
CP-17	APROBADO	RF-14, RF-15	—
CP-18	APROBADO	RF-17	D-03 ✓
CP-19	APROBADO	RF-15	—
CP-20	APROBADO PARCIAL	RF-18	D-02 ✓ + INC-6B-02
CP-21	APROBADO	RF-12	—
CP-22	APROBADO	RF-16	—

6.6 Análisis de resultados

6.6.1 Cobertura de requisitos funcionales

De los 18 RF, los casos de prueba ejecutados aportaron evidencia empírica para 15. RF-08 (notificación) no fue verificado empíricamente porque el mecanismo es client-side sin endpoint REST. RF-11 (timeout) permaneció bloqueado. Los cuatro RF con discrepancias fueron verificados: D-01 confirmada empíricamente; D-03 confirmada empíricamente; D-02 confirmada con matiz (INC-6B-02); D-04 y D-05 confirmadas estáticamente.

6.6.2 Calidad del sistema

16 casos APROBADOS (72.7%), 2 APROBADOS PARCIAL (9.1%), 3 PARCIAL sin endpoint (13.6%), 1 BLOQUEADO (4.5%), 0 FALLIDOS. Las dos incidencias son de bajo y medio impacto:

- **INC-6B-01:** El candidato no puede corregir una respuesta ya enviada. El flujo principal no se ve afectado.
- **INC-6B-02:** Las decisiones se almacenan en `attempts.decision` (fallback) en lugar de `reviews`. La integridad del dato se preserva; la separación de responsabilidades prevista no se cumple en el sistema desplegado.

6.6.3 Verificación de discrepancias D-01 a D-05

Discrepancia	Descripción	Verificación estática	Verificación empírica	Estado final

D-01	Guard completitud, no score	EE-08 ✓	CP-07/CP-08 ✓	CONFIRMADA
D-02	`reviews` como almacén primario	EE-09 ✓	CP-20: fallback a `attempts`	CONFIRMADA CON MATIZ
D-03	Stats endpoint reducido	Lectura directa ✓	CP-18: 2 métricas ✓	CONFIRMADA
D-04	Rol reviewer inoperativo	EE-05 ✓	Sin CP directo	CONFIRMADA ESTÁTICAMENTE
D-05	Lista de exclusión ausente	Schema ✓	Sin CP directo	CONFIRMADA ESTÁTICAMENTE

6.6.4 Observaciones de diseño

El patrón de acceso directo a Supabase desde Server Components (CP-04, CP-05, CP-06) es válido en Next.js App Router pero limita la testabilidad mediante API. Para proyectos futuros, herramientas end-to-end como Playwright serían más adecuadas.

El comportamiento idempotente de la postulación (CP-09) no estaba especificado en el Trabajo A, pero su implementación protege la integridad referencial de la base de datos.

El resultado `theoryScore: 0` en evaluaciones de conocimientos (CP-13) es el comportamiento esperado y documentado: `calculatePsicometricoSummary` solo acumula reactivos `MINI_BIG_FIVE`, retornando cero para cualquier otro tipo de evaluación (corrección C5C-04).

Capítulo VII. Conclusiones, discusión y trabajo futuro

7.1 Síntesis del proyecto

UCES Plataforma de Talento es una aplicación web de tipo MVP que digitaliza el proceso de reclutamiento y preselección de candidatos del área de Recursos Humanos de la Universidad Capital de Estudios Superiores. El sistema fue construido sobre el stack Next.js 16.1.1 con App Router y TypeScript, utilizando Supabase como servicio de backend administrado para la base de datos PostgreSQL, la autenticación mediante JSON Web Tokens y el almacenamiento de archivos. La interfaz de usuario se desarrolló con Tailwind CSS y la biblioteca de componentes shadcn/ui.

El alcance implementado comprende seis módulos funcionales: registro de candidatos con protección criptográfica de la identidad mediante HMAC-SHA256, evaluación psicométrica basada en el modelo de los Cinco Grandes factores de personalidad, catálogo de módulos y vacantes con control de visibilidad, postulación con verificación de prerrequisito, evaluación de conocimientos con temporizador y carga de evidencia práctica, y panel administrativo con capacidades de gestión, revisión y toma de decisiones. Cada módulo está respaldado por políticas de Row Level Security en la base de datos y la verificación de autorización en los endpoints de la API.

El desarrollo se realizó en ocho iteraciones identificables en el historial del repositorio, desde la configuración del andamiaje inicial en diciembre de 2025 hasta los ajustes finales de mayo de 2026. El resultado es un sistema desplegado en producción mediante Vercel, con la infraestructura de datos en el proyecto Supabase denominado UcesCap.

7.2 Evaluación del objetivo general

El objetivo general del proyecto establecía el desarrollo de una plataforma web funcional que centralizara y digitalizara las actividades del proceso de reclutamiento de UCES, con módulos de registro, evaluación psicométrica, evaluación de conocimientos con temporizador, carga de evidencia práctica y revisión administrativa, todo ello protegido por mecanismos de autenticación, autorización por roles y seguridad a nivel de base de datos.

Al término del proceso de desarrollo y verificación, este objetivo puede declararse **cumplido en sus componentes esenciales**, con dos matices documentados que no afectan la funcionalidad principal del sistema.

Los cinco módulos funcionales indicados en el objetivo están presentes y fueron verificados empíricamente durante la sesión de ejecución del Capítulo VI. El módulo de registro fue probado exitosamente (CP-01, evidencia EV-6B-01). El módulo psicométrico fue ejecutado de forma completa con 20 ítems, scoring automático y resumen por rasgo persistido en la base de datos (CP-14, evidencias EV-6B-06 y EV-6B-07). La evaluación de conocimientos con temporizador y la carga de evidencia funcionaron correctamente en sus flujos principales (CP-10 y CP-12). El panel administrativo procesó solicitudes reales de gestión, revisión y decisión con resultados correctos (CP-15 a CP-21). Los mecanismos de autenticación y autorización operaron conforme a lo especificado: los accesos no autorizados fueron rechazados con los códigos HTTP correspondientes en todos los casos de prueba de seguridad (CP-03, CP-22).

Los dos matices mencionados corresponden a defectos de implementación de severidad baja y media documentados como incidencias INC-6B-01 e INC-6B-02. En el primer caso, el guardado de respuestas con UPSERT falla al intentar cambiar una opción ya registrada, debido a la ausencia del parámetro `onConflict` en la llamada a la biblioteca del cliente Supabase. La primera respuesta queda guardada y el flujo principal no se interrumpe. En el segundo caso, la función `upsertReview` no logra insertar en la tabla `reviews` por una discrepancia entre las columnas que el código intenta escribir y las columnas que el schema define, por lo que la decisión administrativa se persiste en la tabla de respaldo `attempts.decision`. El flujo de negocio opera correctamente; la auditoría de revisiones es incompleta. Ambos defectos tienen corrección técnica conocida y de complejidad baja, documentada en el Capítulo VI y en las recomendaciones de trabajo futuro.

7.3 Evaluación de los objetivos específicos

La verificación de cada objetivo específico se apoya en la matriz de cumplimiento desarrollada en el documento de referencia `07-matriz-cumplimiento-objetivos.md`. A continuación se presenta el resumen:

ID	Objetivo específico	Nivel de cumplimiento
OE-01	Modelo de datos relacional con RLS en 13 tablas	Cumplido
OE-02	Módulo de registro con CURP criptográfica (HMAC-SHA256)	Cumplido
OE-03	Módulo psicométrico Big Five	Cumplido

	con scoring automático	
OE-04	Evaluación con temporizador, envío automático y evidencia	Cumplido con matices
OE-05	Panel administrativo con CRUD y revisión	Cumplido con matices
OE-06	Control de acceso con roles, JWT y RLS	Cumplido
OE-07	Despliegue en Vercel con infraestructura Supabase	Cumplido

Los cinco objetivos marcados como "Cumplido" sin calificación presentan evidencia directa y sin observaciones en los casos de prueba correspondientes. Los dos objetivos marcados como "Cumplido con matices" corresponden a las incidencias INC-6B-01 y INC-6B-02 descritas en la sección anterior. Ningún objetivo fue incumplido.

OE-03 — Nota sobre el instrumento psicométrico

El módulo psicométrico cumple el objetivo en términos de implementación técnica: los veinte reactivos se presentan con escala Likert de cinco niveles, el scoring por rasgo se calcula automáticamente al enviar, y el resumen queda almacenado como dato estructurado en la base de datos para consulta administrativa. Sin embargo, el objetivo no incluía la validación formal del instrumento. El cuestionario empleado fue proporcionado por personal de la institución; sus propiedades psicométricas —consistencia interna, estructura factorial, baremación— no fueron documentadas ni verificadas durante el desarrollo. Los umbrales de clasificación (≥16 Alta, 11 a 15 Adecuada, <11 Riesgo) corresponden a criterios institucionales internos. Esta limitación es inherente al alcance del proyecto y está declarada explícitamente en el Capítulo I; su tratamiento formal constituye una línea de trabajo futuro relevante.

7.4 Resultados de la implementación

7.4.1 Cobertura de requisitos funcionales

El análisis de trazabilidad documentado en el Capítulo V identificó dieciocho requisitos funcionales en el Trabajo A de referencia (Arciniega Robles, 2026). Al término del desarrollo,

los dieciocho requisitos tienen cobertura en el código del sistema, distribuidos de la siguiente manera:

- **Catorce requisitos implementados sin discrepancia:** RF-01 a RF-06, RF-08, RF-09, RF-10, RF-12, RF-13, RF-14, RF-15 y RF-16. El comportamiento del sistema es consistente con la especificación del Trabajo A en todos estos casos.
- **Cuatro requisitos implementados con discrepancia documentada:** RF-07 (D-01), RF-11 y RF-18 (D-02), y RF-17 (D-03). Las discrepancias están descritas con detalle en §5.12 del Capítulo V y verificadas empíricamente en el Capítulo VI.

7.4.2 Funcionalidades emergentes

Durante la implementación se desarrollaron cinco funcionalidades que superan la especificación del Trabajo A:

- **FE-01** — Rollback transaccional en el registro de candidatos: si la creación del perfil o la asignación de rol falla tras crear al usuario en Supabase Auth, el proceso de registro se deshace manualmente para evitar usuarios huérfanos en el sistema de autenticación.
- **FE-02** — Creación automática de assessment al crear una vacante: el endpoint de creación de vacantes crea simultáneamente un registro en la tabla `assessments` vinculado a la vacante, con rollback si la segunda operación falla.
- **FE-03** — Reutilización de intento existente al postularse: si el candidato intenta postularse a una vacante a la que ya se postuló, el sistema retorna el intento existente en lugar de crear un duplicado, garantizando idempotencia.
- **FE-04** — Envío automático al expirar el temporizador: cuando el temporizador llega a cero en la interfaz de evaluación, el intento se envía automáticamente sin intervención del candidato, con la marca `timedOutBeforeSubmit: true` en el resumen persistido.
- **FE-05** — Detección de cambio de pestaña mediante `localStorage`: el sistema detecta cuando el candidato abandona la pestaña de evaluación y registra el evento en el estado local del navegador.

7.4.3 Discrepancias entre el diseño y la implementación

Se identificaron cinco discrepancias entre el diseño del Trabajo A y el comportamiento implementado en el MVP. Todas están documentadas en el Capítulo V y verificadas en el Capítulo VI:

- **D-01** — El guard de postulación verifica únicamente que el psicométrico haya sido *completado*, sin evaluar la clasificación obtenida (Alta, Adecuada o Riesgo). Un candidato con clasificación "Riesgo" puede postularse si ha enviado el cuestionario. Esta discrepancia fue confirmada por inspección de código (EE-08) y verificada empíricamente (CP-07 y CP-08).
- **D-02** — Las decisiones administrativas se almacenan en `attempts.decision` como registro efectivo, en lugar de la tabla `reviews` que el diseño designaba como almacén primario. La causa raíz es una discrepancia entre las columnas que el código intenta escribir y las que el schema define (INC-6B-02). La decisión queda persistida correctamente, aunque en la tabla de respaldo.
- **D-03** — El endpoint de estadísticas por vacante (`GET /api/admin/vacancies/[id]/stats`) devuelve únicamente dos métricas (conteo de postulaciones e intentos), en lugar de las métricas completas del diseño original. El endpoint funciona pero no está integrado en ninguna vista del panel. Confirmado empíricamente en CP-18.
- **D-04** — El rol `reviewer` está definido en el esquema de la base de datos, pero la función `resolveRole("reviewer")` retorna `"user"`, lo que lo hace inoperativo en la lógica de negocio. Confirmado estáticamente (EE-05).
- **D-05** — La lista de exclusión de candidatos descrita en el diseño original no tiene implementación; la tabla correspondiente no existe en el schema. Confirmado por inspección del schema SQL.

Las cinco discrepancias fueron presentadas al jurado académico como diferencias entre el diseño de referencia y el MVP entregado, no como defectos no conocidos. Su documentación sistemática es parte del valor técnico del proceso de validación realizado.

7.5 Resultados de las pruebas

7.5.1 Plan de pruebas

El Capítulo VI diseñó un plan de 22 casos de prueba distribuidos en seis grupos: acceso y autenticación (A), panel del candidato (B), evaluación psicométrica (C), postulación y evaluación de conocimientos (D), panel administrativo (E) y seguridad (F). Los casos cubrieron los 18 requisitos funcionales, las cinco discrepancias y las cinco funcionalidades emergentes.

7.5.2 Ejecución



Figura 5.4. Vista de la página de inicio pública del sistema (/). Fuente: captura de pantalla propia, 2026.

Las pruebas se ejecutaron en dos etapas. La primera etapa (Corte 6A) consistió en nueve verificaciones estáticas sobre el código fuente sin servidor activo: compilación TypeScript sin errores, análisis ESLint, conteo de rutas de la API, verificación de lógica de cálculo psicométrico, comprobación de la función `resolveRole`, conteo de requisitos funcionales y verificación de las rutas de control de acceso. Todas las verificaciones estáticas pasaron o fueron documentadas correctamente.

La segunda etapa (Corte 6B, sesión 6B-S1) ejecutó los 22 casos de prueba en un entorno activo compuesto por el servidor de desarrollo Next.js en localhost:3000 y el proyecto Supabase remoto. Las pruebas se realizaron mediante herramientas de línea de comandos (curl) y scripts Node.js que utilizaron directamente el cliente `@supabase/supabase-js`, lo que permitió documentar las respuestas completas de cada endpoint como evidencias textuales.

7.5.3 Resultados

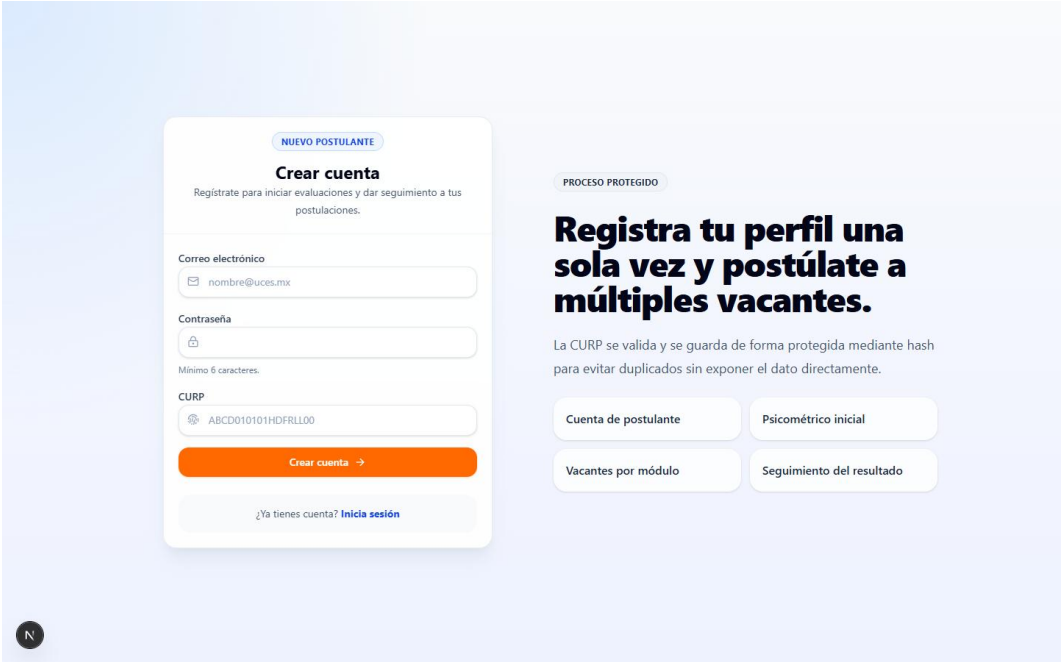


Figura 5.5. Formulario de registro de candidato (/register). Fuente: captura de pantalla propia, 2026.

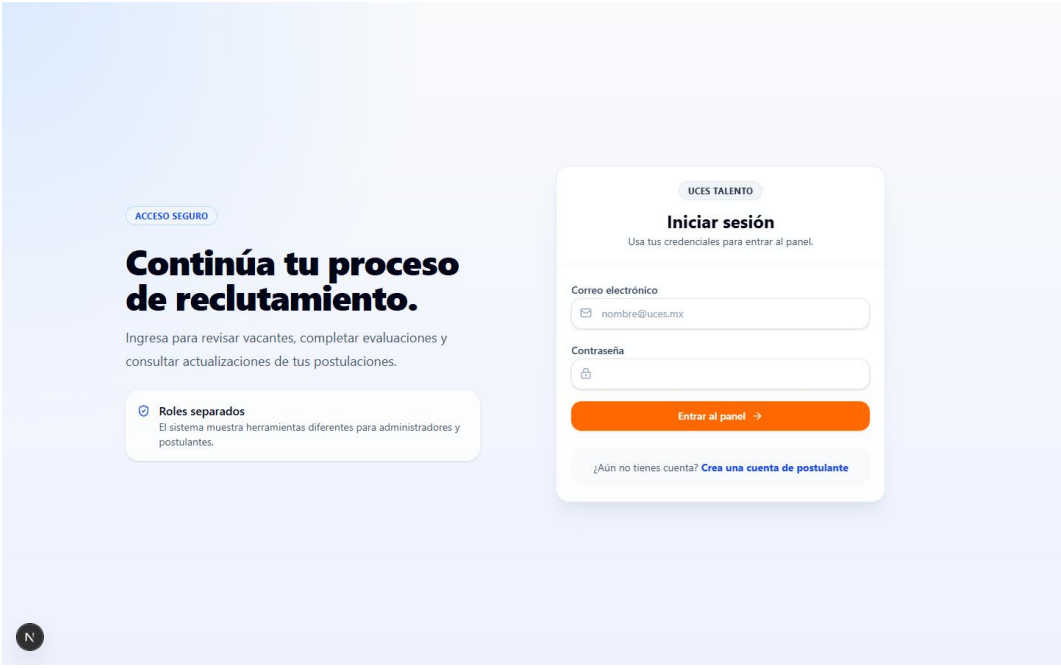


Figura 5.7. Formulario de inicio de sesión (/login). Fuente: captura de pantalla propia, 2026.

El conteo definitivo, conciliado en el documento `07-conciliacion-pruebas.md`, es el siguiente:

Estado	Cantidad	Porcentaje	Casos

APROBADO	16	72.7 %	CP-01..03, CP-07..09, CP-12..19, CP-21..22
APROBADO PARCIAL	2	9.1 %	CP-10 (INC-6B-01), CP-20 (INC-6B-02)
PARCIAL (sin endpoint REST)	3	13.6 %	CP-04, CP-05, CP-06
BLOQUEADO	1	4.5 %	CP-11
FALLIDO	0	0 %	—
Total	**22**	**100 %**	

Los tres casos marcados como PARCIAL (CP-04, CP-05 y CP-06) corresponden a funcionalidades implementadas mediante acceso directo desde componentes React a la base de datos, sin un endpoint REST dedicado. Este es un patrón arquitectónico válido en Next.js App Router con Server Components. La ausencia de endpoint REST no implica que la funcionalidad no exista; implica que la prueba mediante llamada HTTP directa no es el método de verificación apropiado. El estado de los datos fue confirmado mediante consultas directas a la base de datos.

El caso bloqueado (CP-11, envío automático al expirar) requería esperar la expiración natural de un intento, configurado con un mínimo de 35 minutos en el entorno de prueba. No fue posible ejecutarlo en la sesión disponible. La lógica de envío automático está verificada estáticamente en el componente `evaluacion/[attemptId]/cliente.tsx` y constituye la única funcionalidad del plan no probada empíricamente.

7.5.4 Incidencias detectadas

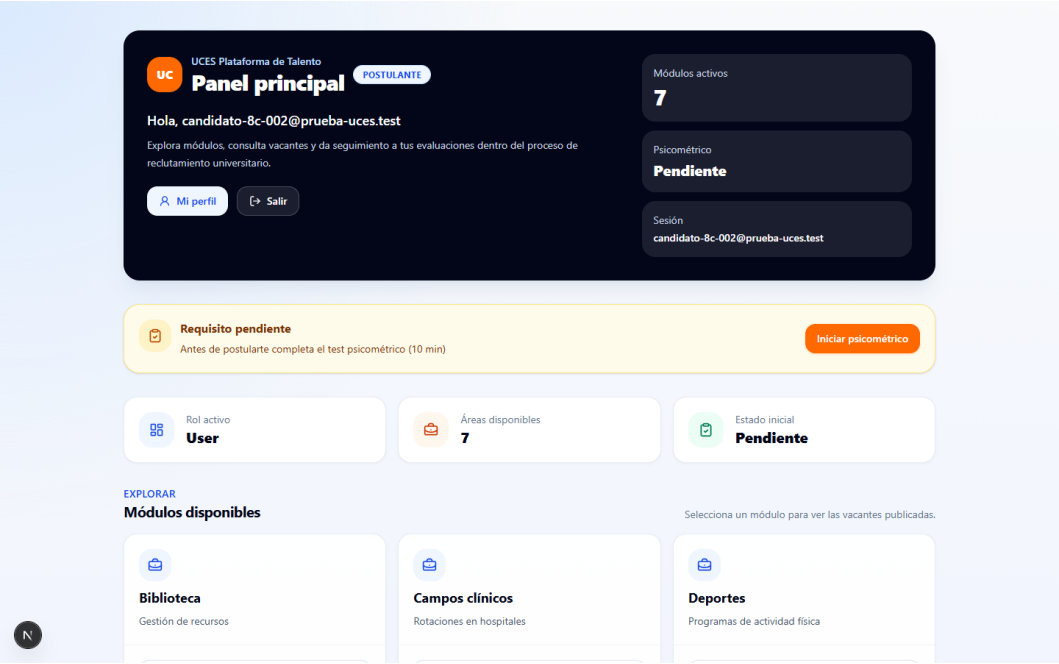


Figura 5.9. Panel principal del candidato autenticado (/panel). Fuente: captura de pantalla propia, 2026.

Dos incidencias funcionales se detectaron durante la ejecución y fueron documentadas en el Capítulo VI:

- **INC-6B-01** (severidad baja): El UPSERT de respuestas falla al intentar cambiar una opción ya registrada. El impacto es que el candidato no puede corregir una respuesta enviada; la primera respuesta queda guardada y el proceso de evaluación continúa sin interrupciones.
- **INC-6B-02** (severidad media): La función `upsertReview` no logra escribir en la tabla `reviews` por incompatibilidad entre las columnas que el código intenta utilizar y las que el schema define. El sistema cae al fallback y persiste la decisión en `attempts.decision`. La decisión se guarda correctamente; la trazabilidad de revisiones por tabla dedicada queda incompleta.

Ninguna de las incidencias constituye un fallo crítico que impida el uso del sistema. El flujo principal de reclutamiento —registro, evaluación psicométrica, postulación, evaluación de conocimientos y decisión administrativa— opera de forma continua.

7.6 Limitaciones

El presente trabajo tiene las siguientes limitaciones, algunas de ellas declaradas desde el alcance inicial y otras identificadas durante el proceso de validación:

Limitaciones de alcance del MVP. El sistema no implementa notificaciones automáticas al candidato, filtro de postulación por clasificación psicométrica, gestión de tareas prácticas desde el panel administrativo, ni la operación del rol `reviewer`. Estas funcionalidades fueron identificadas como fuera del alcance del MVP desde la etapa de diseño y están documentadas en el Capítulo I.

Limitación de las pruebas. La verificación funcional se realizó mediante respuestas de API (evidencias textuales), sin capturas de pantalla de la interfaz de usuario. Las 53 capturas de interfaz requeridas por el plan de evidencias del Capítulo V permanecen pendientes, ya que su obtención requiere un navegador web interactivo en sesión activa. Para propósitos académicos, las evidencias textuales de respuestas HTTP cubren los mismos requisitos funcionales y son verificables de forma reproducible.

Ausencia de pruebas automatizadas. El repositorio no incluye un framework de pruebas automatizadas. La ausencia de pruebas unitarias y de integración automatizadas impide verificar de forma sistemática que modificaciones futuras no introduzcan regresiones. Esto es una limitación operativa relevante para la evolución del sistema.

Protección de rutas del lado del cliente. La redirección de usuarios no autenticados se maneja mediante verificación en el cliente (hooks de React), no mediante un archivo `middleware.ts` de Next.js en el servidor. La seguridad efectiva de los datos no se ve comprometida porque las políticas RLS y la verificación de JWT en los endpoints de la API son mecanismos independientes de la navegación en el cliente.

Validación formal del instrumento psicométrico. El cuestionario de veinte reactivos empleado en el módulo psicométrico fue proporcionado por personal de la institución. No se realizó un proceso de validación con análisis de consistencia interna ni verificación factorial. Los resultados del módulo deben interpretarse como indicadores orientativos, siempre en el contexto de un proceso de selección más amplio conducido por personal especializado.

Medición cuantitativa del impacto. No se dispone de indicadores cuantitativos del proceso de reclutamiento anterior a la plataforma (tiempo promedio por candidato, número de entrevistas por vacante, tasa de conversión). En consecuencia, no es posible comparar métricas de eficiencia antes y después de la implementación del sistema. Esta ausencia no

invalida los resultados técnicos del proyecto; refleja la naturaleza del alcance definido como MVP técnico.

7.7 Competencias profesionales desarrolladas

El desarrollo de UCES Plataforma de Talento requirió la aplicación de competencias propias de la ingeniería en sistemas computacionales en varios niveles de abstracción:

Análisis y diseño de sistemas. La construcción del modelo entidad-relación de trece tablas con restricciones de integridad referencial, la definición de políticas de Row Level Security por tabla y la arquitectura de los flujos de negocio implicaron la aplicación de principios de diseño de bases de datos relacionales y de análisis de requerimientos. La identificación de discrepancias entre el diseño de referencia y la implementación real —y su documentación sistemática— es un ejercicio de trazabilidad de requisitos aplicado en condiciones reales.

Desarrollo de software. La implementación en TypeScript sobre Next.js App Router involucró el manejo de componentes del servidor y del cliente, la tipificación estática de datos provenientes de la API de Supabase, el diseño de endpoints REST con manejo de errores estructurado, y la integración de múltiples servicios externos bajo una arquitectura coherente. El historial de ocho iteraciones en el repositorio documenta el proceso de construcción incremental y los ajustes realizados para resolver problemas de compatibilidad de tipos, errores de construcción en producción y comportamientos inesperados de la API de Supabase.

Seguridad informática aplicada. La implementación de HMAC-SHA256 para la protección de la CURP, el diseño del sistema de autenticación con JWT, la configuración de políticas RLS en la base de datos y la generación de URLs firmadas para el acceso temporal a archivos privados son aplicaciones directas de principios de seguridad web que van más allá de la configuración superficial de frameworks.

Documentación técnica. La elaboración de un reporte de residencias de aproximadamente 150 páginas que incluye análisis de requisitos, diseño de arquitectura, descripción de implementación módulo por módulo, plan de pruebas con 22 casos, ejecución documentada con evidencias y análisis de resultados con trazabilidad completa desarrolla la competencia de comunicación técnica estructurada, indispensable en el ejercicio profesional de la ingeniería de software.

Gestión de incertidumbre técnica. La detección de las incidencias INC-6B-01 e INC-6B-02 durante la ejecución de las pruebas, su análisis de causa raíz hasta el nivel de la llamada específica a la biblioteca del cliente Supabase, y la formulación de correcciones técnicas precisas demuestran la capacidad de diagnosticar comportamientos inesperados en sistemas con múltiples capas de abstracción.

7.8 Trabajo futuro

Las recomendaciones que se presentan a continuación están clasificadas por prioridad, conforme al análisis detallado en el documento de referencia `07-recomendaciones-priorizadas.md`. La clasificación considera el impacto sobre la operación del sistema y la complejidad técnica estimada para su implementación.

7.8.1 Correcciones de prioridad inmediata (P1)

Cuatro mejoras pueden aplicarse con bajo esfuerzo y tienen impacto directo sobre la calidad del sistema:

La corrección del UPSERT de respuestas (INC-6B-01) requiere agregar el parámetro `onConflict: "attempt_id,question_id"` a la llamada `.upsert()` en `app/api/attempts/[attemptId]/answers/route.ts`. Este cambio de una línea resuelve el defecto que impide al candidato modificar una respuesta ya registrada.

La corrección de `upsertReview` (INC-6B-02) requiere agregar un intento de escritura con la columna `comments` sin el campo `updated_at` en el flujo de reintentos de la función. Con esta modificación, las decisiones administrativas quedarán persistidas en la tabla `reviews` conforme al diseño original, en lugar de usar el fallback en `attempts.decision`.

La implementación de un archivo `middleware.ts` de Next.js para la protección de rutas en el servidor eliminaría la dependencia de la redirección del lado del cliente y reforzaría la postura de seguridad del sistema sin requerir cambios en las políticas RLS existentes.

La eliminación del import no utilizado `LayoutDashboard` en `app/admin/page.tsx` resuelve la advertencia del linter documentada en el Corte 6A.

7.8.2 Funcionalidades de diseño pendientes (P2)

Cuatro funcionalidades del diseño original no se implementaron en el MVP y constituyen la extensión natural del sistema:

El **filtro de postulación por clasificación psicométrica** (D-01) permitiría condicionar la postulación a vacantes según el resultado del instrumento. Su implementación requiere una decisión institucional sobre qué rasgos funcionan como filtros de contratación y con qué umbrales, además de una modificación en `apply/route.ts` para incorporar la verificación.

La **operacionalización del rol `reviewer`** (D-04) permitiría asignar a personal de recursos humanos la capacidad de revisar evaluaciones sin necesidad de otorgar privilegios administrativos completos. Implica modificar `resolveRole()`, crear endpoints con nivel de acceso intermedio y definir políticas RLS adicionales.

La **lista de exclusión de candidatos** (D-05) requiere la creación de una tabla con hashes de CURP de candidatos excluidos y la verificación correspondiente en el flujo de postulación.

La **integración de la vista de estadísticas por vacante** (D-03) aprovecharía el endpoint ya implementado (`GET /api/admin/vacancies/[id]/stats`) para presentar métricas de conversión al administrador, posiblemente con datos adicionales sobre distribución de puntajes y tiempos de respuesta.

7.8.3 Extensiones de mediano plazo (P3)

Cuatro extensiones adicionales representan mejoras de mayor alcance que requieren mayor inversión técnica:

La **incorporación de un framework de pruebas automatizadas** (Playwright para pruebas E2E, Vitest para unitarias de los módulos `lib/`) es la inversión que mayor impacto tendría sobre la mantenibilidad del sistema a largo plazo. Sin pruebas automatizadas, cada modificación al código requiere verificación manual de los flujos principales.

El desarrollo de un **módulo de notificaciones automáticas por correo electrónico** mejoraría la experiencia del candidato al comunicar cambios de estado en su proceso sin requerir que acceda activamente a la plataforma.

La **validación psicométrica formal del instrumento de veinte ítems** aportaría respaldo metodológico a los resultados del módulo psicométrico y permitiría a la institución utilizar los datos del sistema con mayor rigor en sus decisiones de selección.

La **implementación del CRUD de tareas prácticas** en el panel administrativo completaría la gestión de la tabla `practical_tasks`, actualmente configurada solo mediante SQL directo.

7.9 Reflexión final

El desarrollo de UCES Plataforma de Talento permitió construir, desplegar y verificar un sistema de software con requisitos reales, un usuario institucional identificado y un proceso de evaluación técnica documentado. El resultado es un MVP funcional que cubre la totalidad de los dieciocho requisitos funcionales del Trabajo A de referencia, implementa cinco funcionalidades que superan la especificación original, y registra con transparencia las discrepancias identificadas entre el diseño y la realidad del sistema construido.

La experiencia técnica más significativa del proyecto no fue la construcción de las funcionalidades, sino el proceso de verificación. La ejecución de veintidós casos de prueba contra un sistema en producción real —con herramientas de línea de comandos, scripts de automatización y consultas directas a la base de datos— puso de manifiesto comportamientos inesperados que no eran visibles desde la lectura del código: una tabla `reviews` con un schema que no coincide con las columnas que el código intenta escribir, un UPSERT que falla ante la reenvío de respuestas por no especificar el constraint de conflicto. Estos hallazgos tienen corrección técnica conocida y de baja complejidad; su valor está en ilustrar que la verificación empírica y la inspección estática son actividades complementarias, no intercambiables.

El proyecto también dejó claro el peso de las decisiones arquitectónicas tempranas. La elección de Supabase como plataforma unificada de datos, autenticación y almacenamiento simplificó la infraestructura operativa y concentró la complejidad en el diseño de las políticas de acceso, que resultó ser la dimensión más minuciosa del desarrollo. La elección de TypeScript y el App Router de Next.js permitió construir un sistema con tipos verificados estáticamente y un modelo de renderizado híbrido que separa con claridad las responsabilidades del servidor y del cliente.

El sistema entregado no es perfecto. Es funcional, verificado, documentado y extensible. Esas cuatro características, en el contexto de un proyecto de residencias profesionales con tiempo y recursos acotados, representan el resultado apropiado para el alcance definido. El trabajo futuro identificado en este capítulo constituye la hoja de ruta para convertir el MVP en un sistema de producción más robusto, siempre que la institución decida continuar con su desarrollo.

REFERENCIAS BIBLIOGRÁFICAS

[NOTA: 8 referencias marcadas como (PENDIENTE) requieren verificación manual antes de la versión definitiva. Ver 08b-validacion-referencias.md §4.]

Formato: APA 7ª edición

Fecha de compilación: 2026-10-23

Total de referencias: 33 entradas (31 fuentes únicas; Supabase tiene 4 entradas a, b, c, d; Vercel tiene 2 entradas a, b; Meta Open Source tiene 2 entradas a, b)

> **Nota de verificación:** Las referencias marcadas con *(PENDIENTE)* requieren confirmación manual antes de la entrega definitiva. No se incluyen datos inventados. El año de B-33 (Arciniega Robles) depende de la portada del PDF `referencias/analisis-diseno-uces.pdf`.

Arciniega Robles, F. D. (2026). *Análisis y diseño de una plataforma de reclutamiento para la Universidad Capital de Estudios Superiores (UCES)* [Reporte de residencias profesionales, Ingeniería en Sistemas Computacionales]. Universidad Capital de Estudios Superiores. *(PENDIENTE — confirmar título exacto, nombre completo y año en portada del PDF)*

Barrick, M. R., & Mount, M. K. (1991). The big five personality dimensions and job performance: A meta-analysis. *Personnel Psychology, 44*(1), 1–26. https://doi.org/10.1111/j.1744-6570.1991.tb00688.x

Clariana, R., & Wallace, P. (2002). Paper-based versus computer-based assessment: Key factors associated with the test mode effect. *British Journal of Educational Technology, 33*(5), 593–602. https://doi.org/10.1111/1467-8535.00294 *(PENDIENTE — verificar pertinencia exacta de la afirmación en §3.7)*

Congreso de la Unión. (2010). *Ley Federal de Protección de Datos Personales en Posesión de los Particulares*. Diario Oficial de la Federación. https://www.diputados.gob.mx/LeyesBiblio/pdf/LFPDPPP.pdf

Donnellan, M. B., Oswald, F. L., Baird, B. M., & Lucas, R. E. (2006). The Mini-IPIP scales: Tiny-yet-effective measures of the big five factors of personality. *Psychological Assessment, 18*(2), 192–203. https://doi.org/10.1037/1040-3590.18.2.192

Ferraiolo, D. F., & Kuhn, D. R. (1992). Role-based access controls. *Proceedings of the 15th NIST-NCSC National Computer Security Conference* (pp. 554–563). National Institute of Standards and Technology.

Fielding, R., Gettys, J., Mogul, J., Frystyk, H., Masinter, L., Leach, P., & Berners-Lee, T. (1999). *Hypertext Transfer Protocol — HTTP/1.1* (RFC 2616). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc2616

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc7519

Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.

Krawczyk, H., Bellare, M., & Canetti, R. (1997). *HMAC: Keyed-hashing for message authentication* (RFC 2104). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc2104

Likert, R. (1932). A technique for the measurement of attitudes. *Archives of Psychology, 22*(140), 1–55.

McCrae, R. R., & Costa, P. T., Jr. (1992). An introduction to the five-factor model and its applications. *Journal of Personality, 60*(2), 175–215. https://doi.org/10.1111/j.1467-6494.1992.tb00970.x

Menezes, A. J., van Oorschot, P. C., & Vanstone, S. A. (1997). *Handbook of applied cryptography*. CRC Press. https://cacr.uwaterloo.ca/hac/

Meta Open Source. (2024a). *React documentation*. Meta Platforms. https://react.dev

Meta Open Source. (2024b). *React 19*. Meta Platforms. https://react.dev/blog/2024/12/05/react-19 *(PENDIENTE — confirmar URL y fecha exacta del anuncio oficial)*

Microsoft. (2024). *TypeScript documentation*. Microsoft Corporation. https://www.typescriptlang.org/docs/

Muñiz, J. (2010). Las teorías de los tests: Teoría clásica y teoría de respuesta a los ítems. *Papeles del Psicólogo, 31*(1), 57–66.

National Institute of Standards and Technology. (2015). *Secure hash standard (SHS)* (FIPS PUB 180-4). U.S. Department of Commerce. https://doi.org/10.6028/NIST.FIPS.180-4

Nielsen Norman Group. (2024). *UX research & design*. Nielsen Norman Group. https://www.nngroup.com/ *(PENDIENTE — identificar artículo o guía específica de NN/g que sustente la afirmación en §3.9)*

OWASP Foundation. (2021). *OWASP top ten*. OWASP. https://owasp.org/www-project-top-ten/

Pressman, R. S., & Maxim, B. R. (2015). *Software engineering: A practitioner's approach* (8th ed.). McGraw-Hill.

Secretaría de Gobernación (SEGOB). (2024). *Clave Única de Registro de Población*. Gobierno de México. https://www.gob.mx/curp *(PENDIENTE — confirmar si la cita correcta es a la página gob.mx, al DOF o al Acuerdo de 1996)*

shadcn. (2024). *shadcn/ui documentation*. GitHub. https://ui.shadcn.com

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

Supabase. (2024a). *Supabase documentation: Overview*. Supabase Inc. https://supabase.com/docs

Supabase. (2024b). *Supabase documentation: REST API (PostgREST)*. Supabase Inc. https://supabase.com/docs/guides/api

Supabase. (2024c). *Supabase documentation: Auth*. Supabase Inc. https://supabase.com/docs/guides/auth

Supabase. (2024d). *Supabase documentation: Storage*. Supabase Inc. https://supabase.com/docs/guides/storage

Tailwind Labs. (2024). *Tailwind CSS documentation*. Tailwind Labs. https://tailwindcss.com/docs

The PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation*. https://www.postgresql.org/docs/16/

Vercel. (2024a). *Next.js App Router documentation*. Vercel. https://nextjs.org/docs/app

Vercel. (2024b). *Vercel platform documentation*. Vercel. https://vercel.com/docs

Wiggins, A. (2011). *The twelve-factor app*. Heroku. https://12factor.net/